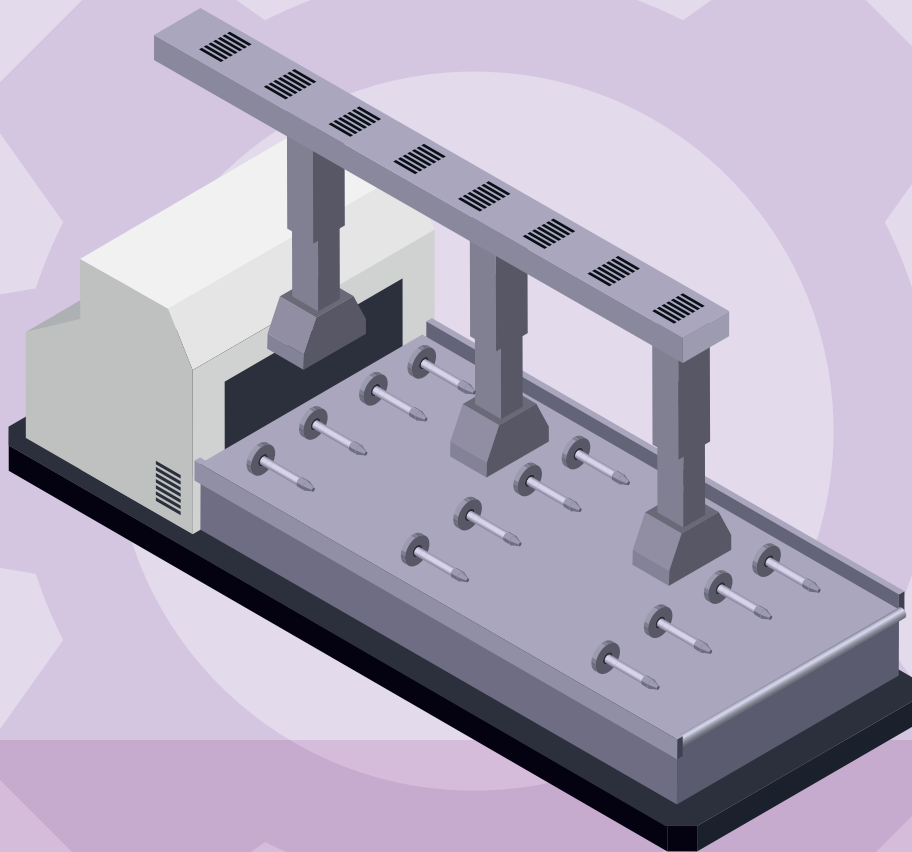
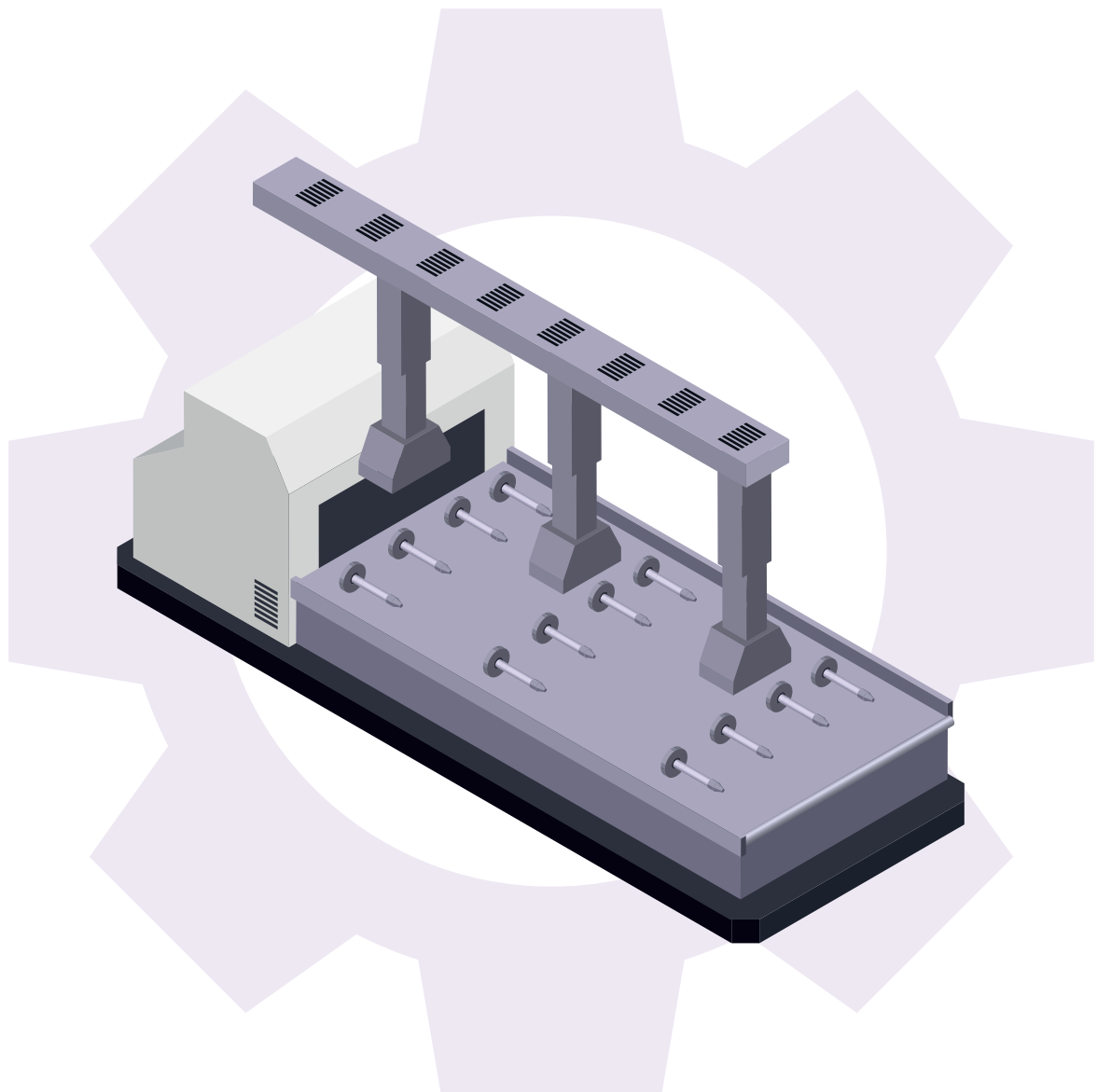


「공정운영 최적화 AI 데이터셋」 분석실습 가이드북



「공정운영 최적화 AI 데이터셋」 분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	06
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	08
2.1 제조데이터 소개	08
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	11
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	22
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 데이터 준비	
[단계 ②] 데이터 탐색	
[단계 ③] 데이터 정제(전처리)	
[단계 ④] 알고리즘 선택	
[단계 ⑤] 학습, 평가 데이터 준비	
[단계 ⑥] 모델링	
[단계 ⑦] 모델 훈련	
[단계 ⑧] 모델 튜닝	
[단계 ⑨] 모델 평가 및 해석	
3. 유사 타현장의 「공정운영 최적화 AI 데이터셋」 분석 적용	46
부 록	
분석환경 구축을 위한 설치 가이드	47
데이터 품질 전처리 [실습 코드]	67

「공정운영 최적화 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda, Jupyter Notebook
- 필요 패키지 : pandas, sklearn, matplotlib, numpy, matplotlib, datetime, seaborn, math, os
- 분석 환경 : [CPU] AMD Ryzen 7 5800X, [Memory] 64GB
- 필요 데이터 : kemp-abh-sensor-2021.*.csv, kemp-process-rate.csv

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		산제 전처리 설비에서 발생하는 데이터셋을 AI를 이용하여 공정 운영 최적화를 위한 분석을 할 수 있다. 데이터 간의 상관관계를 찾고 다양한 AI 알고리즘을 활용한 산제 전처리 설비 공정 운영 최적화 모델을 학습할 수 있다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : Index, LoT, Time, pH, Temp, Process Rate 2) 데이터 수집 방법 : PLC 및 센서를 통해 실시간 데이터를 수신하여 수집된 시간을 기준으로 CSV 파일 형태로 저장한다. 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 252,648개 - 데이터셋 총량 : 2MB
4	분석적용 알고리즘	알고리즘	Decision Tree(의사결정나무)를 사용한다.
		알고리즘 간략소개	Decision Tree는 머신러닝에서 분류 또는 예측 모형으로 사용되는 지도학습 방법론으로써, 중요 변수 중심의 의사 결정 규칙을 Tree 구조로 도표화하여 분석을 수행하며, 목표변수가 범주형인 경우와 연속형인 경우로 나눌 수 있다.
5	분석결과 및 시사점		- 산제 전처리 설비에서 수집된 데이터를 Decision Tree를 통한 데이터 분석으로 공정 운영 최적화 모델을 도출하였다. - 공정 운영 최적화 모델을 통해 해당 공정을 가진 중소 제조기업현장의 실질적인 공정 운영 최적화 및 비용 절감을 기대한다.

2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

- 산제 전처리: 이노징크 전기 도금처리 시 전처리 공정의 첫 번째 단계로 제품 표면의 산도 및 피막을 제거하는 작업이다. 산제 반응을 통해 제품에 존재하는 산화층을 제거하여 새로운 금속 면을 노출시키고, 표면의 활성화를 유도하여 이후 공정들의 밀착성을 향상시키는 전처리 방법이다.



[그림 1] 산제 전처리 공정 모습

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 산제 전처리 공정의 경우, 제품의 색상 불량, 흑색 불량 등이 발생할 수 있으며, 공정에서 사용하는 사용 용액의 농도가 일정 농도 이상이면 산처리 품질 저하로 전체 도금 품질 저하가 발생하여 도금 불량으로 취급되는 경우가 발생할 수 있다. 산제 반응 과정에서 발생하는 수소가스가 제품의 금속 내부로 침투하면서 금속 결합 내부에 들

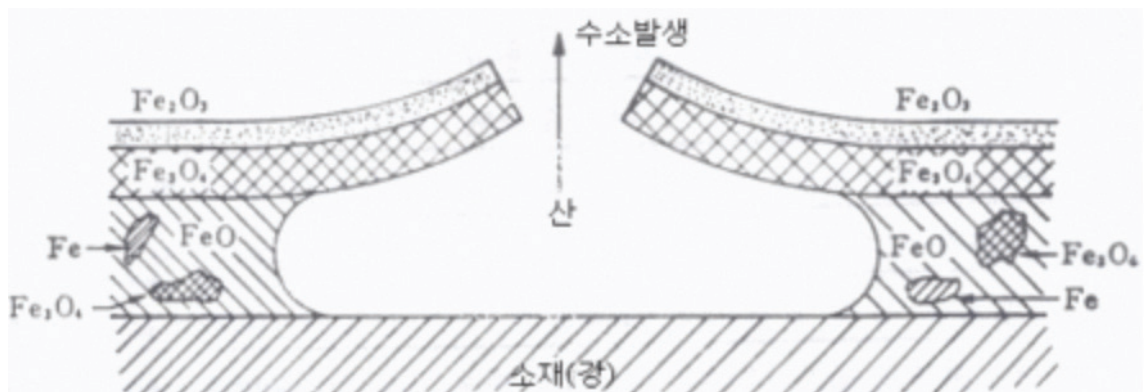
어갈 수 있다. 이러한 수소취성이 발생할 경우, 제품 표면이 갈라지거나 심할 경우 제품이 수소분자 압력에 의해서 깨질 수 있다. 또한, 산제 전처리를 수행함에 있어 공정 시간이 되도록 짧아질 수 있도록 조절하는 것이 중요하다.

• 문제해결 장애요인

- 현재 다양한 데이터가 수집되고 있으나 활용을 못 하는 상황으로, 이는 대부분의 중소 기업 제조 현장에서 공통으로 겪고 있는 것으로 판단된다.

• 극복 방안

- 도금공정에서 발생하는 다양한 데이터 분석을 통해 공정 운영 최적화, 품질 예측, 설비 예지보전 등에 사용 가능한 것으로 판단된다. 각 공정에서 수집되는 데이터는 공정에서 발생하는 문제를 파악하는 데 사용될 수 있다.



[그림 2] 산제 반응 과정 모식도 [출처:도금기술사전]

1.2 분석 목표

1) 분석 목표

• 산제 전처리

- 공정 데이터(pH, 온도, 시간)를 이용하여 AI를 통한 공정 운영 최적화에 사용할 수 있도록 분석을 진행한다.
- 산제 전처리 공정에서 발생하는 공정 품질 데이터와 생산 조건 데이터들의 분석을 통해 데이터 간의 상관관계 분석을 진행하고 주요 문제별 원인 인자를 분석하여 기계 학습 알고리즘을 활용하여 산제 전처리 공정 운영 최적화 모델을 만들고자 한다.
- 산제 전처리에 있어서, 산화 피막 제거 및 표면 활성화를 위한 산 물질의 pH 최소 기준은 정해져 있으며, 산제 반응에서 발생하는 반응열이 존재하므로 온도 역시 전면적으로 조절할 수는 없는 변수이다. 그러나, 공정 시간은 자유롭게 조절이 가능하다. 공정 시간이 길어지면 공정 운영에 대한 비용 문제와 수소취성으로 인한 품질 문제가

발생하며, 공정 시간이 짧아지면 산화 피막이나 불순물이 남아있고 금속 표면이 비활성 상태로 존재하여 이후 공정에서의 품질 저하 문제가 발생한다. 따라서 시간 변수에 대한 최적화 모델을 구성하고자 한다.

2) 데이터 정의 및 소개

• 산재 전처리 설비 데이터

- 제품 생산 시 설비에 설치된 센서를 통해 수집된 설비 데이터
- Index, LoT, Time, pH, Temp 데이터로 이루어진다.

3) 데이터 분석 기대효과

• 산재 전처리 설비 데이터

- 설비 데이터를 이용하여 학습된 모델에 적용 시, 공정 운영 최적화를 위한 기반 데이터 계산이 가능하다.

4) 시사점(implication) 요약기술

• 산재 전처리

- 원재료의 전처리를 수행하는 첫 번째 단계로 공정 품질에 따라 후공정 및 완제품 품질에 미치는 영향이 클 것으로 예상된다.
- 현재 주식회사 켄프의 제품은 균일한 표면 상태가 매우 중요하기 때문에 전처리 단계에서 피도금물이 균일하게 전 처리되는 것이 중요하며, 설비 운영 데이터 분석을 통한 품질 예측이 필요하다.
- 설비 운영 변수에 따른 완제품 품질 영향은 완제품의 표면적에 따라서도 달라질 것으로 예상되어, 제품 종류에 따른 품질 예측 분석에 의미가 있을 것으로 판단된다.
- 전처리 탱크의 산도 및 온도는 표면 반응에 있어서 핵심적인 요인이기에 데이터의 상관관계에 따른 분석 방식이 다양한 산업군에서 도움이 될 것으로 판단된다.
- 산재를 사용한 전처리의 경우, 다양한 뿌리산업에서 사용되고 있으며 주로 최종가격에 큰 영향을 미치는 시각적인 불량을 만들 수 있는 공정이기때문에 고부가가치 데이터로 판단된다.
- 현재 켄프 공장에는 공정에 영향을 줄 것으로 생각되는 공정변수를 통해 공정 운영 최적화를 위한 데이터 분석을 바탕으로 AI 분석 모델을 구축하고 AI로 공정 운영 최적화를 위한 분석을 수행하고자 한다.

• 시사점 분석

- 켄프에서 생성하는 제품은 육안으로 제품의 양품선별을 진행하지만, 사람에 의한 판

단으로 품질의 균일성을 확보하기 힘들었다. 또한, 공정 데이터 수집은 되지만 데이터 분석이 되지 않고 있었으며 이를 활용할 수 있는 AI 모델을 구축하여 각 공정에서 목적한 분석을 수행한다.

- 본 데이터 분석은 현재 열악한 중소기업에 AI를 적용하여 생산성 증대, 품질 향상, 비용 절감 등의 부분에 기여 할 수 있다는 점에서 시사하는 바가 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조 분야 : 이노징크 전기도금
- 제조 공정명 : 전처리
- 수집장비 : 산제 전처리 설비 내 PLC Data, HDH Data
- 수집 기간 : 2021년 09월 06일 ~ 2021년 10월 27일
- 수집 주기 : 5 sec

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(tabular)

- 데이터 개수

(Numerical Input Data) column 수 5개, row 수 50,094개, 데이터수 250,470개

(Numerical Output Data) column 수 3개, row 수 726개, 데이터수 2,178개

- AI 데이터셋 주요 변수 정의

속성(column)	설명	비고
Index	데이터 수집 시 자동 생성 값	int
LoT	LoT 추적을 위해, 동일 LoT에 동일 숫자 부여	int
Time	측정 시 시간을 초 단위까지 기록(H:MM:SS)	timestamps
pH	전처리 설비 내 공정 pH 측정값	float
Temp	전처리 설비 내 공정 온도 측정값	float
Date	각 공정이 수행된 날짜 기록(YYYY-MM-DD)	string
LoT	LoT 추적을 위해 각 LoT 별로 부여	int
Process Rate	전처리가 완료된 결과물의 완성도를 정량 지표화	float

- 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Index	int	50,094	759.5	438.213133	1	759.5	1518.0
LoT	int	50,094	11.5	6.344352	1	11.5	22
Time	timestamps	50,094	NaN	NaN	NaN	NaN	NaN
pH	float	50,094	2.006488	0.551698	1.01	2.0	3.99
Temp	float	50,094	49.876127	1.345322	38.02	49.97	54.19
Date	string	726	NaN	NaN	NaN	NaN	NaN
LoT	int	726	11.5	6.348663	1	11.5	22
Process Rate	float	726	96.064229	3.212802	80.78	96.635	98.45

- 독립변수 / 종속변수

구분	명칭
독립변수	pH
	Temp
종속변수	Process

- 분석에서는 공정 품질 결과에 영향을 줄 것으로 판단되는 pH와 Temp를 고려하였다. 따라서 pH, Temp가 독립변수, Process가 종속변수가 된다.

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 값이 의미 없거나 누락 및 오타가 발생하여 품질이 떨어진다.
- 아무리 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없다.
- 그렇기에 데이터 (품질) 전처리는 데이터 분석에 있어서 필수적인 단계이다,
- 따라서 데이터들의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상한다.

• 데이터 품질 지수

- 완전성(Completeness) : 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복되어서는 안 된다.
- 유효성(Validity) : 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.

- 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
- 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 결측치 값의 비율을 확인하여 삭제한다.
- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 이용하여 총 결측치 개수를 구한다.
- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

- 유일성 품질 지수 = $((\text{유일한 데이터 수} / \text{전체 데이터 수})) * 100$

- ① 이 데이터는 유일한 값을 가지는 열이 존재하지 않으므로 유일성을 판단하지 않는다.
- ② 유일성을 판단하고 싶다면 날짜와 시간 두 열을 이용하여 합성키를 만든 뒤 기본키로 설정하여 중복을 판단하면 된다.

- 유효성 품질 지수 = $(\text{유효성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터가 유효범위 내에 있는가?
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.

- 일관성 품질 지수 = $(\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터 테이블 간 종속관계 확인
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.

데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.

- 정확성 품질 지수 = $(1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) * 100$

해당 최대수요전력 예측과 관련하여 중복되는 예측값은 존재할 수 있으나 누락이 발생해서는 안된다.

- 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) * 100$
- 무결성 품질 지수는 고려되는 모든 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지수이다. 본 가이드북에서는 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질 지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 3] 데이터 흐름도

- 제공하는 데이터의 경우 각 LoT의 Process Rate를 담아 둔 파일인 kemp-process-rate.csv를 제외한 나머지 데이터 파일은 데이터가 수집되는 당시 결측치가 자동으로 채워지는 방식을 실행하고 있으므로, 파일 내에는 결측치가 나타나지 않는다. 현재 설비의 데이터 수집장치에서는 결측치가 연속적으로 나타나면 이상으로 탐지하여 알람을 주고 있다.
- 이상치의 경우 설비의 이상 탐지를 위해 이상치 제거 및 변경을 진행하지 않고 있다.
- 결측치, 이상치 제거

(1) 결측치 제거

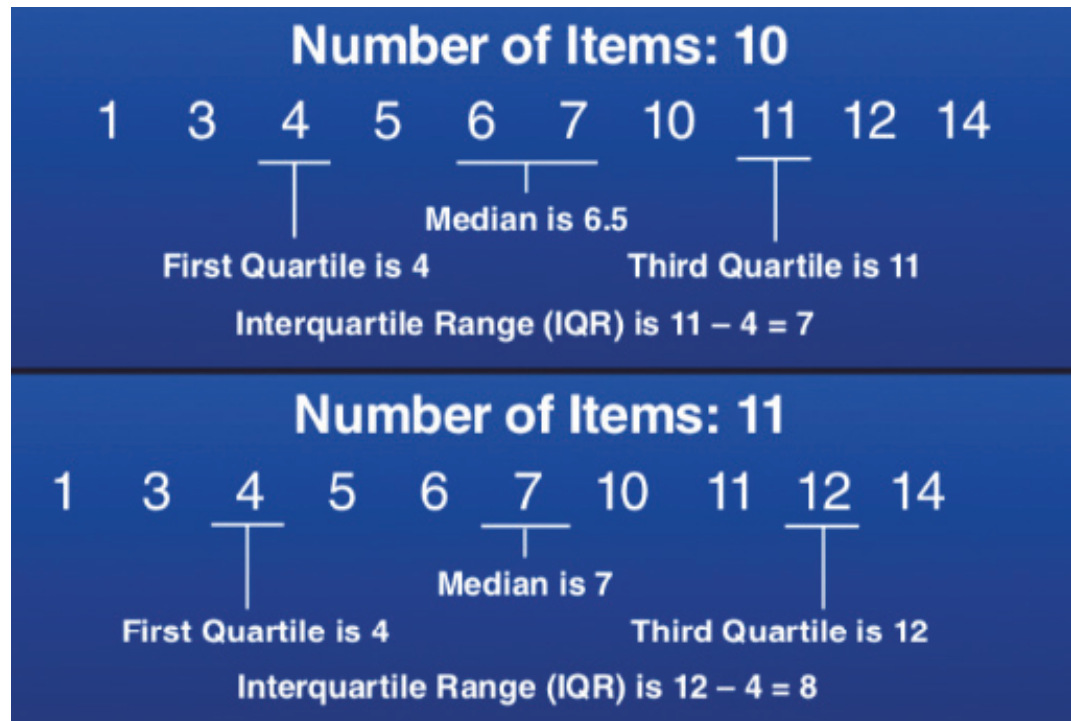
- **Index 제거 방법 (dropna())**
 - excel 내 결측치가 존재할 경우 해당 행을 없애는 방법
- **데이터 흐름을 통한 결측치 채우는 방법(fillna())**
 - 이동평균을 통해 결측치를 채우는 방법
 - 앞/뒤 데이터 평균을 통해 결측치를 채우는 방법

(2) 이상치 제거

- **이상치 탐지 방법**

- Tukey Fences

- 사분위 범위(IQR, interquartile range)를 기반으로 하는 알고리즘으로 세 번째 사분위에서 첫 번째 사분위를 뺀 값으로 나타내며 이를 식으로 나타내면 $IQR = Q3 - Q1$ 으로 나타낼 수 있다.



[그림 4] Tukey Fences 예시

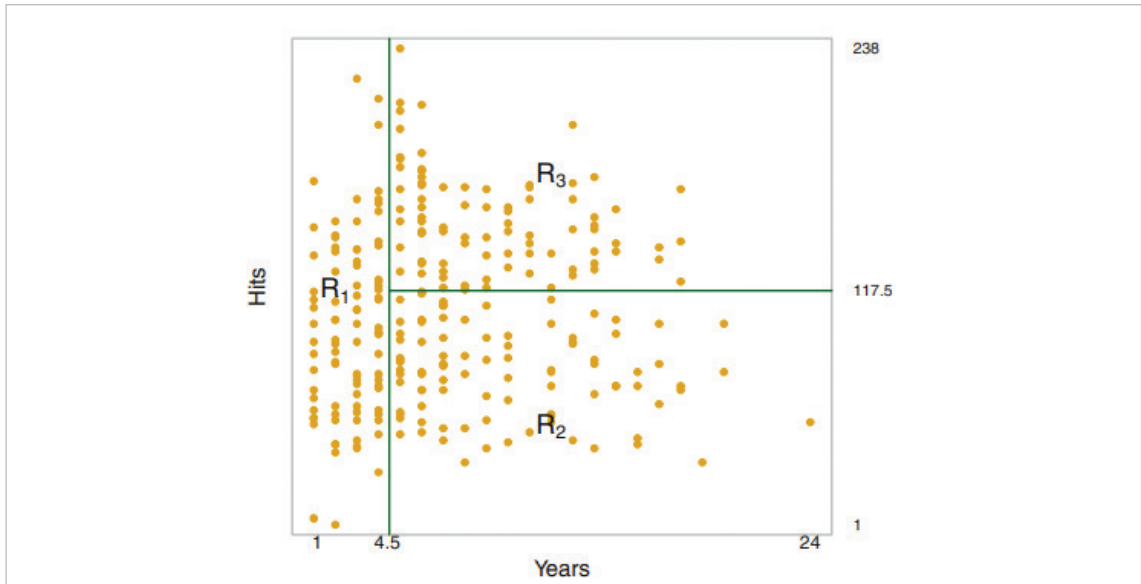
- Tukey Fences의 경우 $Q3 + (1.5 * IQR)$ 을 초과하거나, $Q1 - (1.5 * IQR)$ 미만인 경우 이상치로 판단한다.

- Z Score

- 데이터의 평균과 표준편차의 차이를 이용하는 방법으로 표준점수(Z Score)를 구하면 변환한 데이터의 평균값이 0이 되고, 표준편차는 1이 된다.
- Z Score의 경우 표준점수의 값이 3보다 크거나 -3 보다 작은 것을 이상치로 판단한다.

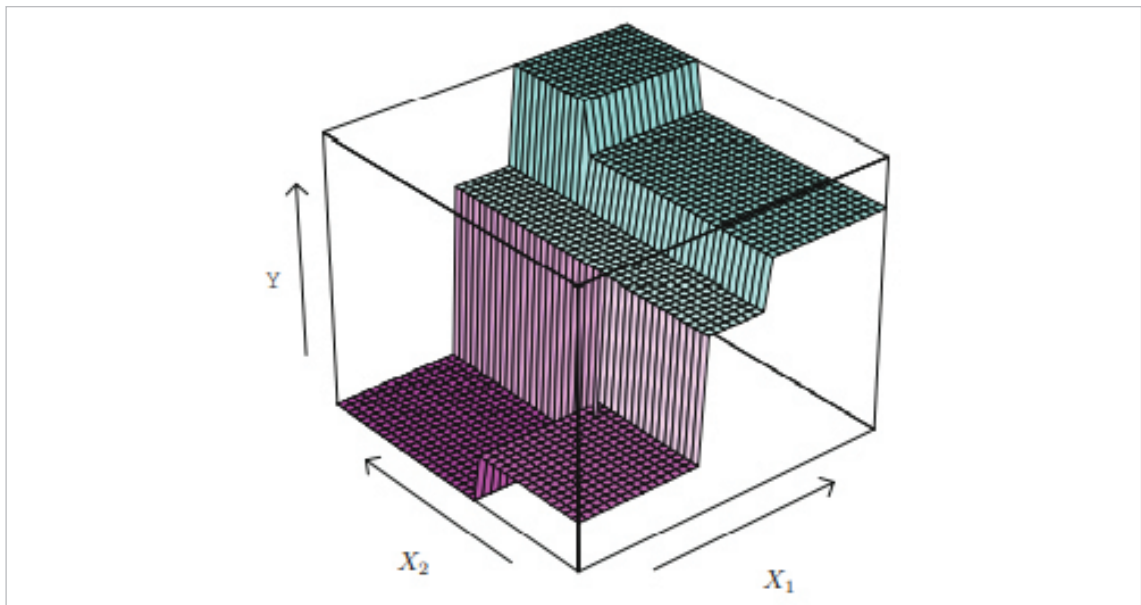
◦ 데이터 흐름을 통한 이상치 채우는 방법

- 이동평균을 통해 결측치를 채우는 방법
- 앞/뒤 데이터 평균을 통해 결측치를 채우는 방법



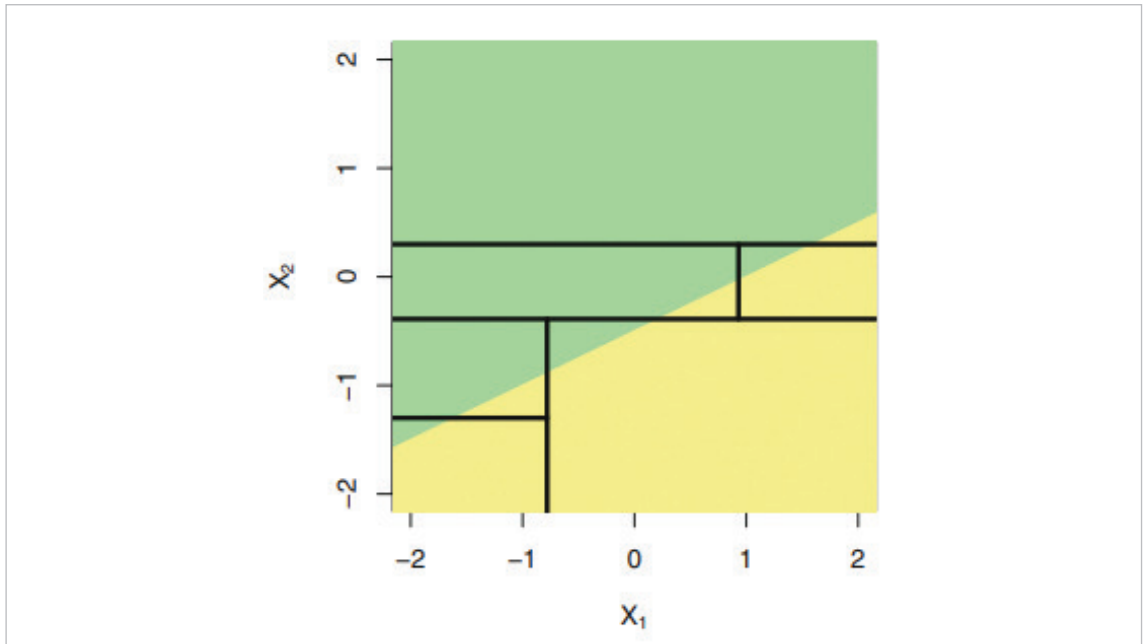
[그림 6] Domain Separation

- Data가 분포한 Feature space에서 살펴보면, 각각의 branch는 주어진 space를 분할(separation)하는 경계(boundary)에 해당하며, 각각의 leaf는 분할된 영역(Region)에 해당한다. 또한, 이를 고차원에서 보면 아래 그림과 같이 곡면(Surface)의 계단형 근사(Step function Approximation)에 해당한다.

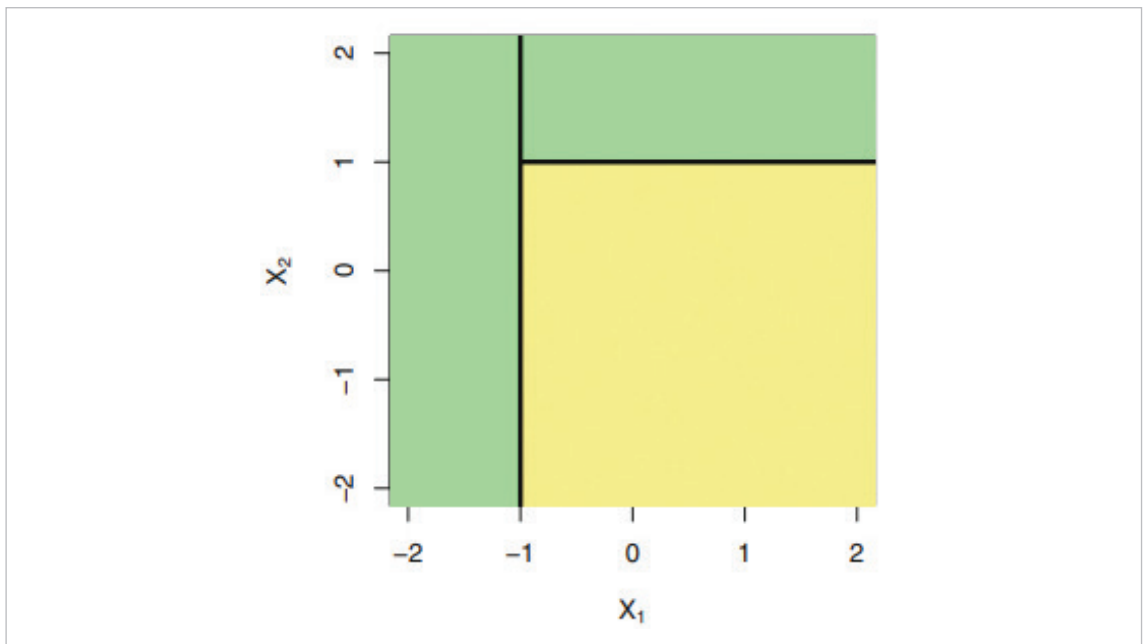


[그림 7] Surface Approximation

- Decision Tree는 Regression과 Classification 양쪽 모두에 사용 가능하지만, 분기점을 통해 영역을 분할한다는 특성에 주목하여 classification 방향으로 사용되는 쪽이 더 선호된다. 그러나, 어느 측면으로 활용되든 True-Value의 분포가 직교 구조(Orthogonal Structure)에 근접할수록 성능이 잘 나온다는 한계점은 존재한다.



[그림 8] Incline Domain



[그림 9] Orthogonal Domain

- Decision Tree는 영역을 분할한 후 각 영역 별로 데이터가 분포하므로, 검증에 있어서도 각 영역별로 집계 후 전체 영역이 합산된다. Regression Tree의 경우, 각 영역을 $R_j (j=1,2,\dots,J)$, True-value를 y_i , Prediction-value를 $\hat{y}_{R_j}$ 라 하면 전체 성능 평가는 다음과 같다.

$$\min \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

- Classification Tree의 경우에도 분할된 영역에 대해서, 각각 분류해야 할 Category Error를 고려해야 한다. $\hat{p}_{mk}$ 를 m 번 영역에서의 k -th class가 차지하는 Training Data 비율이라고 하면, 가장 기본적으로 사용되는 Classification Tree Error는 다음과 같다.

$$E = 1 - \max_k [\hat{p}_{mk}]$$

해당 표현을 기반으로, Tree의 크기에 대한 민감성(Sensitivity)을 높인 형태로는 Gini Index와 Entropy가 있다.

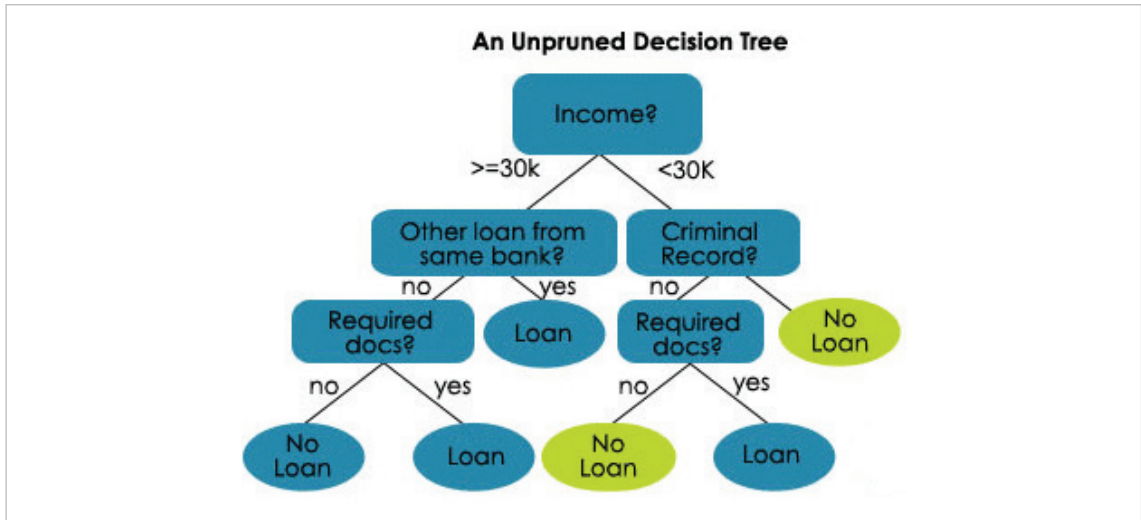
$$\text{Gini Index } G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

$$\text{Entropy } D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

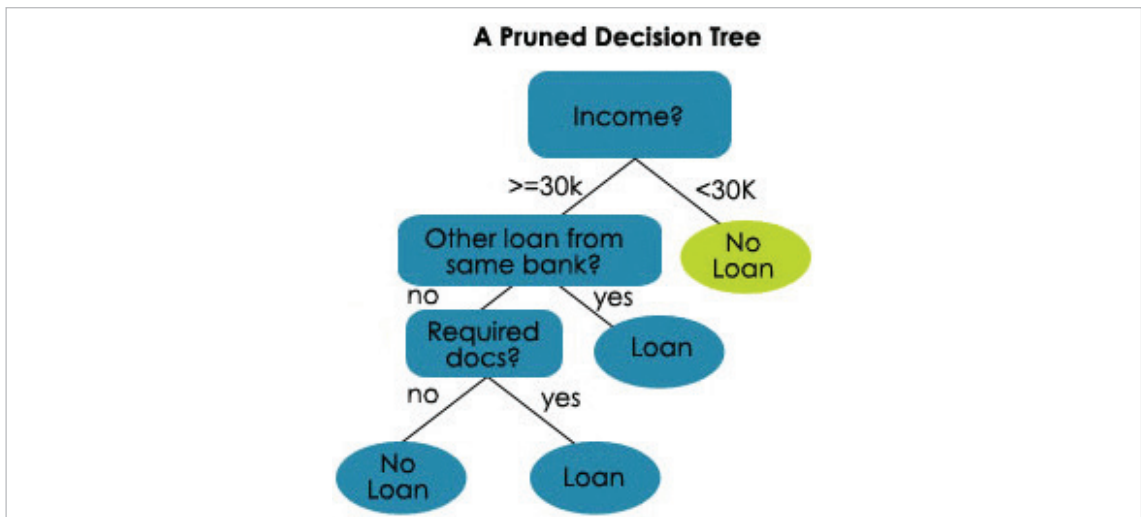
- 모델의 성능을 높이기 위해서는, 분기점의 수가 많아져야 한다. 그러나 분기점을 최대한 높이면 각 데이터가 하나의 Terminal Node를 형성하게 되며, 이는 궁극적으로 과적합(Over-fitting) 오류를 일으키게 된다. 따라서, 나무의 크기가 무한정 커지지 않도록 그 크기에 대한 상계 또는 penalty term이 있어야 한다. Decision Tree에서 해당 과정을 Tree Pruning이라고 하며, Terminal node의 수를 $|T|$ 라 할 때, 최소화 해야하는 목표 함수(objective function)는 다음과 같다.

$$\min \left[\sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 + \alpha |T| \right]$$

- 위에 제시된 계산 식에서, 앞쪽의 기준에 제시되었던 항은 Loss Term 또는 Fidelity Term이라고 하며 오류를 최소화하기 위해서 Terminal node $|T|$ 를 늘리고자 한다. 뒤쪽에 추가된 항은 Penalty Term 또는 Regularization Term이라고 하며, 전체 목표 함수 최소화를 위해서 Terminal node $|T|$ 를 줄이고자 한다. α 는 Tuning parameter로 Cross-Validation을 통해서 결정한다.



[그림 10] Unpruned Decision Tree



[그림 11] Pruned Decision Tree

• Decision Tree의 장/단점

- Decision Tree는 가시성이 매우 뛰어나기 때문에, Tree 구조를 직접 보여주면서 설명하기가 매우 좋다. 또한, 분기점을 두고 기준에 맞추어 데이터를 나누어가는 과정은 인간의 인지구조 및 결정과정과 매우 유사하기 때문에, Decision Tree는 직관적으로 이해하고 받아들이기 매우 유리하다. 끝으로, 분기점을 가지고 있기 때문에 Network 방식을 제외한 기존의 학습법들과 달리 추가적인 Dummy Variable 없이 바로 학습을 수행할 수 있다.
- 그러나, 단일 Decision Tree는 그 성능 자체에는 상당한 한계가 존재한다. 단일 Tree는 Regression 및 Classification에 있어서 기존에 존재하는 거의 모든 학습 방식보다 그 성능이 뒤쳐진다(즉, 오류율이 높다). 또한, 그 구조가 매우 취약해서 데이터의 작은 변화에도 전체 Tree가 상당히 흔들리기 쉬운 High-variance Structure이다. 따

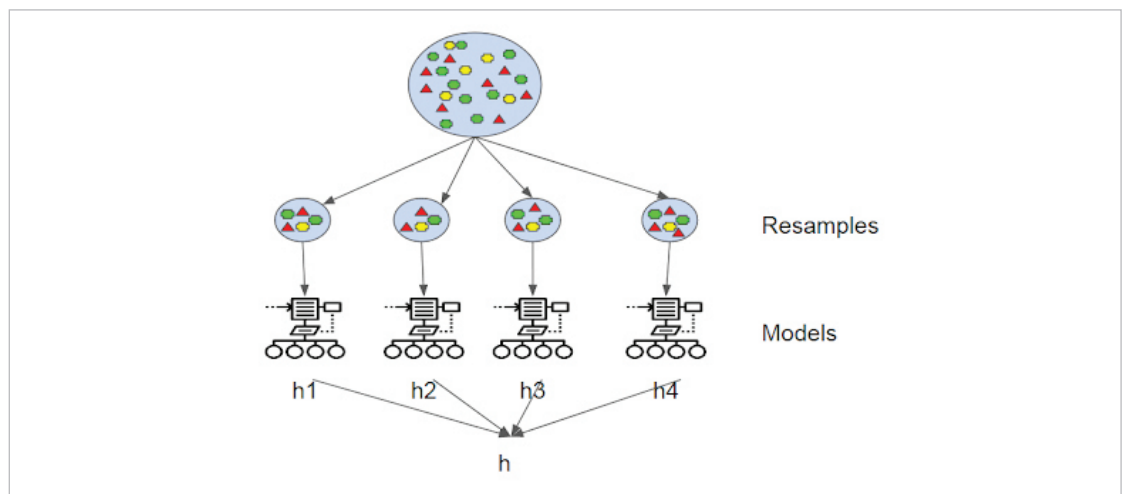
라서 이를 극복하기 위해 가시성을 일부 포기하고 성능을 높이는 기법들이 있으며, 각각 Bagging, Random Forest, Boosting이라고 불린다.

• Bagging/Random Forest/Boosting

- Bagging은 기본적으로 Bootstrap의 개념을 따른다. 먼저 전체 데이터 모집합 Z 에서 데이터를 n 개씩 복원 추출하여 B 개의 Sampling Set $Z_1, Z_2, \dots, Z_B$ 을 구성한다. 이후, 각각의 Sampling Set에 대한 Tree $\hat{f}^{(b)}$ 를 학습한다. Regression Tree의 경우 최종 결론은 각 Tree들의 평균으로 도출한다.

$$\hat{f}_{bag} = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}$$

Classification의 경우, 각 Tree들의 분류 결과를 취합한 이후에 다수결 투표 (majority vote)의 결과를 따라 최종 결과를 도출한다.

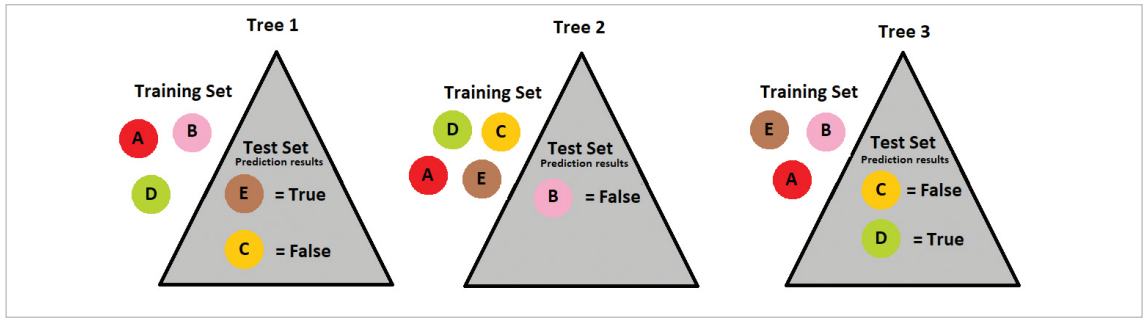


[그림 12] Bagging Structure

- Bagging의 경우, 성능 평가를 out-of-bag(OOB)로 수행한다. 해당 방식은 교차 검증 중 데이터를 하나씩 제거하여 검증하는 Leave-one-out Cross-validation(LOOCV)에 기반을 두고 있다. Bagging을 수행하기 위해 Bootstrap을 수행하고, 여러 개의 Tree를 구성할 때, 특정 데이터가 포함/배제되는 확률이 sampling 수 n 에 따라 약 $1/3$ 로 수렴하는 특징이 있기 때문이다.

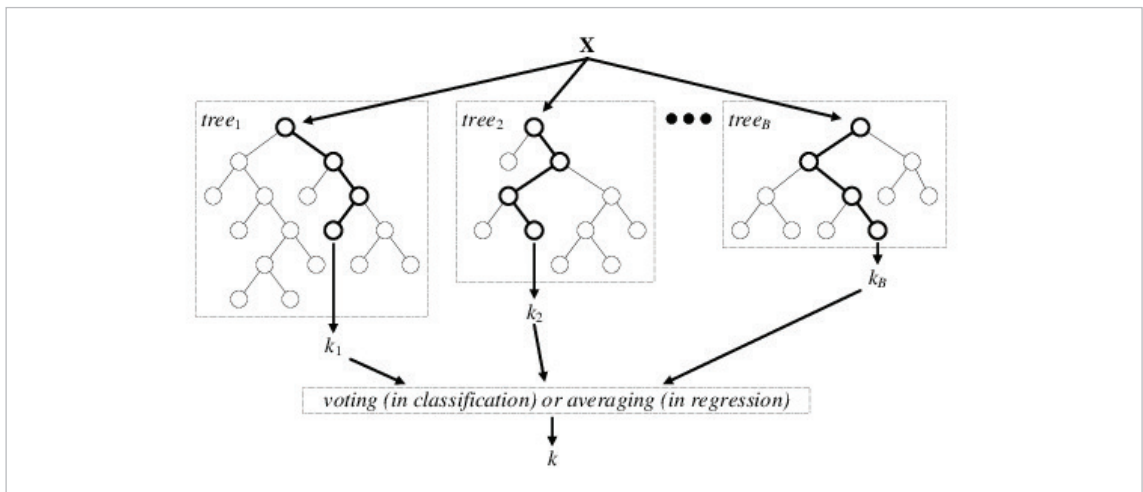
$$\lim_{n \rightarrow \infty} p_n = \frac{1}{e} \simeq \frac{1}{3}$$

해당 계산에 근거하여, 전체 데이터셋의 $1/3$ 을 Validation Set으로 분할하고 나머지 $2/3$ 의 데이터셋만으로 학습을 수행한다. 이 때, 학습에 포함되지 못한 $1/3$ 개의 데이터셋을 out-of-bag이라고 부르며, 해당 검증법의 어원이다.



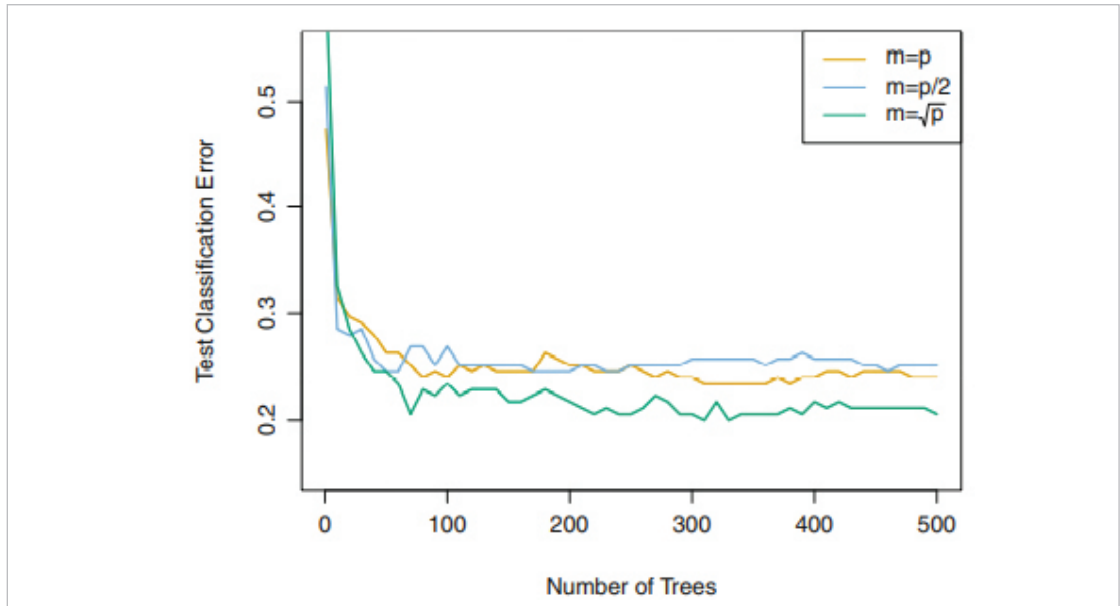
[그림 13] Out of Bag(OOB)

- Random Forest는 Bagging의 좀 더 진화된 형태이다. Bagging이 데이터셋만을 Random Sampling하여 재구성하였다면, Random Forest는 데이터를 학습/구분하기 위한 Feature들까지 Random Sampling한다. 즉, Bagging에서 각각의 sub-tree가 동일한 Feature space에서 전개되는 반면, Random Forest는 각각의 sub-tree가 서로 다른 Feature space에서 전개된다.



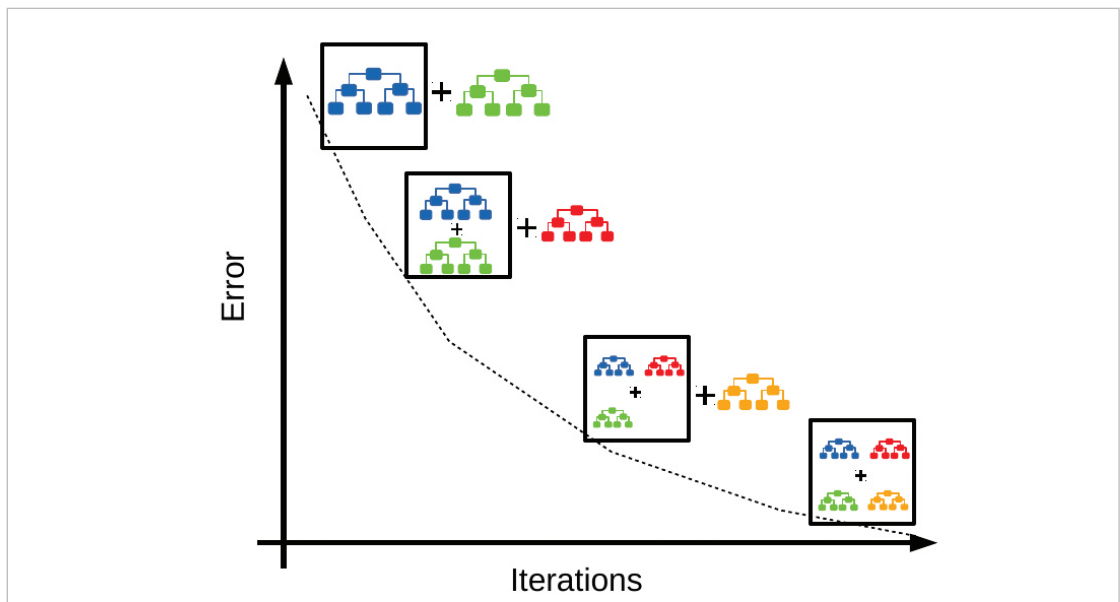
[그림 14] Random Forest Structure

- Random Forest가 이렇게 서로 다른 Feature space를 구성하는 것은 sub-tree 사이에서 발생하는 상호연관성(Correlation)을 제거하여 성능을 더욱 높이기 위함이다. 해당 방식을 Decorrelation 또는 Tweak이라고 한다. 전체 Feature space가 R^m 이라고 하면, sub-Feature space의 크기는 R^p ; $p \approx \sqrt{m}$ ($p=m$ 으로 하면 앞서 Bagging과 동일)이며, 각각의 sub-tree는 p 개의 sub-feature를 서로 다르게 구성한다.



[그림 15] Comparison with Bagging and Random Forest

- Boosting은 앞서 두 가지 방식과는 조금 다르게 Tree-based Method를 발전시킨다. Boosting의 가장 큰 특징은 앞서 학습된 Tree를 참고하면서, 전체를 천천히 학습해 나간다는 것이다. 이는 선행 단계에서 수행된 학습 결과를 보고, 선행 단계에서 제대로 학습하지 못한 부분을 집중적으로 학습하겠다는 개념이다. 따라서 각 Algorithm Step끼리의 의존성이 매우 높다.



[그림 16] Boosting Structure

- 전체 B 개의 sub-tree $\hat{f}^{(b)}$ 를 구성 및 학습하면서, 전체 학습은 다음과 같다.

(1) 초기 전체 Tree $\hat{f}=0$ 및 Residual $r_i=y_i$

(2) b 단계에서 sub-tree $\hat{f}^{(b)}$ 를 구성

(3) $(b-1)$ 단계까지 학습된 전체 Tree를 $\hat{f}_{b-1}$, Residual을 $r_i^{(b-1)}$ 이라 할 때, b 단계에서는 각각 다음과 같이 학습된다.

$$\begin{aligned}\hat{f}_b &= \hat{f}_{b-1} + \lambda \hat{f}^{(b)} \\ r_i^{(b)} &= r_i^{(b-1)} - \lambda \hat{f}^{(b)}\end{aligned}$$

(4) 최종적으로 학습된 Tree $\hat{f} = \sum_{b=1}^B \lambda \hat{f}^{(b)}$

• Bagging/Random Forest/Boosting의 장/단점

- Bagging/Random Forest/Boosting의 장단점은 단일 Decision Tree의 장단점과 Trade-off 관계에 있다. 개선 모델은 모두 Bootstrap에 기반하고 있으므로, 단일 Decision Tree와 달리 적은 양의 데이터셋으로도 상당히 우수한 성능을 보여준다. 또한, 여러 개의 sub-tree를 구성 후 합산하므로 데이터의 변동에 대해서 상당히 안정적이다. 즉, 단일 Decision Tree에 비해서 Low-variance & Low-Bias라는 비교할 수 없는 상당한 성능 차이를 보여준다.

- 그러나, 단일 Decision Tree가 하나의 나무로 모든 것을 가시화하여 직관적인 이해 가능성을 보여준 것에 비해, 여러 sub-tree가 복합적으로 작용하므로 개선 모델은 그 구조를 직관적으로 바로 이해하기 어렵다. 또한, 차원이 높아지면서 가시화 자체가 불가능한 경우도 존재한다. 끝으로, 여러 sub-tree를 구성하고 학습하는 과정 자체가 추가되므로, 학습에 수행되는 computational cost가 기존에 비해 매우 높아지는 단점 또한 존재한다.

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 변수 분할

• 모든 변수 중 독립변수와 종속변수를 분할하고, 학습 데이터와 테스트 데이터로 분할한다. 산제전 처리설비 데이터의 경우 pH와 Temperature 데이터를 독립변수로 설정하고 품질 결과 변수를 종속변수로 설정한다. 이렇게 나눈 데이터를 일부는 학습 데이터로, 나머지는 테스트 데이터로 나누어 머신러닝을 진행한다.

- Decision Tree 학습 및 시각화

- Decision Tree 알고리즘을 활용하여 데이터를 예측하도록 학습한다. 과적합이 발생하지 않도록 max_depth를 조정하여 적절한 수준을 찾도록 하며 시각화를 통해 Tree의 모델링 규칙을 확인한다.

- 모델 평가 및 해석

- Confusion Matrix를 통해 모델의 성능을 평가하며, 정확도를 산출한다. 이후 시각화 결과를 통해 어떠한 규칙으로 예측이 되었는지 확인한다.

2.3 분석 체험

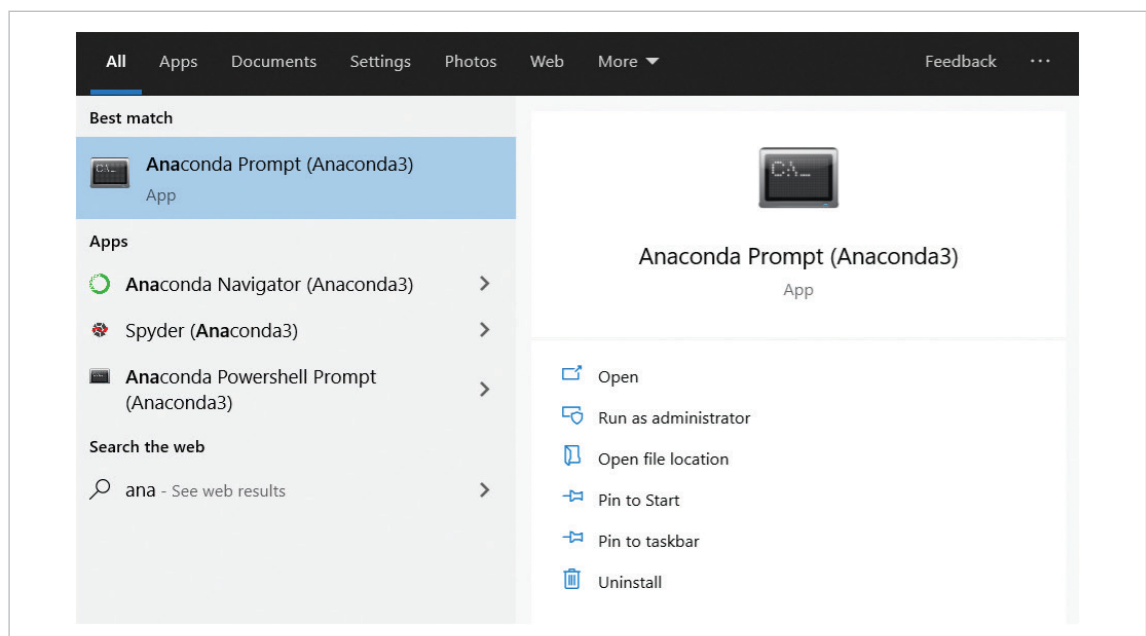
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

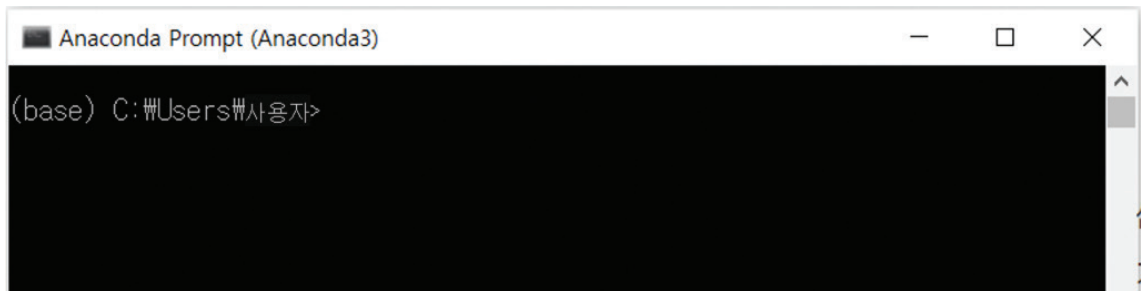
▶ SW 설치는 '부록_분석환경 구축을 위한 설치 가이드' 참고

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

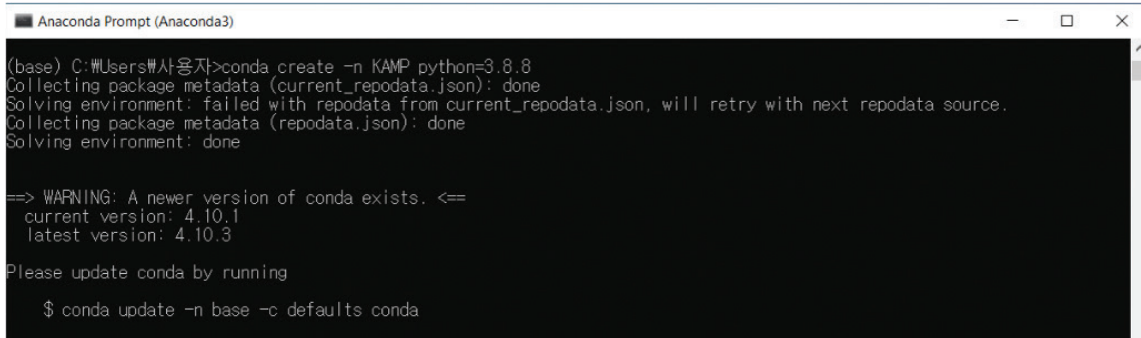
- 아나콘다 프롬프트 실행



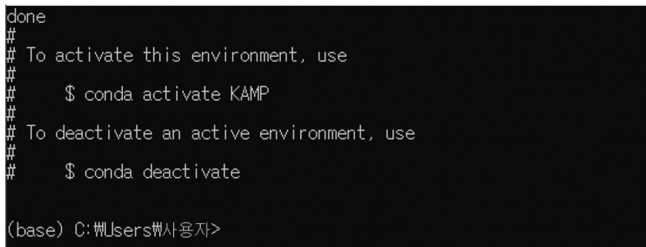


- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전  
conda create -n KAMP python=3.8.8
```



Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.

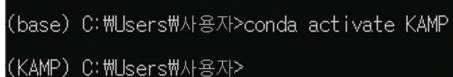


- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```



```
pip install ipykernel
```

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"  
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

```
# jupyter notebook 실행  
jupyter notebook
```

```
(KAMP) C:\Users\User>pip install ipykernel  
Collecting ipykernel  
  Using cached ipykernel-6.6.0-py3-none-any.whl (126 kB)  
Collecting debugpy<2.0,>=1.0.0  
  Using cached debugpy-1.5.1-cp37-cp37m-win_amd64.whl (4.3 MB)  
Collecting matplotlib-inline<0.2.0,>=0.1.0  
  Using cached matplotlib-inline-0.1.3-py3-none-any.whl (8.2 kB)  
Collecting importlib-metadata<5  
  Using cached importlib_metadata-4.8.2-py3-none-any.whl (17 kB)  
Collecting tornado<7.0,>=4.2  
  Using cached tornado-6.1-cp37-cp37m-win_amd64.whl (422 kB)  
Collecting jupyter-client<8.0  
  Using cached jupyter_client-7.1.0-py3-none-any.whl (129 kB)  
Collecting ipython<7.23.1
```

```
(KAMP) C:\Users\User>python -m ipykernel install --user --name KAMP --display-name "KAMP"  
Installed kernelspec KAMP in C:\Users\User\AppData\Roaming\Jupyter\kernels\KAMP  
(KAMP) C:\Users\User>
```

```
(KAMP) C:\Users\User>jupyter notebook  
[W 16:23:41.477 NotebookApp] All authentication is disabled. Anyone who can connect to this server will be able to run code.  
[I 16:23:41.788 NotebookApp] JupyterLab extension loaded from C:\Users\User\anaconda3\lib\site-packages\jupyterlab  
[I 16:23:41.788 NotebookApp] JupyterLab application directory is C:\Users\User\anaconda3\share\jupyterlab  
[I 16:23:41.790 NotebookApp] Serving notebooks from local directory: C:\Users\User\PycharmProjects\AIPOC  
[I 16:23:41.790 NotebookApp] Jupyter Notebook 6.1.4 is running at: http://localhost:8888/  
[I 16:23:41.790 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
```



2) 분석 단계별 프로세스 - Flow Chart

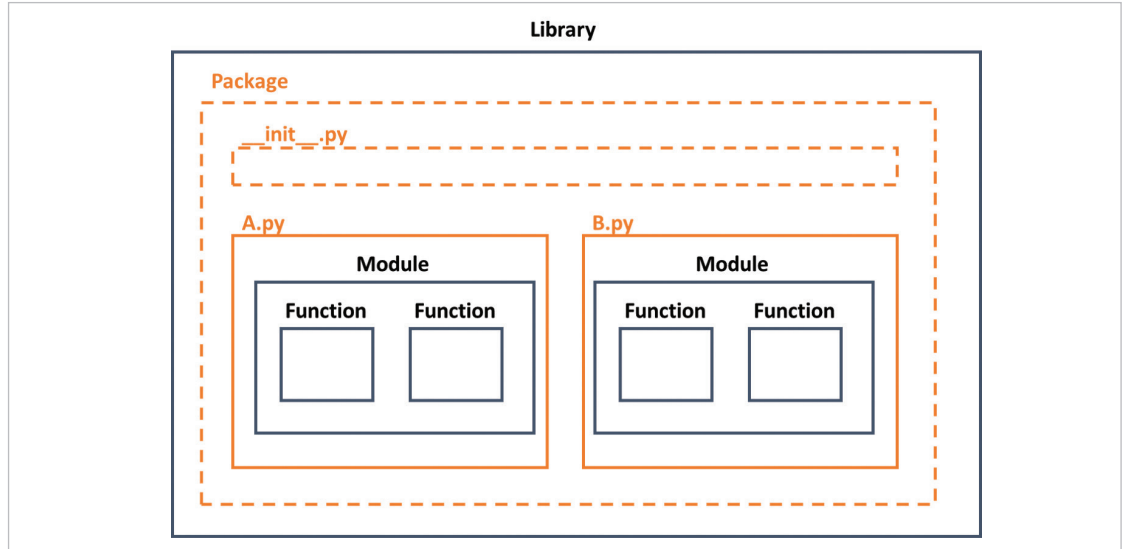


[그림 17] Flow Chart

[단계 ①] 데이터 준비

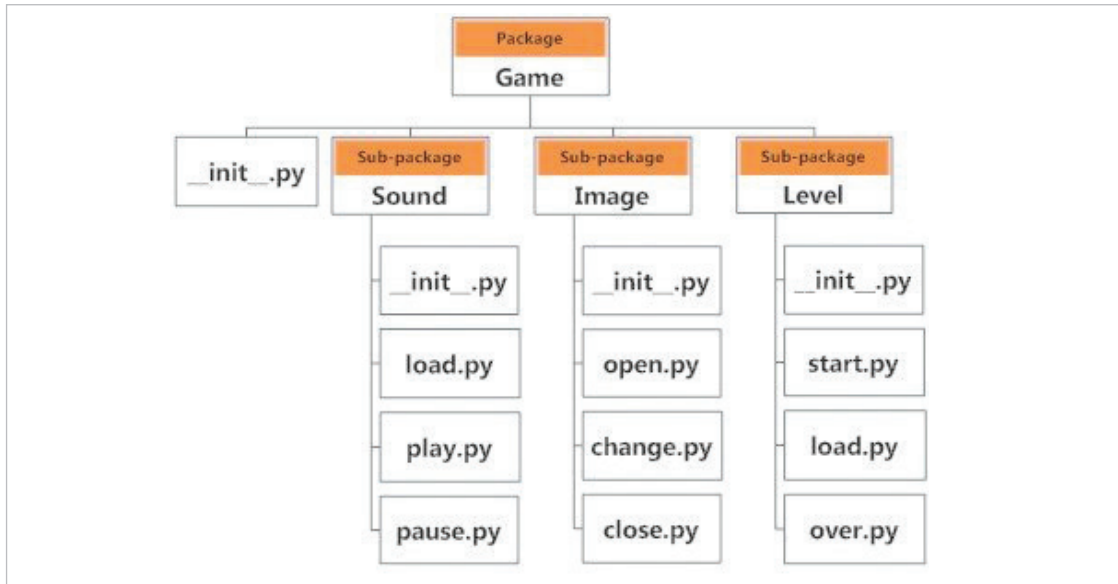
①-1. 라이브러리/패키지 설치

- 파이썬에서는 여러 함수, 모듈, 패키지, 라이브러리 등이 존재한다. 이에 대한 개념은 아래 그림과 같다.



[그림 18] Library, Package, Module, Function 구조도

- 라이브러리와 패키지는 다른 프로그램에서 사용할 수 있는 일종의 코드 모음으로 볼 수 있다.
- 라이브러리의 경우 여러 모듈과 패키지를 묶어서 통칭하는 것이다.
- 파이썬을 설치 시, 기본적으로 설치되는 라이브러리가 있는데, 이를 표준라이브러리라고 하며, 예시로는 time, sys, os, math, urllib, random 등이 있다.
- 이외에 외부 3rd party에서 개발한 모듈과 패키지를 묶어서 외부 라이브러리라고 하며, 예시로는 requests, scrapy, webbrowser 등이 있다.
- 표준라이브러리보다 우수하거나 사용하기 쉬운 외부 라이브러리들이 존재하는데 예시로는 numpy, scipy 등이 있다.
- 모듈의 경우 확장자가 .py로 끝나는 일체의 파일을 의미한다. 모듈은 재사용이 가능하며, 모듈을 이루고 있는 것은 소스 코드이며, 소스 코드로 함수를 정의하여 사용한다.
- 하나의 패키지를 도식화 하여 표현한다면 아래 그림과 같다.



[그림 19] 패키지 도식화

```

!pip install pandas
!pip install numpy
!pip install sklearn
!pip install autokeras
!pip install seaborn
!pip install tensorflow
!pip install datetime
!pip install matplotlib
!pip install pydot
!pip install graphviz
Requirement already satisfied: pandas in c:\users\user\anaconda3\envs\work\lib\site-packages (1.3.2)
Requirement already satisfied: numpy>=1.17.3 in c:\users\user\appdata\roaming\python\python37\site-packages (from pandas) (1.19.5)
Requirement already satisfied: pytz>=2017.3 in c:\users\user\anaconda3\envs\work\lib\site-packages (from pandas) (2021.1)
Requirement already satisfied: python-dateutil>=2.7.3 in c:\users\user\anaconda3\envs\work\lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\users\user\appdata\roaming\python\python37\site-packages (from python-dateutil>=2.7.3->pandas) (1.15.0)
Requirement already satisfied: numpy in c:\users\user\appdata\roaming\python\python37\site-packages (1.19.5)
Requirement already satisfied: datetime in c:\users\user\anaconda3\envs\work\lib\site-packages (4.3)
Requirement already satisfied: pytz in c:\users\user\anaconda3\envs\work\lib\site-packages (from datetime) (2021.1)
Requirement already satisfied: zope.interface in c:\users\user\anaconda3\envs\work\lib\site-packages (from datetime) (5.4.0)
Requirement already satisfied: setuptools in c:\users\user\anaconda3\envs\work\lib\site-packages (from zope.interface->datetime) (56.0.0)
Requirement already satisfied: matplotlib in c:\users\user\anaconda3\envs\work\lib\site-packages (3.4.2)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\user\anaconda3\envs\work\lib\site-packages (from matplotlib) (2.4.7)
Requirement already satisfied: cyclor>=0.10 in c:\users\user\anaconda3\envs\work\lib\site-packages (from matplotlib) (0.10.0)
Requirement already satisfied: pillow>=6.2.0 in c:\users\user\anaconda3\envs\work\lib\site-packages (from matplotlib) (8.3.1)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\user\anaconda3\envs\work\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\user\anaconda3\envs\work\lib\site-packages (from matplotlib) (2.8.2)
Requirement already satisfied: numpy>=1.16 in c:\users\user\appdata\roaming\python\python37\site-packages (from matplotlib) (1.19.5)
Requirement already satisfied: six in c:\users\user\appdata\roaming\python\python37\site-packages (from cyclor>=0.10->matplotlib) (1.15.0)
  
```

[코드 1] Library 설치 완료 메시지

- 분석에 필요한 Library를 설치하기 위해서는 위 그림과 같이 기입하여 설치를 진행한다. 설치 시, 셀(Cell)의 경우 Code(코드)로 되어있는지 확인하고 진행하며, 메뉴에서 ▶| 버튼을 클릭하거나 'shift + enter' 키보드를 눌러서 실행시킨다.

①-2. 라이브러리/패키지 불러오기

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime
from sklearn import tree
import seaborn as sns
from sklearn.metrics import mean_squared_error
from math import sqrt
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from math import sqrt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_curve
from sklearn.metrics import classification_report
```

[코드 2] Library import

- 분석에 필요한 Library를 위와 같이 import 한다.
- Library를 불러오는 코드는 'import Library'를 기본으로 한다.
- Library를 불러와서 다른 이름(alias)으로 지정하는 코드는 'import Library as alias'를 기본으로 한다.
- Library 내 Function을 불러오는 코드는 'from Library import Function' 또는 'import Library.Function'을 기본으로 한다. 첫 번째 방법을 사용 시, Function을 사용할 때, Function만 기입하여 사용하면 되며, 두 번째 방법을 사용 시, Function을 사용할 때, Library.Function 형태로 사용해야 한다.
- Library 내 Function을 불러와서 다른 이름(alias)으로 지정하는 코드는 'from Library import Function as alias'를 기본으로 한다.

①-3. 데이터 불러오기

```
root_dir = os.getcwd()
f_lists = os.listdir(root_dir)
print("File Lists : ", f_lists)

File Lists : ['ipywb_checkpoints', 'kemp-abh-sensor-2021.09.06.csv', 'kemp-abh-sensor-2021.09.07.csv', 'kemp-abh-sensor-2021.09.08.csv', 'kemp-abh-sensor-2021.09.09.csv', 'kemp-abh-sensor-2021.09.10.csv', 'kemp-abh-sensor-2021.09.13.csv', 'kemp-abh-sensor-2021.09.14.csv', 'kemp-abh-sensor-2021.09.15.csv', 'kemp-abh-sensor-2021.09.16.csv', 'kemp-abh-sensor-2021.09.17.csv', 'kemp-abh-sensor-2021.09.23.csv', 'kemp-abh-sensor-2021.09.24.csv', 'kemp-abh-sensor-2021.09.27.csv', 'kemp-abh-sensor-2021.09.28.csv', 'kemp-abh-sensor-2021.09.29.csv', 'kemp-abh-sensor-2021.09.30.csv', 'kemp-abh-sensor-2021.10.01.csv', 'kemp-abh-sensor-2021.10.05.csv', 'kemp-abh-sensor-2021.10.06.csv', 'kemp-abh-sensor-2021.10.07.csv', 'kemp-abh-sensor-2021.10.08.csv', 'kemp-abh-sensor-2021.10.12.csv', 'kemp-abh-sensor-2021.10.13.csv', 'kemp-abh-sensor-2021.10.14.csv', 'kemp-abh-sensor-2021.10.15.csv', 'kemp-abh-sensor-2021.10.18.csv', 'kemp-abh-sensor-2021.10.19.csv', 'kemp-abh-sensor-2021.10.20.csv', 'kemp-abh-sensor-2021.10.21.csv', 'kemp-abh-sensor-2021.10.22.csv', 'kemp-abh-sensor-2021.10.25.csv', 'kemp-abh-sensor-2021.10.26.csv', 'kemp-abh-sensor-2021.10.27.csv', 'kemp-process-rate.csv', 'model.png', 'Optimization of Process operation.ipynb', 'regressor_model.h5', 'structured_data_regressor']
```

[코드 3] 데이터 파일 리스트 출력 화면

- 데이터를 불러오기 위해 위 코드를 진행한다.
- 위 코드를 통해 root_dir로 지정한 디렉토리에서 listdir 함수를 통해 해당 디렉토리 내 모든 파일 리스트를 가져온다. 모든 파일 리스트 출력 시 위와 같이 출력되는 것이 확인 가능하다.

```
new_file_lists = [f for f in f_lists if f.endswith('.csv')]
print("File Lists : ", new_file_lists)

File Lists : ['kemp-abh-sensor-2021.09.06.csv', 'kemp-abh-sensor-2021.09.07.csv', 'kemp-abh-sensor-2021.09.08.csv', 'kemp-abh-sensor-2021.09.09.csv', 'kemp-abh-sensor-2021.09.10.csv', 'kemp-abh-sensor-2021.09.13.csv', 'kemp-abh-sensor-2021.09.14.csv', 'kemp-abh-sensor-2021.09.15.csv', 'kemp-abh-sensor-2021.09.16.csv', 'kemp-abh-sensor-2021.09.17.csv', 'kemp-abh-sensor-2021.09.23.csv', 'kemp-abh-sensor-2021.09.24.csv', 'kemp-abh-sensor-2021.09.27.csv', 'kemp-abh-sensor-2021.09.28.csv', 'kemp-abh-sensor-2021.09.29.csv', 'kemp-abh-sensor-2021.09.30.csv', 'kemp-abh-sensor-2021.10.01.csv', 'kemp-abh-sensor-2021.10.05.csv', 'kemp-abh-sensor-2021.10.06.csv', 'kemp-abh-sensor-2021.10.07.csv', 'kemp-abh-sensor-2021.10.08.csv', 'kemp-abh-sensor-2021.10.12.csv', 'kemp-abh-sensor-2021.10.13.csv', 'kemp-abh-sensor-2021.10.14.csv', 'kemp-abh-sensor-2021.10.15.csv', 'kemp-abh-sensor-2021.10.18.csv', 'kemp-abh-sensor-2021.10.19.csv', 'kemp-abh-sensor-2021.10.20.csv', 'kemp-abh-sensor-2021.10.21.csv', 'kemp-abh-sensor-2021.10.22.csv', 'kemp-abh-sensor-2021.10.25.csv', 'kemp-abh-sensor-2021.10.26.csv', 'kemp-abh-sensor-2021.10.27.csv', 'kemp-process-rate.csv']
```

[코드 4] 정제된 데이터 파일 리스트 출력 화면

- 모든 파일 리스트가 담긴 리스트에서 요소를 하나 꺼내와서 f로 명명하며, f가 '.csv'로 끝나면 새로운 리스트로 구성한다.
- 데이터를 불러오는 코드와 비교 시, 기존 f_lists에는 모든 파일이 들어가 있던 반면, new_file_lists는 파일 형식이 csv인 파일만 리스트에 남는다.


```

data_lists = new_file_lists[:-1]
error_list = new_file_lists[-1]
print("Data Lists :", data_lists)
print("Error Data List :", error_list)

Data Lists : ['kemp-abh-sensor-2021.09.06.csv', 'kemp-abh-sensor-2021.09.07.csv', 'kemp-abh-sensor-2021.09.08.csv', 'kemp-abh-sensor-2021.09.09.csv', 'kemp-abh-sensor-2021.09.10.csv', 'kemp-abh-sensor-2021.09.13.csv', 'kemp-abh-sensor-2021.09.14.csv', 'kemp-abh-sensor-2021.09.15.csv', 'kemp-abh-sensor-2021.09.16.csv', 'kemp-abh-sensor-2021.09.17.csv', 'kemp-abh-sensor-2021.09.23.csv', 'kemp-abh-sensor-2021.09.24.csv', 'kemp-abh-sensor-2021.09.27.csv', 'kemp-abh-sensor-2021.09.28.csv', 'kemp-abh-sensor-2021.09.29.csv', 'kemp-abh-sensor-2021.09.30.csv', 'kemp-abh-sensor-2021.10.01.csv', 'kemp-abh-sensor-2021.10.05.csv', 'kemp-abh-sensor-2021.10.06.csv', 'kemp-abh-sensor-2021.10.07.csv', 'kemp-abh-sensor-2021.10.08.csv', 'kemp-abh-sensor-2021.10.12.csv', 'kemp-abh-sensor-2021.10.13.csv', 'kemp-abh-sensor-2021.10.14.csv', 'kemp-abh-sensor-2021.10.15.csv', 'kemp-abh-sensor-2021.10.18.csv', 'kemp-abh-sensor-2021.10.19.csv', 'kemp-abh-sensor-2021.10.20.csv', 'kemp-abh-sensor-2021.10.21.csv', 'kemp-abh-sensor-2021.10.22.csv', 'kemp-abh-sensor-2021.10.25.csv', 'kemp-abh-sensor-2021.10.26.csv', 'kemp-abh-sensor-2021.10.27.csv']
Error Data List : kemp-process-rate.csv

```

[코드 5] 사용 파일 리스트 출력 화면

- Input과 Output으로 사용될 데이터를 슬라이싱을 통해 지정한다.

```

def csv_read(data_dir, data_list):
    tmp = pd.read_csv(os.path.join(data_dir, data_list), sep=',', encoding='UTF8')
    y, m, d = map(int, data_list.split('-')[:-1].split('.')[::-1])
    time = tmp['Time']
    tmp['DTime'] = '-' + join(data_list.split('-')[:-1].split('.')[::-1])
    ctime = time.apply(lambda _ : _.replace(u'오후', 'PM').replace(u'오전', 'AM'))
    n_time = ctime.apply(lambda _ : datetime.datetime.strptime(_, "%p %!:%M:%S"))
    newtime = n_time.apply(lambda _ : _.replace(year=y, month=m, day=d))
    tmp['Time'] = newtime
    return tmp

```

[코드 6] 데이터를 가져오기 위한 function 정의

- 데이터를 가져오기 위한 function을 정의한다.

```

dd = csv_read(root_dir, data_lists[0])
for i in range(1, len(data_lists)):
    dd = pd.merge(dd, csv_read(root_dir, data_lists[i]), how='outer')
dd

```

	Index	LoT		Time	pH	Temp	DTime
0	1	1	2021-09-06 09:01:18	1.02	47.18	2021-09-06	
1	2	1	2021-09-06 09:01:23	1.05	47.34	2021-09-06	
2	3	1	2021-09-06 09:01:28	1.09	48.45	2021-09-06	
3	4	1	2021-09-06 09:01:33	1.12	48.46	2021-09-06	
4	5	1	2021-09-06 09:01:38	1.15	48.47	2021-09-06	
...	...	...	...	...	...	...	
50089	1514	22	2021-10-27 11:14:41	2.79	51.83	2021-10-27	
50090	1515	22	2021-10-27 11:14:46	3.62	42.20	2021-10-27	
50091	1516	22	2021-10-27 11:14:51	3.40	41.88	2021-10-27	
50092	1517	22	2021-10-27 11:14:56	3.59	40.62	2021-10-27	
50093	1518	22	2021-10-27 11:15:01	3.82	42.08	2021-10-27	

50094 rows × 6 columns

[코드 7] 데이터 프레임 생성 코드

- 위에서 정의한 함수를 이용하여 data_lists를 이용하여 데이터를 불러온다.

- merge 함수의 경우 DataFrame을 합칠 때 사용하는 함수로, how인자를 통해 어떤 방식으로 합칠지 결정한다.

```
dd = dd.drop('Index', axis=1)
dd
```

	LoT	Time	pH	Temp	DTime
0	1	2021-09-06 09:01:18	1.02	47.18	2021-09-06
1	1	2021-09-06 09:01:23	1.05	47.34	2021-09-06
2	1	2021-09-06 09:01:28	1.09	48.45	2021-09-06
3	1	2021-09-06 09:01:33	1.12	48.46	2021-09-06
4	1	2021-09-06 09:01:38	1.15	48.47	2021-09-06
...	...	...	...	...	...
50089	22	2021-10-27 11:14:41	2.79	51.83	2021-10-27
50090	22	2021-10-27 11:14:46	3.62	42.20	2021-10-27
50091	22	2021-10-27 11:14:51	3.40	41.88	2021-10-27
50092	22	2021-10-27 11:14:56	3.59	40.62	2021-10-27
50093	22	2021-10-27 11:15:01	3.82	42.08	2021-10-27

50094 rows × 5 columns

[코드 8] 데이터 컬럼 제거 코드

- drop 함수를 통해 사용하지 않는 컬럼을 없앤다.

```
dd = dd.set_index('Time')
dd
```

	LoT	pH	Temp	DTime
Time				
2021-09-06 09:01:18	1	1.02	47.18	2021-09-06
2021-09-06 09:01:23	1	1.05	47.34	2021-09-06
2021-09-06 09:01:28	1	1.09	48.45	2021-09-06
2021-09-06 09:01:33	1	1.12	48.46	2021-09-06
2021-09-06 09:01:38	1	1.15	48.47	2021-09-06
...	...	...	...	...
2021-10-27 11:14:41	22	2.79	51.83	2021-10-27
2021-10-27 11:14:46	22	3.62	42.20	2021-10-27
2021-10-27 11:14:51	22	3.40	41.88	2021-10-27
2021-10-27 11:14:56	22	3.59	40.62	2021-10-27
2021-10-27 11:15:01	22	3.82	42.08	2021-10-27

50094 rows × 4 columns

[코드 9] 데이터 인덱스 설정 코드

- set_index 함수를 통해 Time 컬럼을 인덱스로 설정한다.

[단계 ②] 데이터 탐색

②-1. 데이터 사본 생성

```
dedicated_data = dd.copy()
dedicated_data
```

Time	LoT	pH	Temp	DTime
2021-09-06 09:01:18	1	1.02	47.18	2021-09-06
2021-09-06 09:01:23	1	1.05	47.34	2021-09-06
2021-09-06 09:01:28	1	1.09	48.45	2021-09-06
2021-09-06 09:01:33	1	1.12	48.46	2021-09-06
2021-09-06 09:01:38	1	1.15	48.47	2021-09-06
...	...	...	...	...
2021-10-27 11:14:41	22	2.79	51.83	2021-10-27
2021-10-27 11:14:46	22	3.62	42.20	2021-10-27
2021-10-27 11:14:51	22	3.40	41.88	2021-10-27
2021-10-27 11:14:56	22	3.59	40.62	2021-10-27
2021-10-27 11:15:01	22	3.82	42.08	2021-10-27

50094 rows × 4 columns

[코드 10] 데이터 사본 생성 코드

- copy() 함수를 통해 데이터 사본을 생성한다.

②-2. 데이터 컬럼 확인

```
dedicated_data.columns
```

```
Index(['LoT', 'pH', 'Temp', 'DTime'], dtype='object')
```

[코드 11] 데이터프레임 컬럼 확인 코드

- columns를 통해 해당 DataFrame의 컬럼을 확인할 수 있다.

②-3. 데이터 프레임 요약

```
dedicated_data.describe()
```

	LoT	pH	Temp
count	50094.000000	50094.000000	50094.000000
mean	11.500000	2.006488	49.876127
std	6.344352	0.551698	1.345322
min	1.000000	1.010000	38.020000
25%	6.000000	1.560000	49.280000
50%	11.500000	2.000000	49.970000
75%	17.000000	2.440000	50.630000
max	22.000000	3.990000	54.190000

[코드 12] 데이터프레임 요약 확인 코드

- describe()를 통해 해당 DataFrame의 내용을 요약하여 확인할 수 있다.

②-4. 데이터프레임 크기 확인

```
dedicated_data.shape  
(50094, 4)
```

[코드 13] 데이터프레임 크기 확인 코드

- shape를 통해 해당 DataFrame의 크기를 확인 할 수 있다.

②-5. 데이터프레임 정보 확인

```
dedicated_data.info()  
<class 'pandas.core.frame.DataFrame'>  
DatetimeIndex: 50094 entries, 2021-09-06 09:01:18 to 2021-10-27 11:15:01  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   LoT         50094 non-null   int64  
1   pH          50094 non-null   float64  
2   Temp        50094 non-null   float64  
3   DTime       50094 non-null   object  
dtypes: float64(2), int64(1), object(1)  
memory usage: 1.9+ MB
```

[코드 14] 데이터프레임 정보 확인 코드

- info()를 통해 해당 DataFrame의 정보를 확인할 수 있다.

②-6. 데이터프레임 null 값 개수 카운트

```
dedicated_data.isna().sum()  
LoT      0  
pH        0  
Temp      0  
DTime     0  
dtype: int64
```

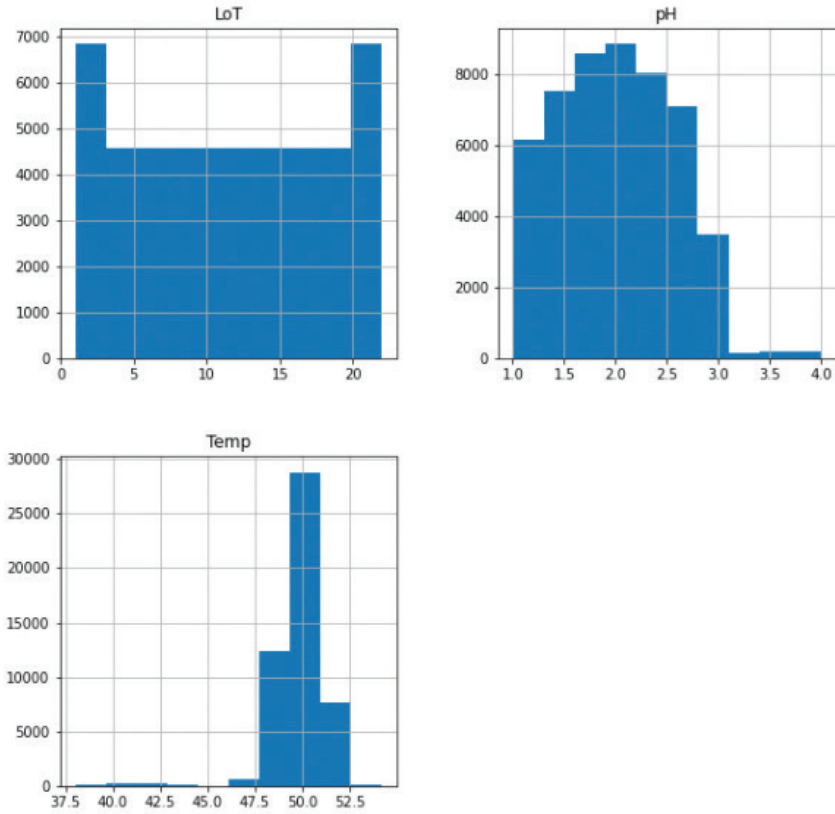
[코드 15] 데이터프레임 null value count 코드

- isna() 함수와 sum() 함수를 함께 사용하여 null 개수를 카운트하여 각 컬럼마다 보여준다.
- isna() 함수의 경우 Index마다 null 값이 있으면 True를 반환해주며, sum() 함수의 경우 True의 개수를 합하여 반환해준다.

②-7. 데이터 시각화를 통한 histogram 확인

```
dedicated_data.hist(figsize=(10,10))
```

```
array([[<AxesSubplot:title={'center':'LoT'}>,  
       <AxesSubplot:title={'center':'pH'}>],  
       [<AxesSubplot:title={'center':'Temp'}>, <AxesSubplot:>]],  
      dtype=object)
```



[코드 16] 데이터프레임 컬럼 히스토그램 시각화 코드

- hist() 함수를 통해 DataFrame의 Column 별 histogram을 시각화하여 위와 같이 출력되는 것이 확인 가능하다.
- 히스토그램으로 데이터 분포를 확인할 수 있다.
- pH의 경우 2.0을 기준으로 데이터가 분포한 것을 확인할 수 있으며, Temp의 경우 50.0을 기준으로 데이터가 분포한 것을 확인할 수 있다.

②-8. 데이터 상관관계 분석

```
correlation = dedicated_data.corr()  
correlation
```

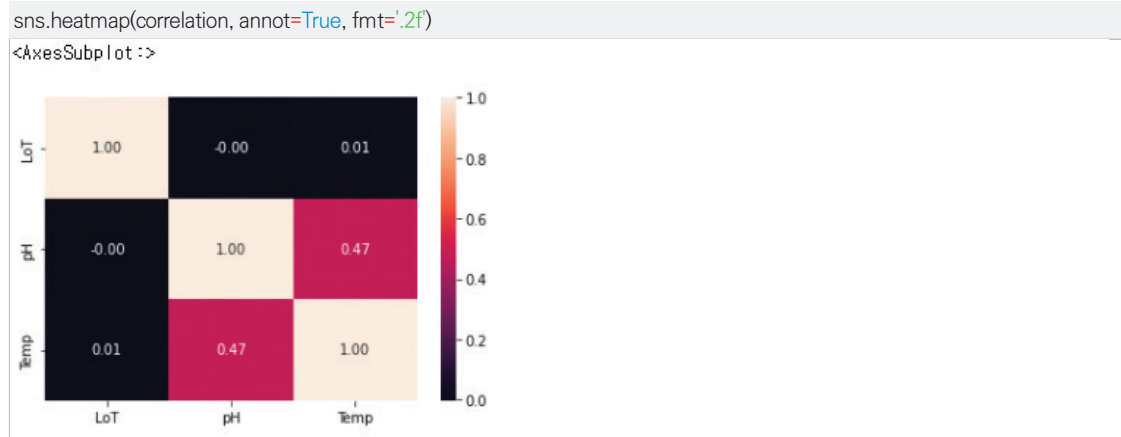
	LoT	pH	Temp
LoT	1.000000	-0.001943	0.011561
pH	-0.001943	1.000000	0.474920
Temp	0.011561	0.474920	1.000000

[코드 17] 데이터프레임 상관관계 분석 코드

- corr() 함수를 통해 Column 간의 상관관계를 구할 수 있다.

- 상관계 분석을 통해 두 변수 간의 선형적인 관계 존재 여부를 확인할 수 있다. 분석에 사용된 상관 계수는 보편적으로 사용되는 피어슨 상관 계수로서 -1과 1 사이의 값을 가진다. 상관 계수의 절대값이 1에 가까울수록 변수 간의 선형관계가 강하다고 볼 수 있으며, 값이 양수일 때는 양의 선형관계, 값이 음수일 때는 음의 선형관계를 가진다. 상관계 분석은 비선형적인 관계를 확인하기 어렵지만, 변수 간의 선형관계를 정량적으로 확인할 수 있다는 장점이 있다.

②-9. 데이터 상관계 시각화

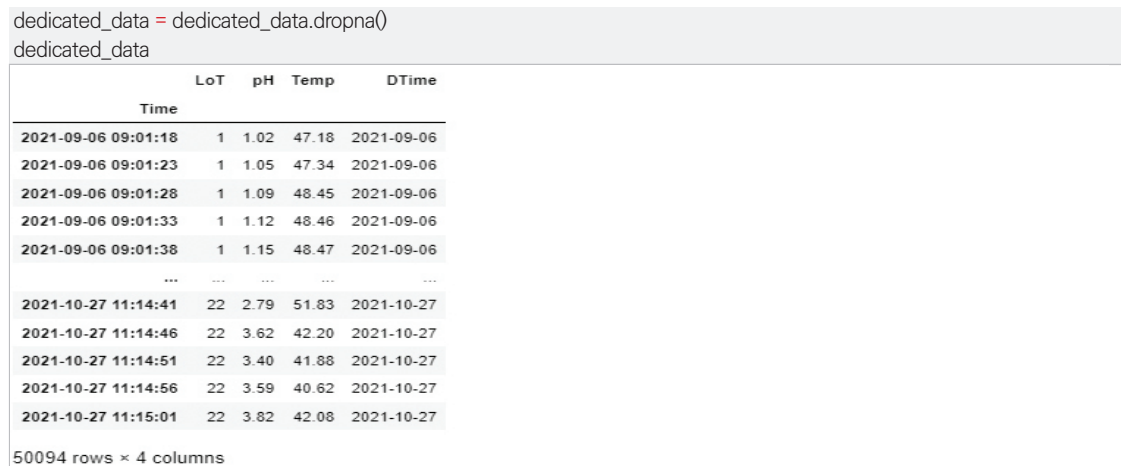


[코드 18] 데이터프레임 상관계 시각화 코드

- seaborn 패키지 내 heatmap() 함수를 통해 상관계 결과를 시각화하여 볼 수 있다.
- 상관계의 경우 값이 클수록 상관계가 있다고 판단할 수 있는 척도가 되며, 위 결과를 통해서 알 수 있는 것은 pH와 Temp의 상관계는 0.47로 낮은 상관을 보이는 것을 알 수 있다.

[단계 ③] 데이터 정제(전처리)

③-1. null 값 제거



[코드 19] 데이터프레임 null 값 제거 코드

- dropna() 함수를 이용하여 DataFrame 내 null 값을 포함하는 행을 제거한다.

[단계 ④] 알고리즘 선택

- ④-1. sklearn 내 Decision Tree 머신러닝 모델
 - Decision Tree의 경우 머신러닝 모델로 가지치기 방식을 통해 현재 값을 통해 미래 값을 예측할 때도 사용 가능한 모델이다. 자세한 설명은 앞단에 설명한 것으로 대체한다.

[단계 ⑤] 학습, 평가 데이터 준비

⑤-1. Lot List 추출

```
lot_lists = dedicated_data['Lot'].unique()
print(lot_lists)
print(len(lot_lists))
```

[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22]

22

[코드 20] 데이터프레임의 특정 컬럼으로부터 유일 값 추출 코드

- DataFrame의 컬럼명과 unique() 함수를 이용하면 해당 컬럼에 있는 값의 유일 값을 뽑을 수 있다.
- unique 값을 통해 list를 만들어 저장하였을 때, 해당 값을 print() 함수를 통해 출력시키고, list의 길이를 print(len(list)) 함수를 통해 출력시킨다.

⑤-2. Date List 추출

```
d_lists = dedicated_data['DTime'].unique()
print(d_lists)
print(len(d_lists))
```

['2021-09-06' '2021-09-07' '2021-09-08' '2021-09-09' '2021-09-10'
 '2021-09-13' '2021-09-14' '2021-09-15' '2021-09-16' '2021-09-17'
 '2021-09-23' '2021-09-24' '2021-09-27' '2021-09-28' '2021-09-29'
 '2021-09-30' '2021-10-01' '2021-10-05' '2021-10-06' '2021-10-07'
 '2021-10-08' '2021-10-12' '2021-10-13' '2021-10-14' '2021-10-15'
 '2021-10-18' '2021-10-19' '2021-10-20' '2021-10-21' '2021-10-22'
 '2021-10-25' '2021-10-26' '2021-10-27']

33

[코드 21] 데이터프레임의 특정 컬럼으로부터 유일 값 추출 코드

- DataFrame의 컬럼명과 unique() 함수를 이용하면 해당 컬럼에 있는 값의 유일 값을 뽑을 수 있다.

⑤-3. Process Data Read

```
process = pd.read_csv(os.path.join(root_dir, error_list), sep=',', encoding='utf-8')
process
```

	Date	LoT	Process Rate
0	2021-09-06	1	96.38
1	2021-09-06	2	97.40
2	2021-09-06	3	95.40
3	2021-09-06	4	96.35
4	2021-09-06	5	94.77
...	...	...	...
721	2021-10-27	18	97.29
722	2021-10-27	19	97.21
723	2021-10-27	20	98.38
724	2021-10-27	21	98.36
725	2021-10-27	22	96.03

726 rows × 3 columns

[코드 22] csv 파일로부터 데이터프레임 생성 코드

- Output Value로 사용될 Process Rate를 가져오기 위해 csv 파일로부터 데이터를 읽는다.

⑤-4. Process Data List 추출

```
lot_process_lists = process['LoT'].unique()
d_process_lists = process['Date'].unique()
print("Unique LoT List :", lot_process_lists)
print("Unique Date List :", d_process_lists)
```

```
Unique LoT List : [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22]
Unique Date List : ['2021-09-06' '2021-09-07' '2021-09-08' '2021-09-09' '2021-09-10'
'2021-09-13' '2021-09-14' '2021-09-15' '2021-09-16' '2021-09-17'
'2021-09-23' '2021-09-24' '2021-09-27' '2021-09-28' '2021-09-29'
'2021-09-30' '2021-10-01' '2021-10-05' '2021-10-06' '2021-10-07'
'2021-10-08' '2021-10-12' '2021-10-13' '2021-10-14' '2021-10-15'
'2021-10-18' '2021-10-19' '2021-10-20' '2021-10-21' '2021-10-22'
'2021-10-25' '2021-10-26' '2021-10-27']
```

[코드 23] 학습 시 사용되는 데이터를 만들기 위한 코드

- 학습 시 데이터를 맞추기 위해 ⑤-1, ⑤-2의 과정을 거친다.

⑤-5. Train/Test Data Set Make

```
X_data = pd.DataFrame(columns=['pH', 'Temp', 'LoT', 'Process'])
```

[코드 24] 학습용 데이터프레임 생성 코드

- Training Data를 만들기 위한 DataFrame을 생성한다.

```

for d in d_lists:
    for lot in lot_lists:
        tmp = dedicated_data[(dedicated_data['DTime']==d)&(dedicated_data['LoT']==lot)]
        tmp = tmp[['pH', 'Temp', 'LoT']]
        process_val = process[(process['Date']==d)&((process['LoT']==lot))]['Process Rate'].values
        trr = np.full((tmp['pH'].shape), process_val)
        tmp['Process'] = trr
        X_data = X_data.append(tmp)
X_data=X_data.apply(pd.to_numeric)
X_data = X_data[['LoT', 'pH', 'Temp', 'Process']]

```

[코드 25] 두 개의 데이터프레임에서 필요한 부분만 합치는 코드

- LoT, DTime을 이용하여 dedicated_data frame 데이터와 process frame 데이터를 추출한다.

X_data

	LoT	pH	Temp	Process
2021-09-06 09:01:18	1	1.02	47.18	96.38
2021-09-06 09:01:23	1	1.05	47.34	96.38
2021-09-06 09:01:28	1	1.09	48.45	96.38
2021-09-06 09:01:33	1	1.12	48.46	96.38
2021-09-06 09:01:38	1	1.15	48.47	96.38
...	...	...	...	...
2021-10-27 11:14:41	22	2.79	51.83	96.03
2021-10-27 11:14:46	22	3.62	42.20	96.03
2021-10-27 11:14:51	22	3.40	41.88	96.03
2021-10-27 11:14:56	22	3.59	40.62	96.03
2021-10-27 11:15:01	22	3.82	42.08	96.03

50094 rows × 4 columns

[코드 26] 학습용 데이터프레임 코드

- 만들어진 Training Data를 확인한다.

X_data.describe()

	LoT	pH	Temp	Process
count	50094.000000	50094.000000	50094.000000	50094.000000
mean	11.500000	2.006488	49.876127	96.064229
std	6.344352	0.551698	1.345322	3.210621
min	1.000000	1.010000	38.020000	80.780000
25%	6.000000	1.560000	49.280000	96.170000
50%	11.500000	2.000000	49.970000	96.635000
75%	17.000000	2.440000	50.630000	97.260000
max	22.000000	3.990000	54.190000	98.450000

[코드 27] 학습용 데이터프레임 요약 코드

- 만들어진 Data를 요약한다.

```
train_data, test_data = train_test_split(X_data, test_size=0.2)
```

[코드 30] 학습용 데이터, 테스트용 데이터 구분 코드

- X_data로부터 학습용/테스트용 데이터 생성한다.
- 학습용/테스트용 데이터셋 비율을 (80 : 20)로 나눈다.

```
train_data.describe()
```

	LoT	pH	Temp	Process
count	40075.000000	40075.000000	40075.000000	40075.000000
mean	11.487985	2.006736	49.874776	96.049513
std	6.348316	0.551525	1.348762	3.237408
min	1.000000	1.010000	38.020000	80.780000
25%	6.000000	1.560000	49.280000	96.170000
50%	11.000000	2.000000	49.970000	96.630000
75%	17.000000	2.440000	50.640000	97.260000
max	22.000000	3.990000	54.190000	98.450000

[코드 31] 학습용 데이터셋 요약 코드

- 학습용 데이터셋 요약을 진행한다.

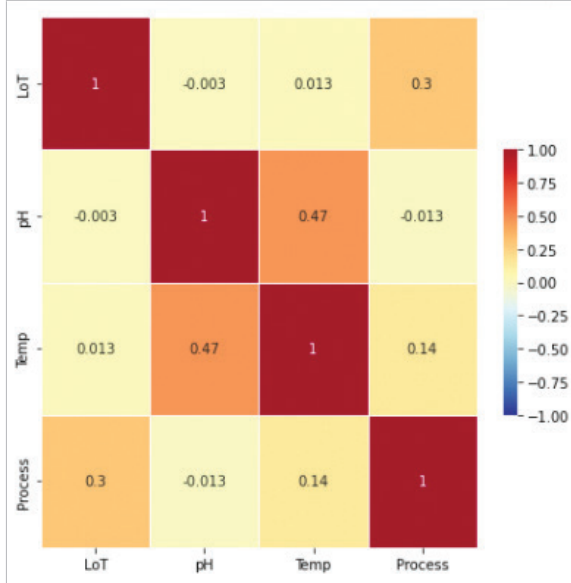
```
train_data.corr()
```

	LoT	pH	Temp	Process
LoT	1.000000	-0.003018	0.012942	0.300762
pH	-0.003018	1.000000	0.470596	-0.012510
Temp	0.012942	0.470596	1.000000	0.144571
Process	0.300762	-0.012510	0.144571	1.000000

[코드 32] 학습용 데이터셋 상관관계 분석

- 학습용 데이터셋 상관관계 분석을 진행한다.

```
fig, ax = plt.subplots(figsize=(7,7))
sns.heatmap(train_data.corr(), cmap='RdYlBu_r', annot=True, linewidths=0.5, cbar_kws={"shrink":.5}, vmin=-1, vmax=1)
plt.show()
```



[코드 33] 학습용 데이터셋 상관관계 시각화 코드

- 학습용 데이터셋 상관관계 결과 시각화를 진행한다.
- 상관관계의 경우 값이 클수록 상관관계가 있다고 판단할 수 있는 척도가 되며, 위 결과를 통해서 알 수 있는 것은 학습용 데이터셋의 상관관계의 경우 pH와 Temp의 상관관계는 0.47로 낮은 상관관계를 보이는 것을 알 수 있다.

```
test_data.describe()
```

	LoT	pH	Temp	Process
count	10019.000000	10019.000000	10019.000000	10019.000000
mean	11.548059	2.005498	49.881530	96.123089
std	6.328560	0.552417	1.331525	3.100620
min	1.000000	1.010000	38.200000	80.780000
25%	6.000000	1.560000	49.280000	96.190000
50%	12.000000	2.000000	49.970000	96.640000
75%	17.000000	2.440000	50.630000	97.260000
max	22.000000	3.990000	54.020000	98.450000

[코드 34] 테스트용 데이터셋 요약 코드

- 테스트용 데이터셋 요약을 진행한다.

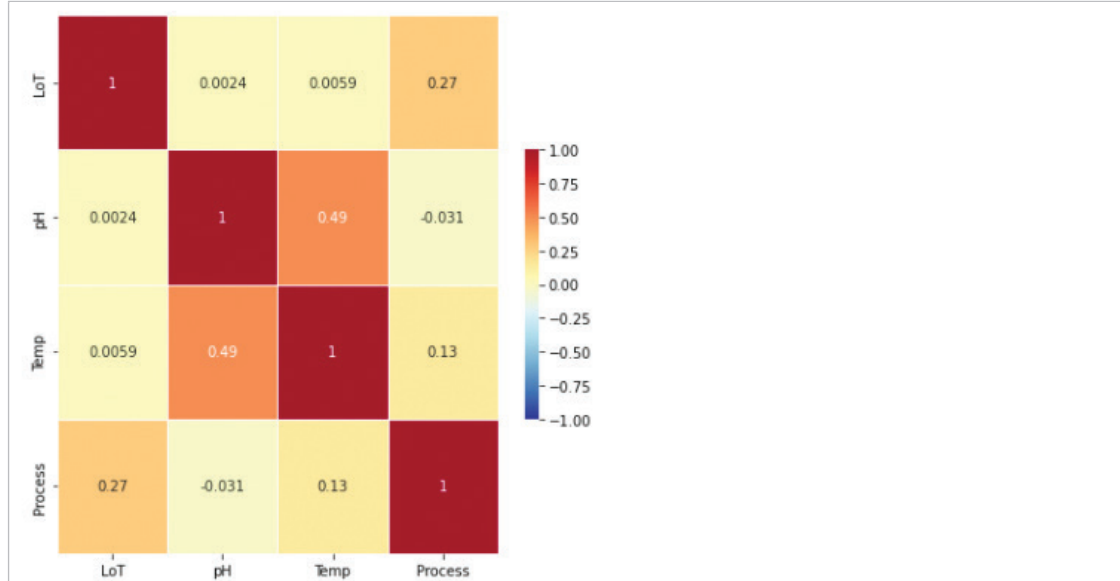
```
test_data.corr()
```

	LoT	pH	Temp	Process
LoT	1.000000	0.002380	0.005910	0.273587
pH	0.002380	1.000000	0.492467	-0.031070
Temp	0.005910	0.492467	1.000000	0.134166
Process	0.273587	-0.031070	0.134166	1.000000

[코드 35] 테스트용 데이터셋 상관관계 분석 코드

- 테스트용 데이터셋 상관관계 분석을 진행한다.

```
fig, ax = plt.subplots(figsize=(7,7))
sns.heatmap(test_data.corr(), cmap='RdYlBu_r', annot=True, linewidths=0.5, cbar_kws={'shrink':.5}, vmin=-1, vmax=1)
plt.show()
```



[코드 36] 테스트용 데이터셋 상관관계 결과 시각화 코드

- 테스트용 데이터셋 상관관계 결과 시각화를 진행한다.
- 상관관계의 경우 값이 클수록 상관관계가 있다고 판단할 수 있는 척도가 되며, 위 결과를 통해서 알 수 있는 것은 테스트용 데이터셋의 상관관계의 경우 pH와 Temp의 상관관계는 0.49로 낮은 상관관계를 보이는 것을 알 수 있다.

[단계 ⑥] 모델링

⑥-1. Decision Tree 모델 모델링

```
clf = tree.DecisionTreeRegressor()
```

[코드 37] Decision Tree Regressor 모델 모델링 코드

- 가장 기본적인 Decision Tree 모델을 사용하였으며, 별다른 Hyper-Parameter를 사용하지 않았다. 이는 최대의 결과가 나올 수 있도록 모든 제한을 풀어둔 모델이 생성될 것이다.

[단계 ⑦] 모델 훈련

⑦-1. Decision Tree 모델 학습

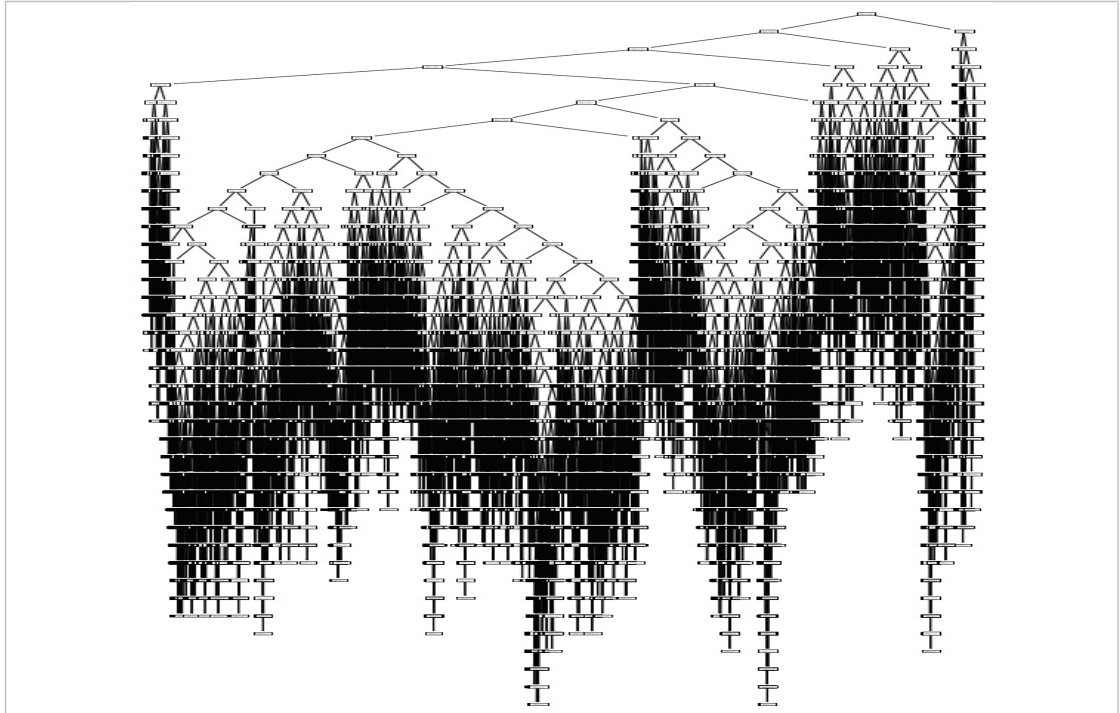
```
clf = clf.fit(train_data[['pH','Temp']], train_data[['Process']])
```

[코드 38] Decision Tree 모델 학습

- 학습은 fit() 함수를 통해 진행하며, Input Data, Output Data 순으로 인자를 넣는다.
- 학습 시 따로 출력되는 결과물이 없다.

⑦-2. Decision Tree 모델 시각화

```
vis = True
if vis:
    plt.figure(figsize=(10, 30))
    tree.plot_tree(clf)
    plt.show()
```



[코드 39] Decision Tree 모델 시각화

- Decision Tree 모델을 tree.plot_tree() 함수를 통해 시각화 진행하였으나, 복잡한 모델이 생성된 것을 확인할 수 있다.

[단계 ⑧] 모델 튜닝

⑧-1. Decision Tree 모델 튜닝

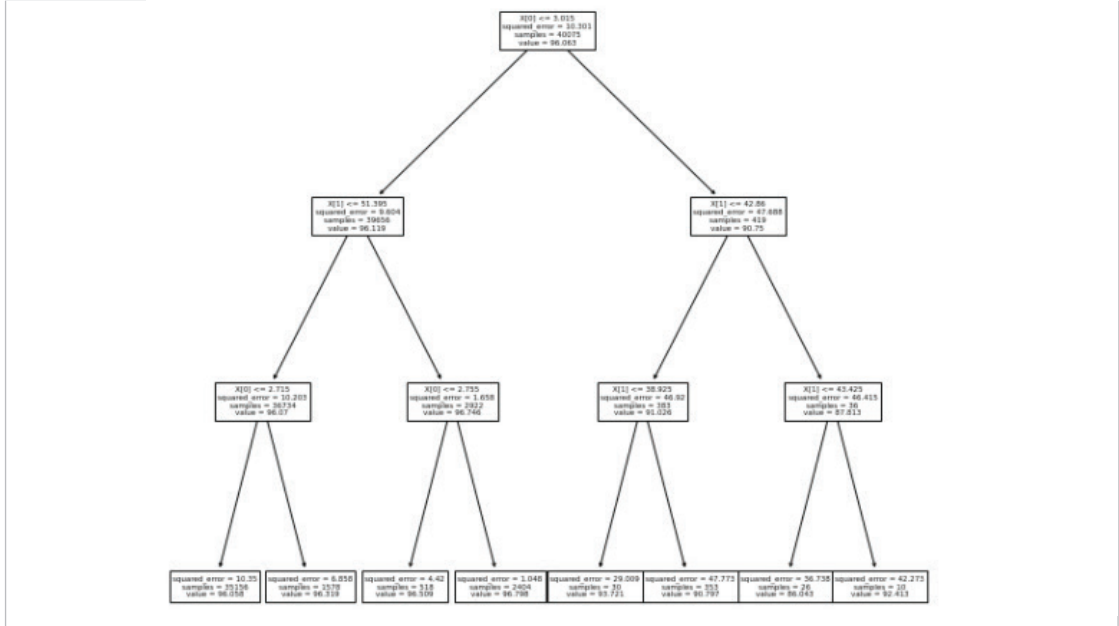
- Decision Tree 모델 튜닝 시 사용할 수 있는 Hyper-Parameter에는 max_depth, min_samples_split, max_leaf_nodes가 있다. 이를 사용자가 원하는대로 수정 가능하며, 가장 좋은 파라미터를 찾기 위해서는 많은 시도와 노력이 필요하다.

```
new_clf = tree.DecisionTreeRegressor(max_depth=3)
new_clf = new_clf.fit(train_data[['pH', 'Temp']], train_data[['Process']])
```

[코드 40] Decision Tree 모델 튜닝

- max_depth를 3으로 변경하여 튜닝 및 학습을 진행한다.

```
plt.figure(figsize=(10, 10))
tree.plot_tree(new_clf)
plt.show()
```



[코드 41] 튜닝된 Decision Tree 모델 시각화

- plot_model() 함수를 통해 max_depth가 3인 모델 시각화를 진행한다.

[단계 ⑨] 모델 평가 및 해석

⑨-1. Decision Tree 모델 평가

```
predicted_data = clf.predict(test_data[['pH', 'Temp']])
print('Decision Tree Model Predict : ', predicted_data)
rmse = sqrt(mean_squared_error(test_data['Process'], predicted_data))
print('Decision Tree Model RMSE : ', rmse)
```

```
Decision Tree Model Predict : [96.95333333 97.275      96.6425      ... 97.215      96.84
96.73142857]
Decision Tree Model RMSE : 3.91421344525677
```

[코드 42] Decision Tree 모델 예측 및 RMSE 코드

- Decision Tree Regressor Model의 평가를 위해 테스트 데이터셋을 사용하여 Predict 함수를 통해 예측하고, 테스트 데이터셋 target 데이터와 비교하여 RMSE 계산 결과 약 3.91로 나타난다.

```
y_test = test_data['Process']
y_test = [round(y, 0) for y in y_test]
y_pred = [round(y, 0) for y in predicted_data]
print('accuracy = ', accuracy_score(y_test, y_pred))
```

```
accuracy = 0.33546262102006186
```

[코드 43] Decision Tree 모델 accuracy

- accuracy_score 함수를 이용하여 accuracy 값을 도출한다.

- 이를 통해 확인할 수 있는 것은 해당 모델의 경우 정확도가 낮다는 것이다.
- RMSE와 비교하였을 때, 알 수 있는 점은 Output Data의 분포가 크지 않아서 일정 범위에서 예측했기 때문에 RMSE는 작지만, 정확한 값을 예측하지는 않는다는 것이다.

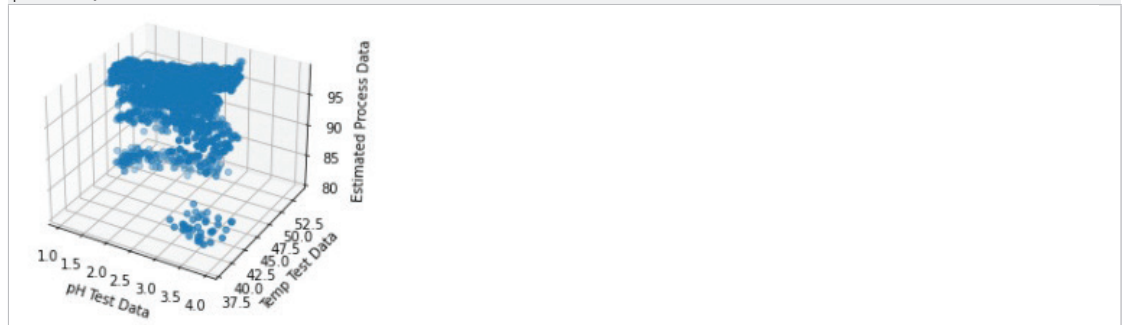
```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
81.0	0.00	0.00	0.00	55
82.0	0.05	0.02	0.02	133
83.0	0.13	0.04	0.07	205
84.0	0.03	0.02	0.02	129
86.0	0.00	0.00	0.00	0
87.0	0.00	0.00	0.00	0
88.0	0.00	0.00	0.00	0
89.0	0.00	0.00	0.00	0
90.0	0.00	0.00	0.00	0
91.0	0.00	0.00	0.00	0
92.0	0.00	0.00	0.00	0
93.0	0.00	0.00	0.00	0
94.0	0.00	0.00	0.00	0
95.0	0.00	0.00	0.00	36
96.0	0.39	0.27	0.32	3637
97.0	0.42	0.53	0.47	4105
98.0	0.21	0.11	0.14	1719
accuracy			0.34	10019
macro avg	0.07	0.06	0.06	10019
weighted avg	0.35	0.34	0.33	10019

[코드 44] Decision Tree 모델 검토 값 요약 정리

- classification_report 함수를 이용하여 각 target value에 따른 결과값을 도출한다. 해당 모델의 경우 데이터 예측을 위한 것이기 때문에 계산되는 결과의 범위가 넓어 float -> int로 변형하여 해당 과정을 진행하였다.

```
fig = plt.figure()
ax = fig.gca(projection='3d')
ax.scatter(test_data[pH], test_data[Temp], predicted_data)
ax.set_xlabel('pH Test Data')
ax.set_ylabel('Temp Test Data')
ax.set_zlabel('Estimated Process Data')
plt.show()
```



[코드 45] Decision Tree 모델 예측 3D 시각화

- 모델의 결과를 3D로 표현 결과 위와 같이 시각화할 수 있다.

⑨-2. Decision Tree 모델 해석

- Target Value가 약 90대로 나타나기 때문에 오차율은 약 3% 정도로 볼 수 있다.
- Input Data로 특정 pH, 온도를 입력하였을 때, 나오는 출력값을 통해 해당 pH와 온도에서 어떠한 품질 결과가 나타나는지 확인할 수 있다.
- 이를 통해, 사용자가 원하는 품질을 얻기 위한 최적 공정을 역산으로 얻을 수 있다.

⑨-3. 튜닝한 Decision Tree 모델 평가

```
predicted_data = new_clf.predict(test_data[['pH', 'Temp']])
print('Decision Tree Model Predict : ', predicted_data)
rmse = sqrt(mean_squared_error(test_data['Process'], predicted_data))
print('Decision Tree Model RMSE : ',rmse)
Decision Tree Model Predict : [96.05840027 96.05840027 96.79752496 ... 96.05840027 96.05840027
96.05840027]
Decision Tree Model RMSE : 3.1641126912353434
```

[코드 46] Tuned Decision Tree 모델 예측 및 RMSE 코드

- Decision Tree Regressor Model의 평가를 위해 테스트 데이터셋을 사용하여 Predict 함수를 통해 예측하고, 테스트 데이터셋 target 데이터와 비교하여 RMSE 계산 결과 약 3.16로 나타난다.

```
y_test = test_data['Process']
y_test = [round(y, 0) for y in y_test]
y_pred = [round(y, 0) for y in predicted_data]
print("accuracy = ", accuracy_score(y_test, y_pred))
accuracy = 0.3652061083940513
```

[코드 47] Tuned Decision Tree 모델 accuracy

- accuracy_score 함수를 이용하여 accuracy 값을 도출한다.
- 이를 통해 확인할 수 있는 것은 해당 모델의 경우 정확도가 낮다는 것이다.
- RMSE와 비교하였을 때, 알 수 있는 점은 Output Data의 분포가 크지 않아서 일정 범위 내에서 예측했기 때문에 RMSE는 작지만, 정확한 값을 예측하지는 않는다는 것이다.

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
81.0	0.00	0.00	0.00	55
82.0	0.00	0.00	0.00	133
83.0	0.00	0.00	0.00	205
84.0	0.00	0.00	0.00	129
86.0	0.00	0.00	0.00	0
91.0	0.00	0.00	0.00	0
92.0	0.00	0.00	0.00	0
94.0	0.00	0.00	0.00	0
95.0	0.00	0.00	0.00	36
96.0	0.36	0.92	0.52	3637
97.0	0.43	0.07	0.13	4105
98.0	0.00	0.00	0.00	1719
accuracy			0.37	10019
macro avg	0.07	0.08	0.05	10019
weighted avg	0.31	0.37	0.24	10019

[코드 48] Tuned Decision Tree 모델 검토 값 요약 정리

- classification_report 함수를 이용하여 각 target value에 따른 결과값을 도출한다. 해당 모델의 경우 데이터 예측을 위한 것이기 때문에 계산되는 결과의 범위가 넓어 float -> int로 변형하여 해당 과정을 진행하였다.

```
fig = plt.figure()
ax = fig.gca(projection='3d')
ax.scatter(test_data[pH], test_data[Temp], predicted_data)
ax.set_xlabel('pH Test Data')
ax.set_ylabel('Temp Test Data')
ax.set_zlabel('Estimated Process Data')
plt.show()
```



[코드 49] Tuned Decision Tree 모델 예측 3D 시각화

- 모델의 결과를 3D로 표현 결과 위와 같이 시각화할 수 있다.

⑨-4. Tuned Decision Tree 모델 해석

- Target Value가 약 90대로 나타나기 때문에 오차율은 약 3% 정도로 볼 수 있다.
- 하지만 정확도가 0.37 정도로 나타나기 때문에 제대로 학습이 진행되었다고는 볼 수 없는 모델로 나타났다.
- Input Data로 특정 pH, 온도를 입력하였을 때, 나오는 출력값을 통해 해당 pH와 온도에서 어떠한 품질 결과가 나타나는지 확인할 수 있다.
- 이를 통해, 사용자가 원하는 품질을 얻기 위한 최적 공정을 역산으로 얻을 수 있다.

3. 유사 타현장의 「공정운영 최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조 현장 소개

- 도금에는 많은 방식이 존재한다. 크게 나누어 전기도금, 화학도금, 용융도금 등이 있으며, 화학도금은 전기가 통하지 않는 제품을 화학도금액에 넣어 도금하는 것을 의미하고, 용융도금은 비철 금속을 녹여서 도금할 제품을 넣어 용융금속의 피막을 입히는 것을 의미하며, 전기도금에는 동도금, 니켈도금, 크롬도금, 아연도금, 주석도금 등이 있다. 이 중에서 공정 운영 최적화 AI 데이터셋의 경우 산제 전처리 공정을 의미하며 유사한 프로세스를 갖는 전기도금 공정에서 사용될 수 있다.

3.2 본 「공정운영 최적화 AI 데이터셋」 분석을 원용하여 타 제조 현장 적용 시, 주요 고려사항

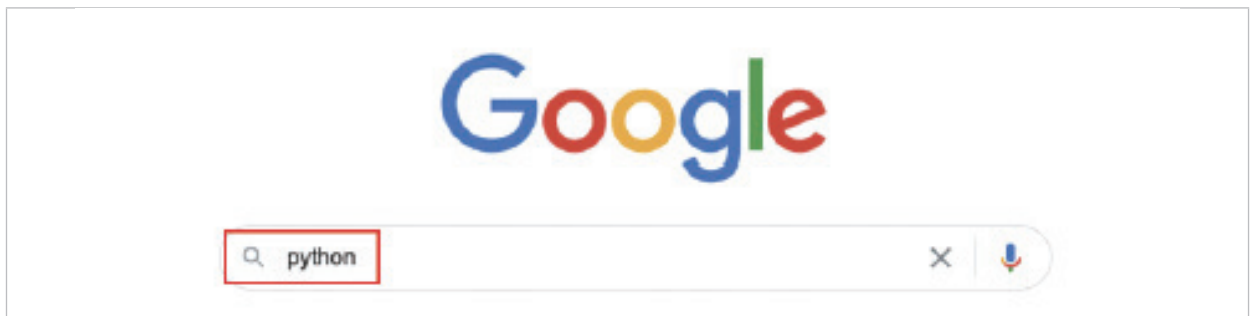
- 공정별 불량데이터 추적이 가능한 제품이 필요하다. 도금공정 특성상 도금이 완료된 제품이 한 번에 나오는데 같은 공정을 진행한 제품에 대한 불량데이터 관리가 필요하다.
- 수집되는 공정 데이터와 불량데이터 관리가 필요하다. 제품이 완료된 후 나오는 불량데이터와 해당 제품에 대한 공정 데이터의 매칭 시 Lot NO와 같은 특정 기준을 통해 통합 관리가 필요하다. 이러한 데이터 매칭을 위해 현장 실무자와 AI 분석 전문가가 이러한 상황을 미리 논의하여 정리한 후 데이터를 수집하는 시스템을 환경을 구축해야 한다.

3 부록 _ 분석환경 구축을 위한 설치 가이드

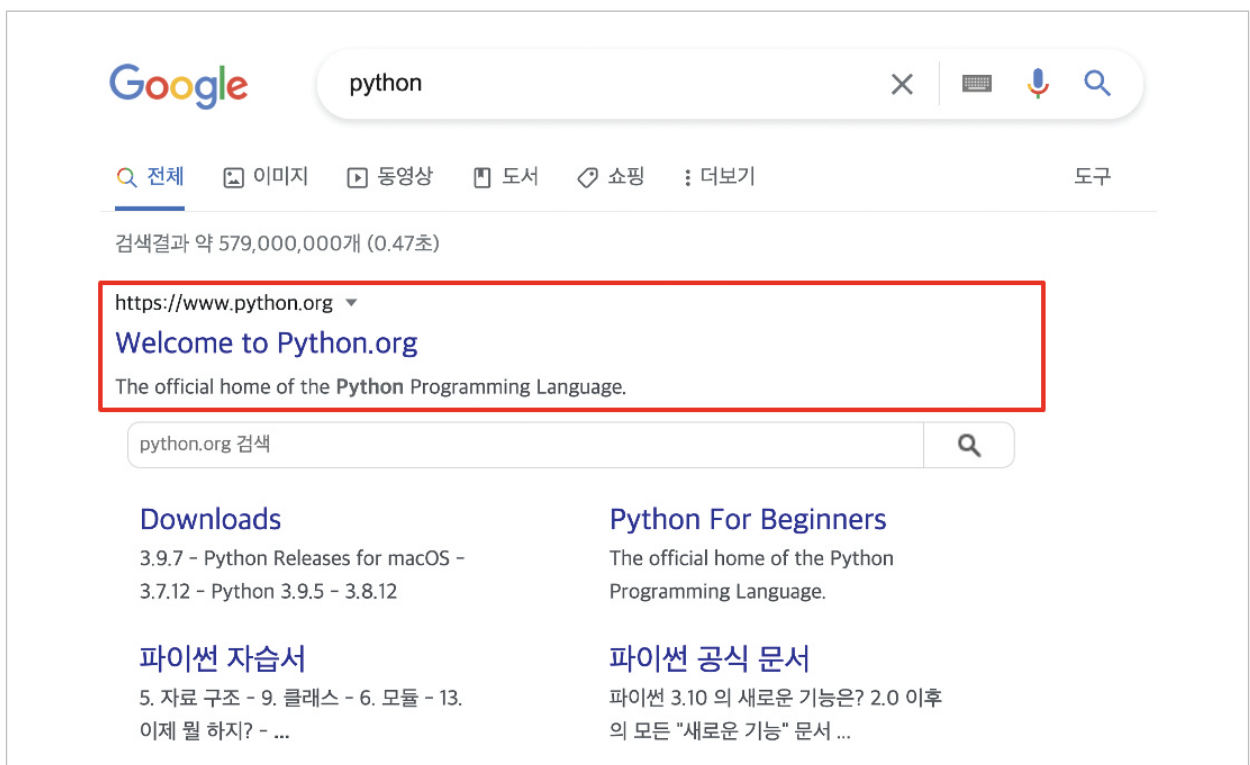
1. 파이썬(Python) 설치

파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

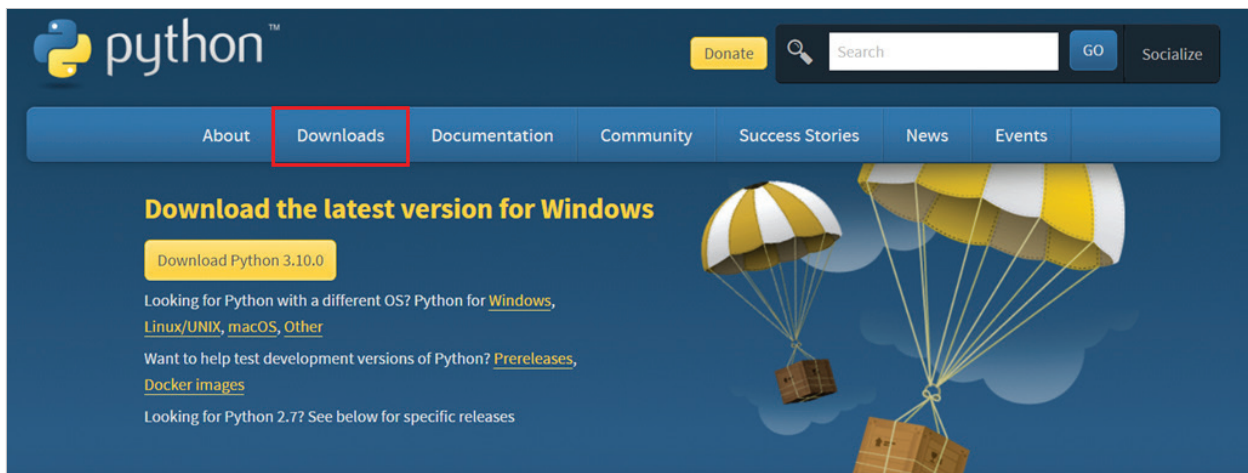
① google.com 등의 검색 엔진에 'python'을 검색한다.



② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



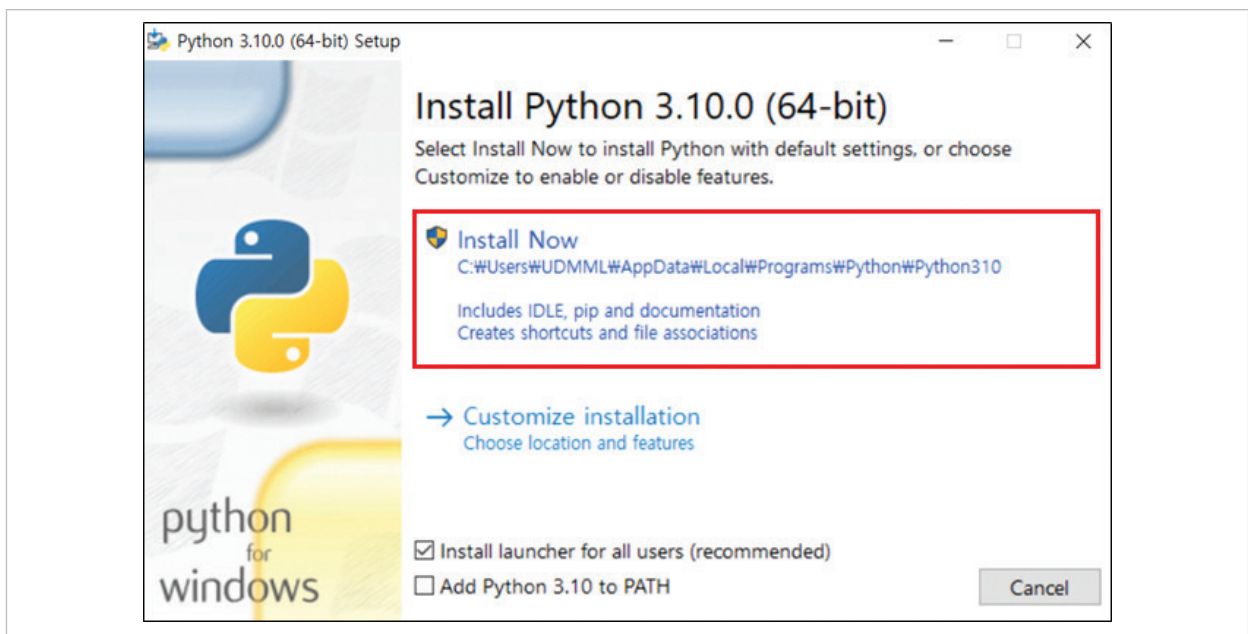
③ 클릭후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



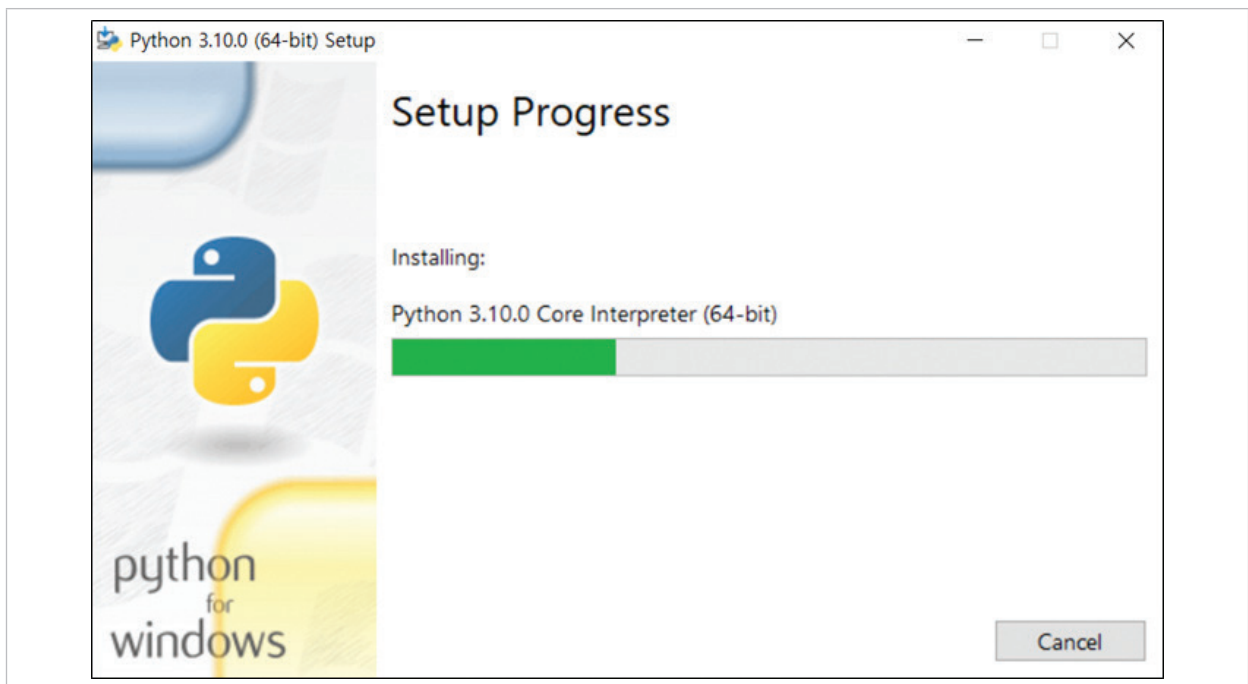
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 3.10.0 버전은 업데이트 될 수 있다)



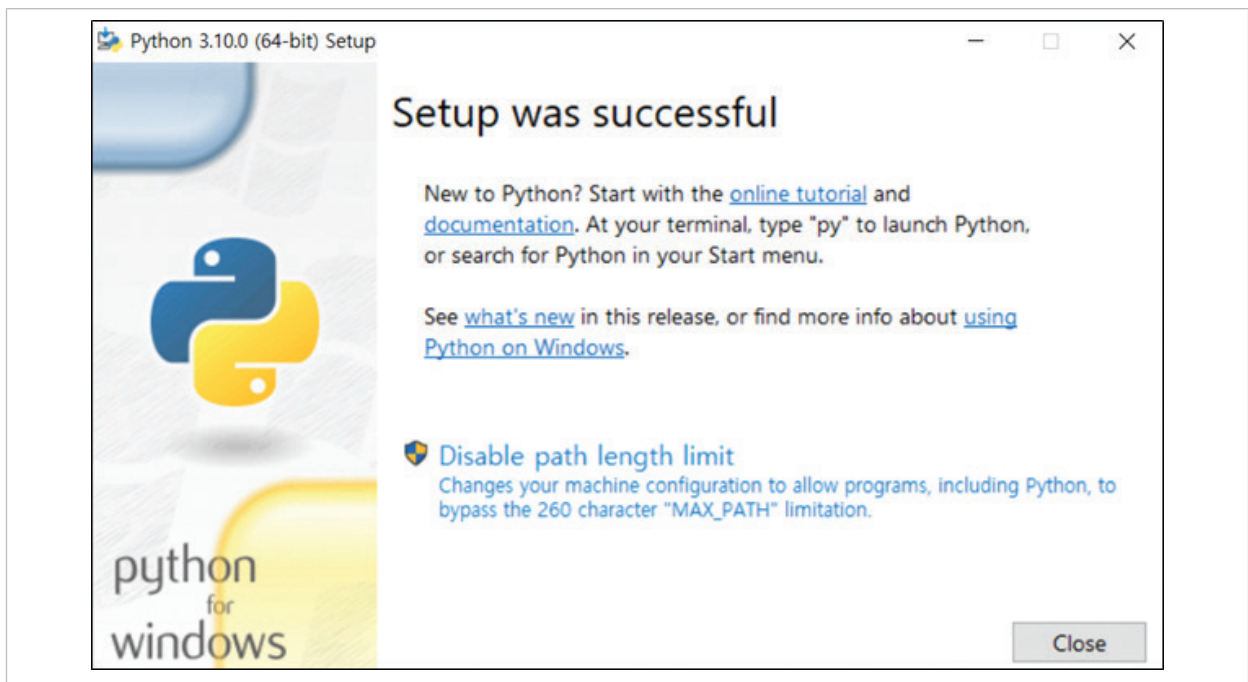
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



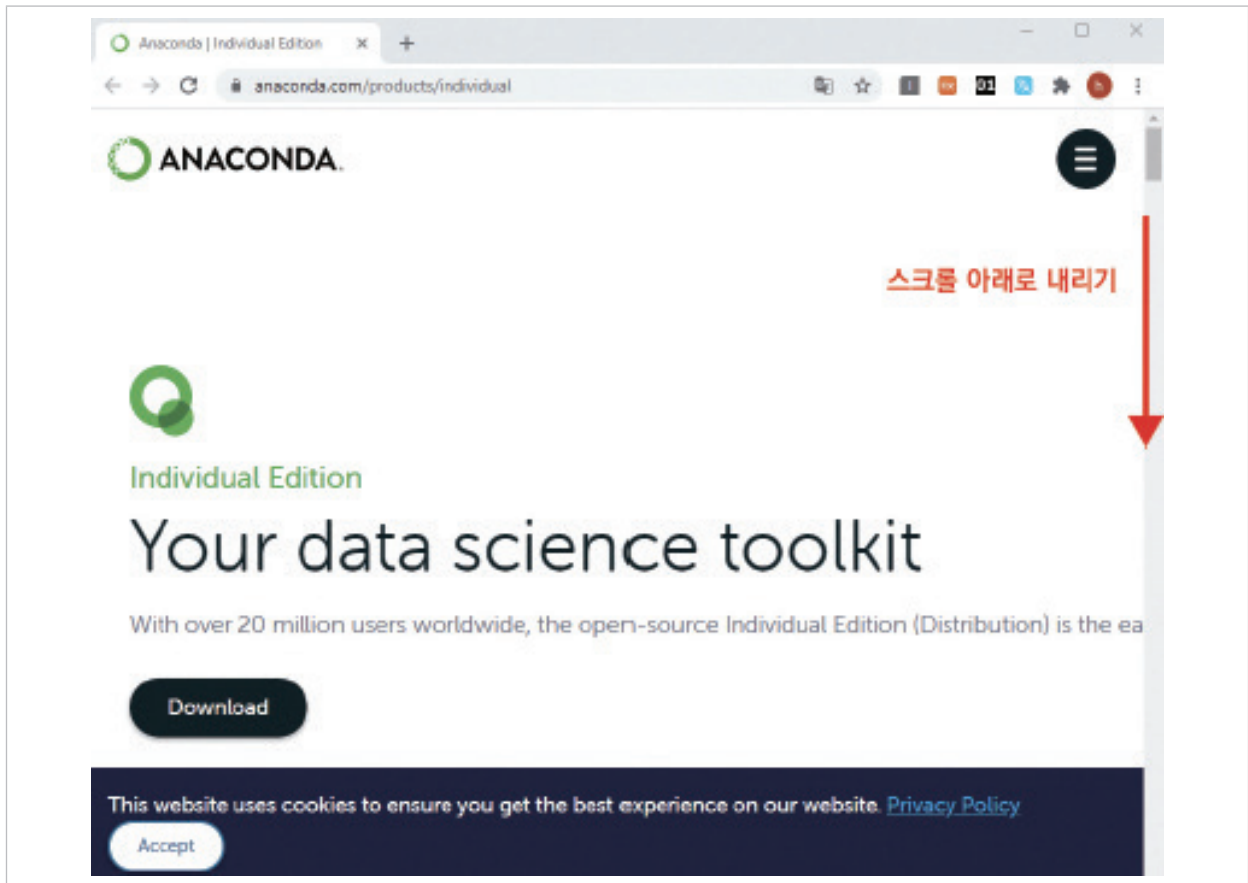
⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]



2. 아나콘다(Anaconda) 설치

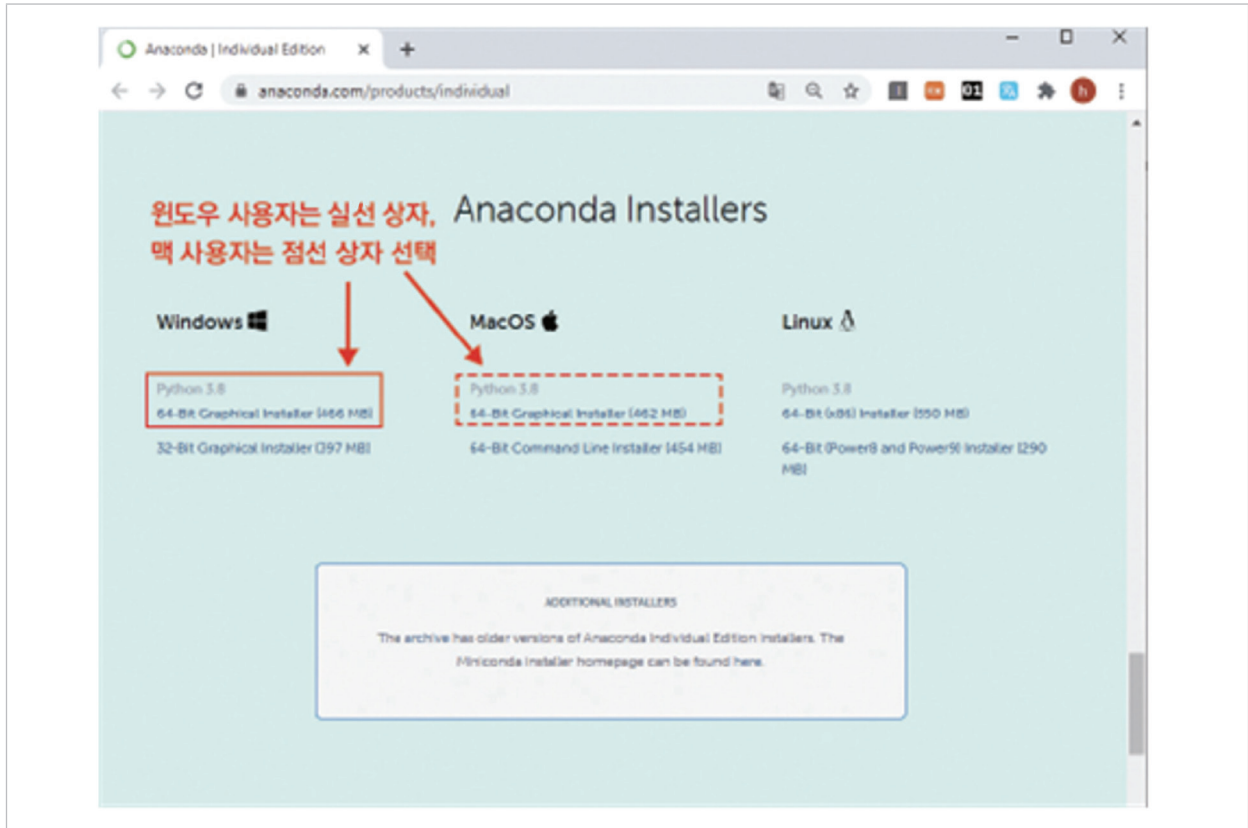
아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



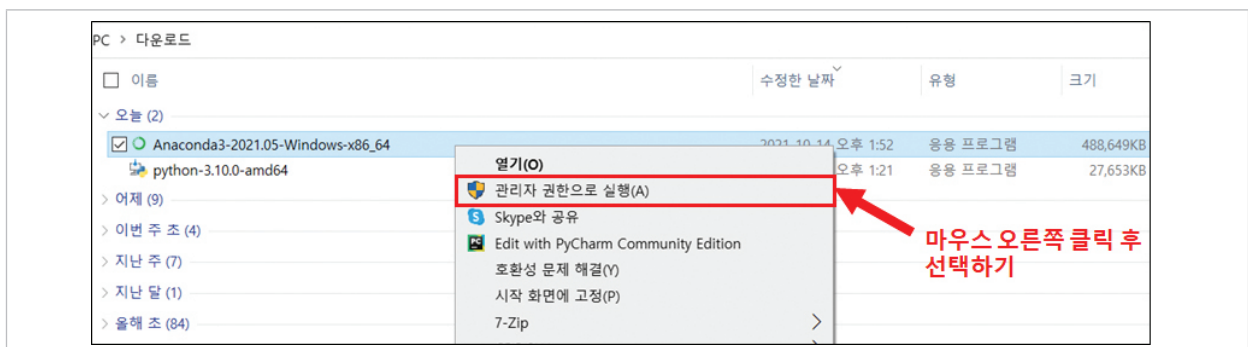
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer

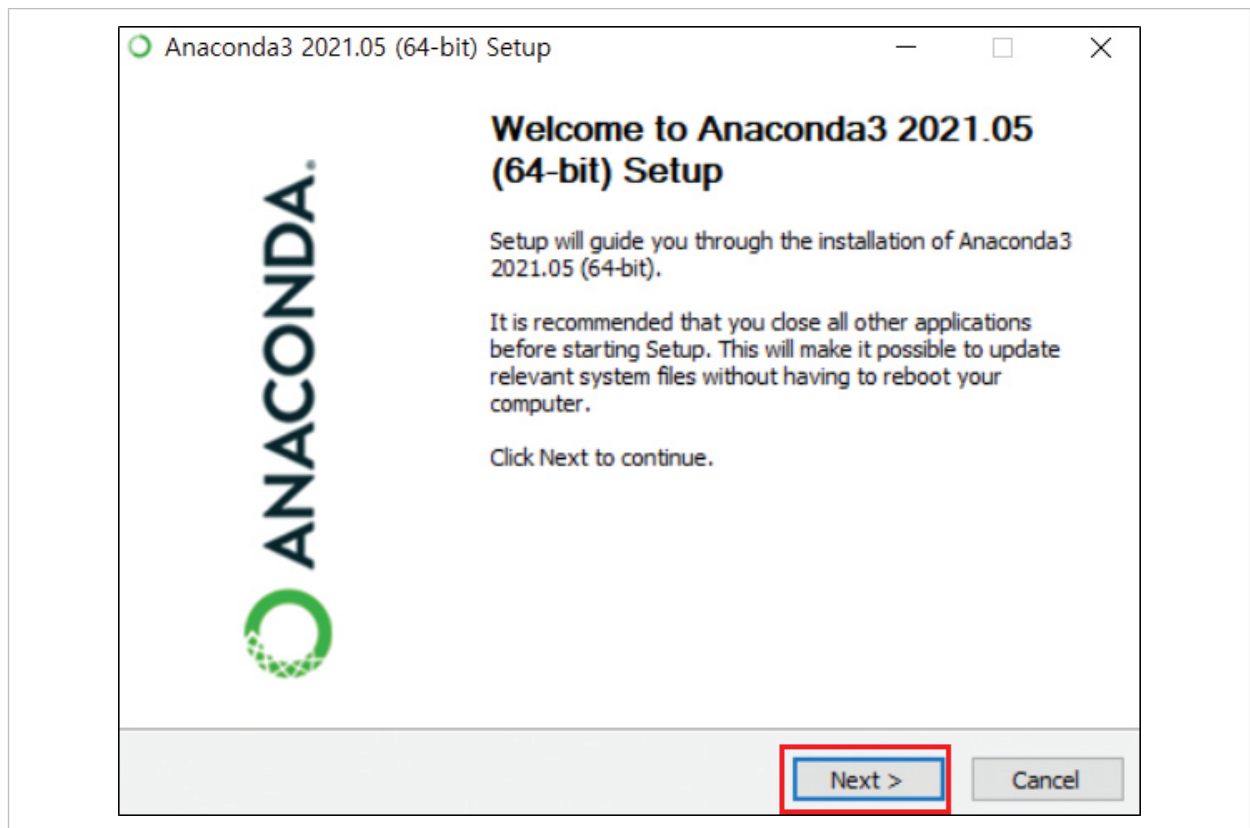


③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽을 클릭한 후, 방패모양의 '관리자 권한으로 실행'을 클릭한다.

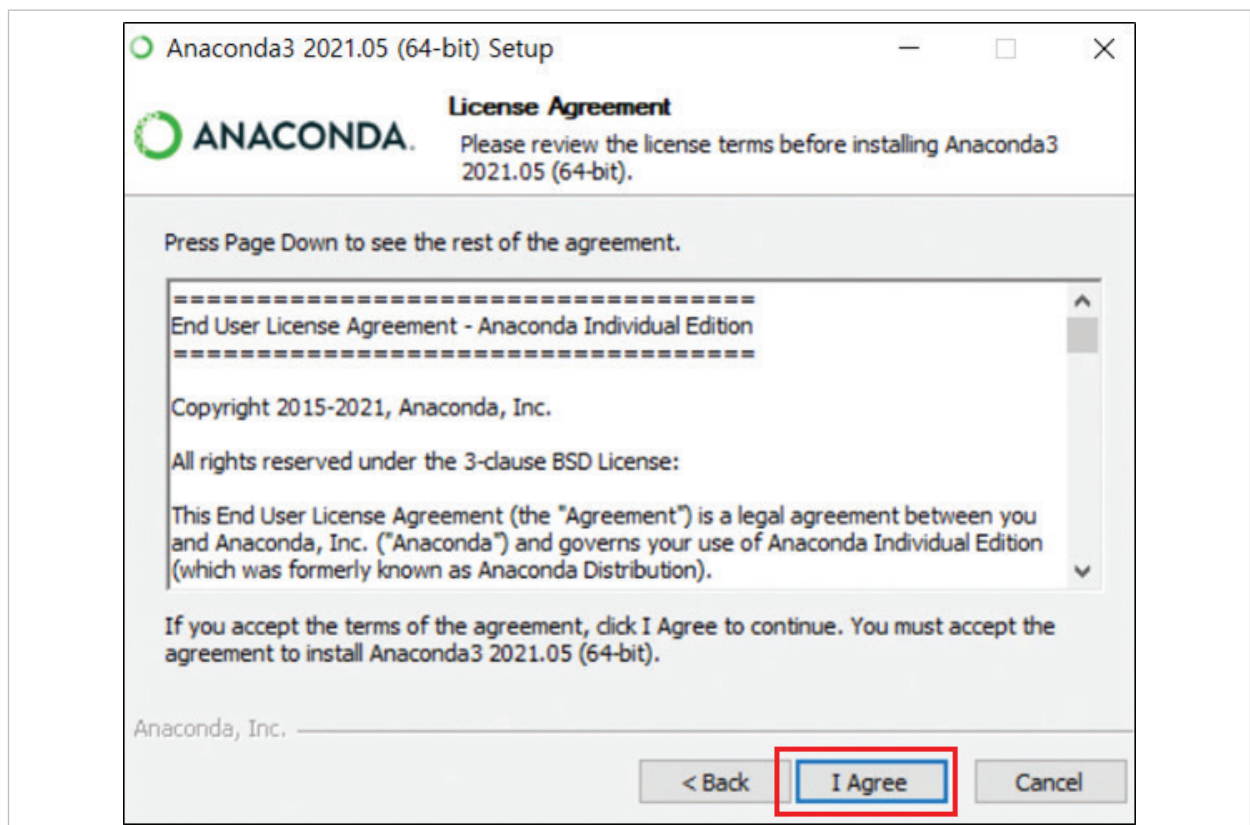
- ▶ (예) '다운로드' 폴더로 아나콘다를 다운 받은 경우



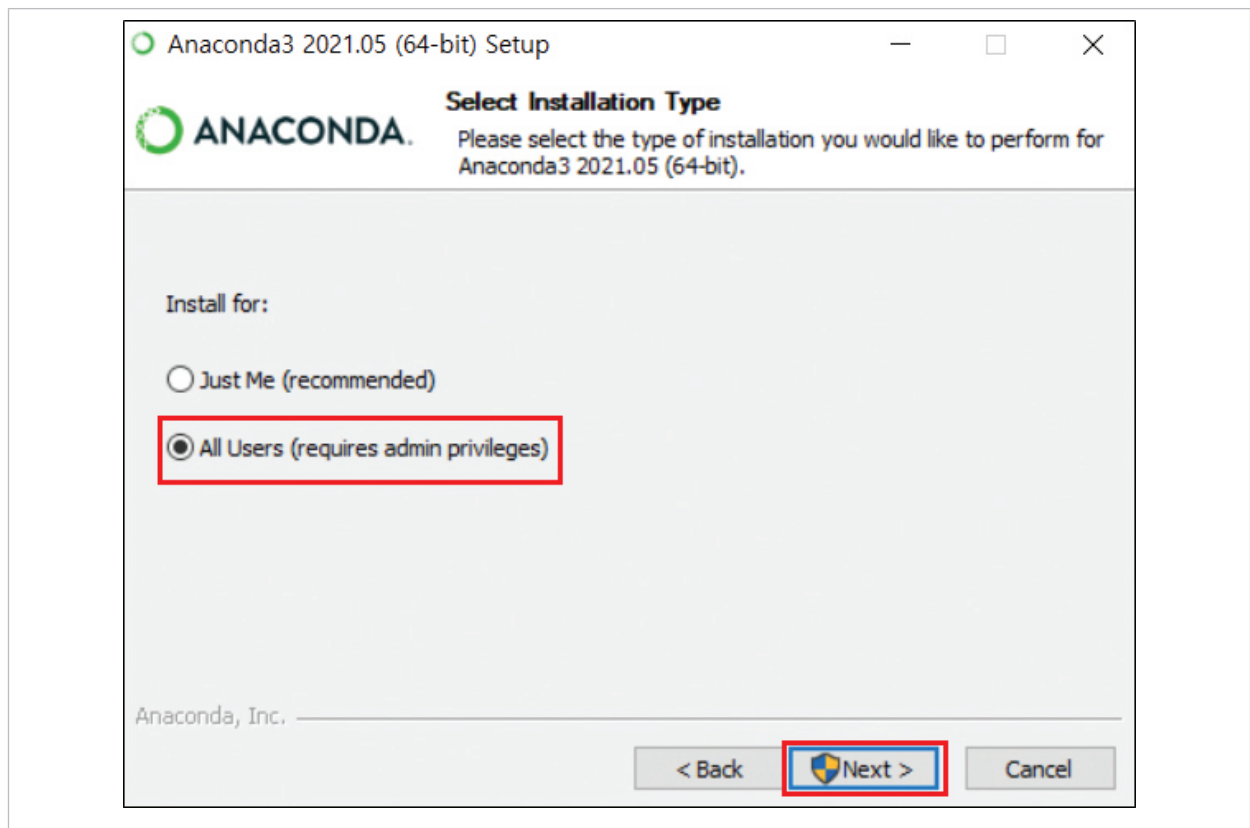
④ 파일을 실행 한 후, 'Next' 버튼을 클릭한다.



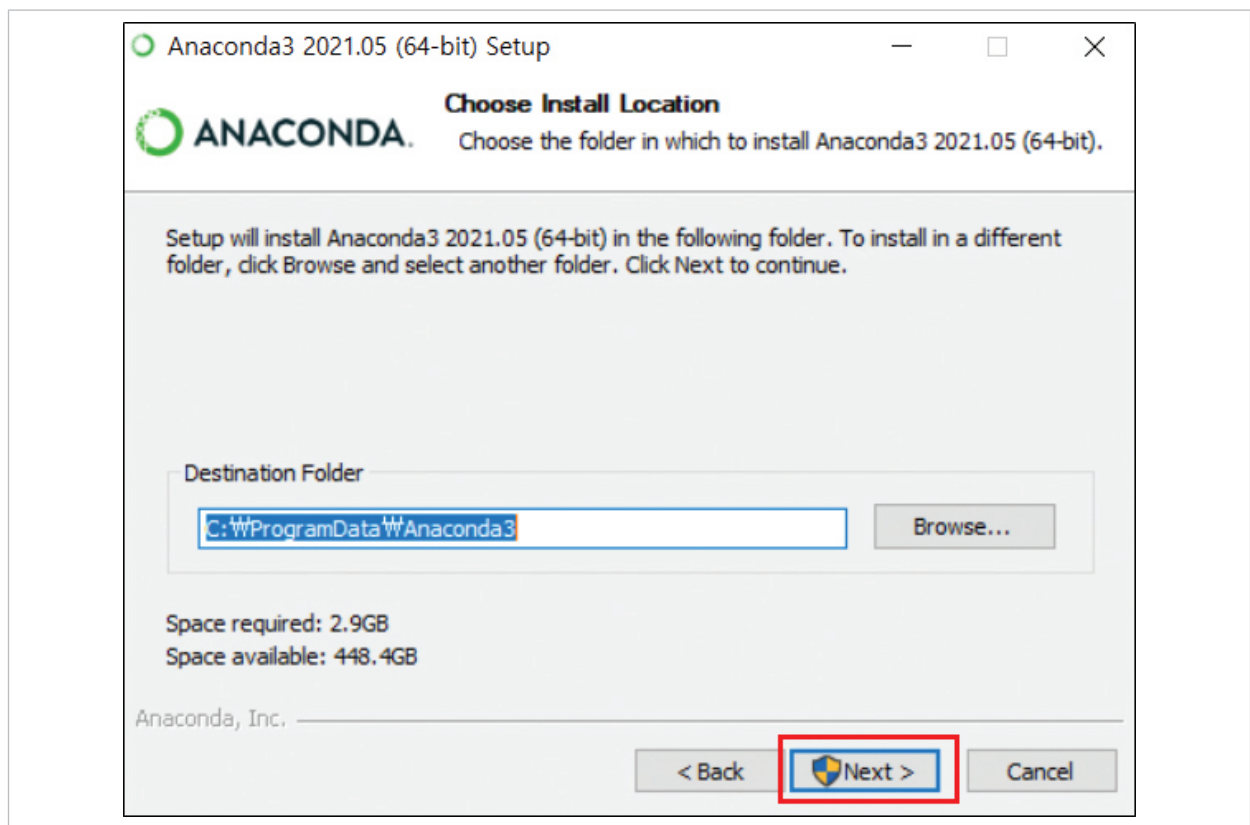
⑤ 다음 창이 나타나면 'I Agree'를 선택한다.



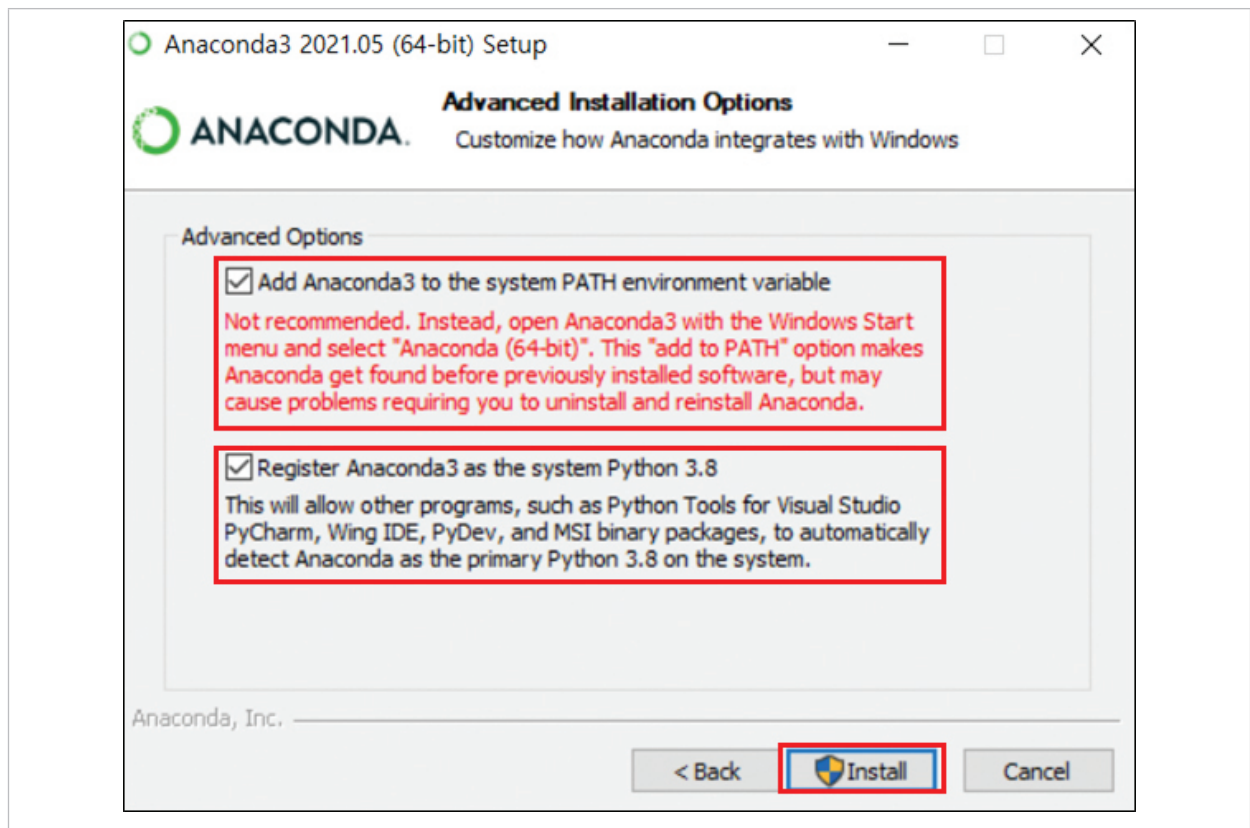
⑥ 셋팅 창이 뜨면 화면의 'All Users'를 선택 후 아래의 'Next'를 클릭한다.



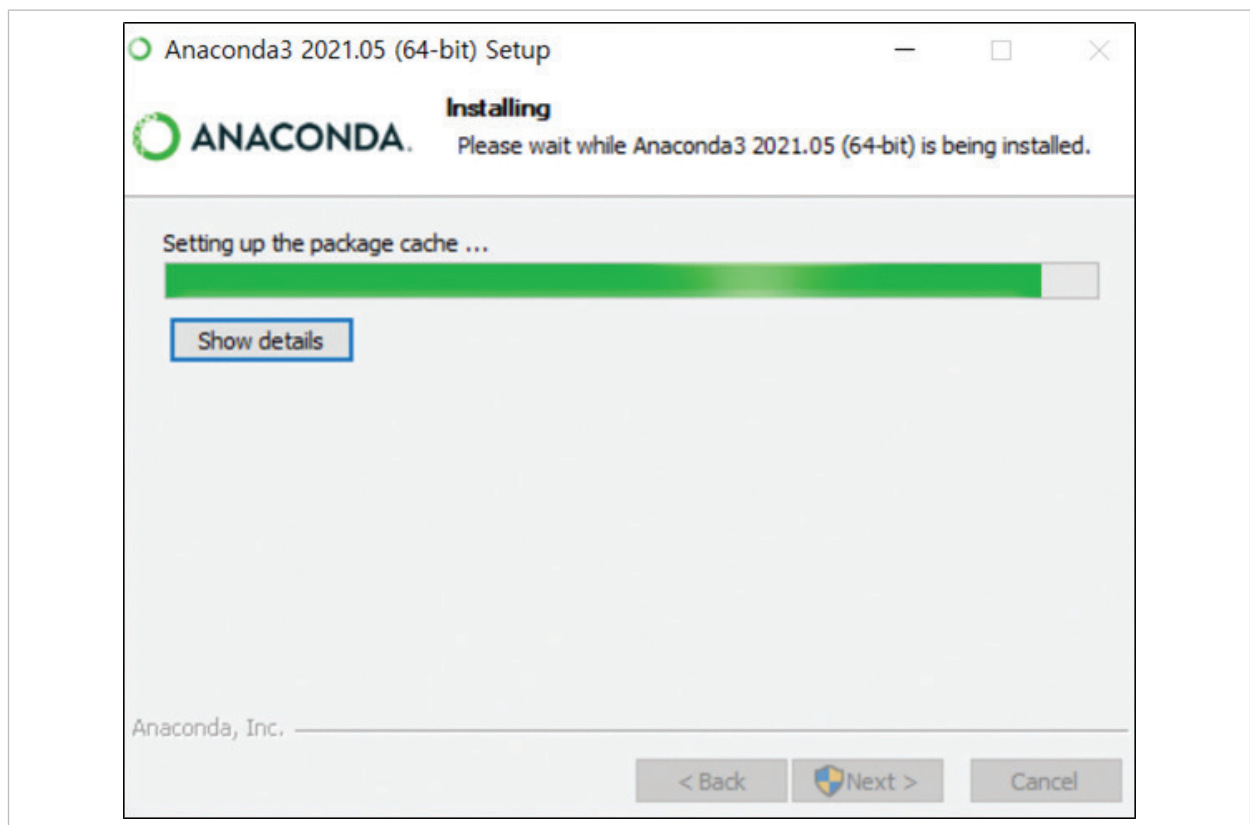
⑦ 다운로드 받을 경로를 물어보는 창이 뜨면, 아래의 'Next'를 클릭한다.



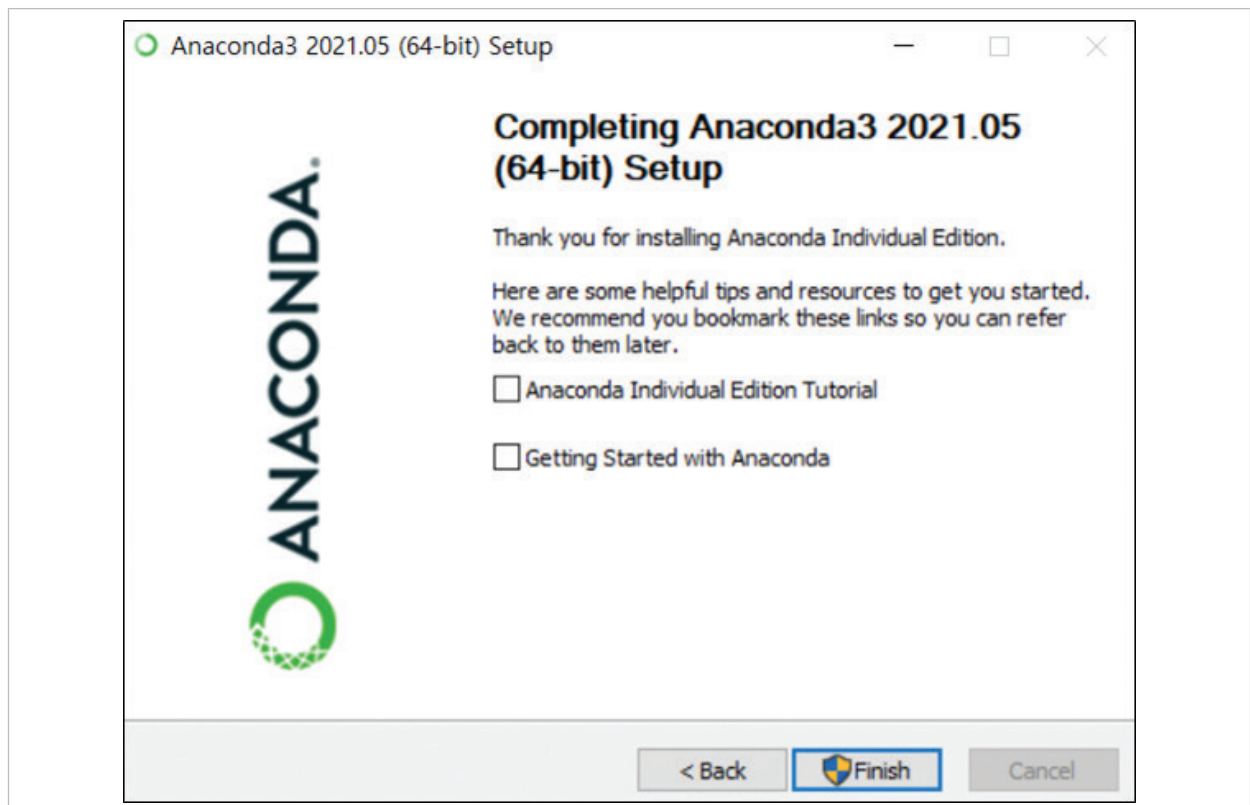
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



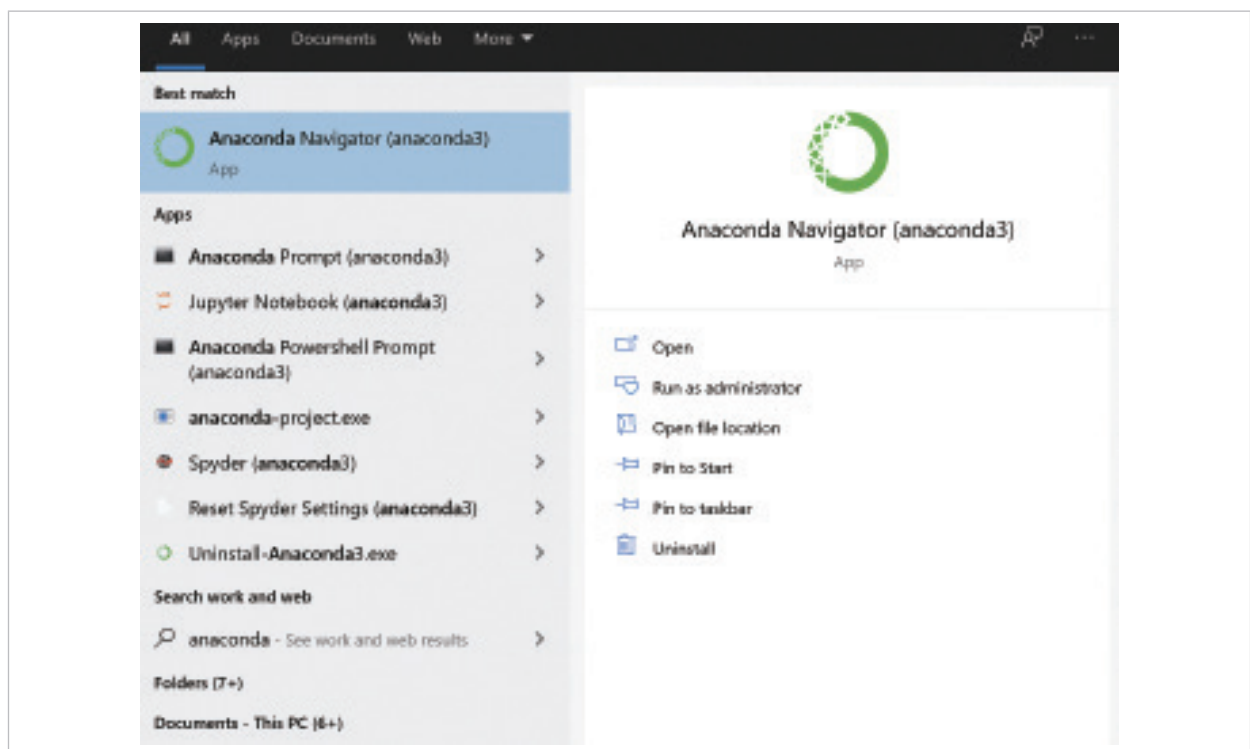
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



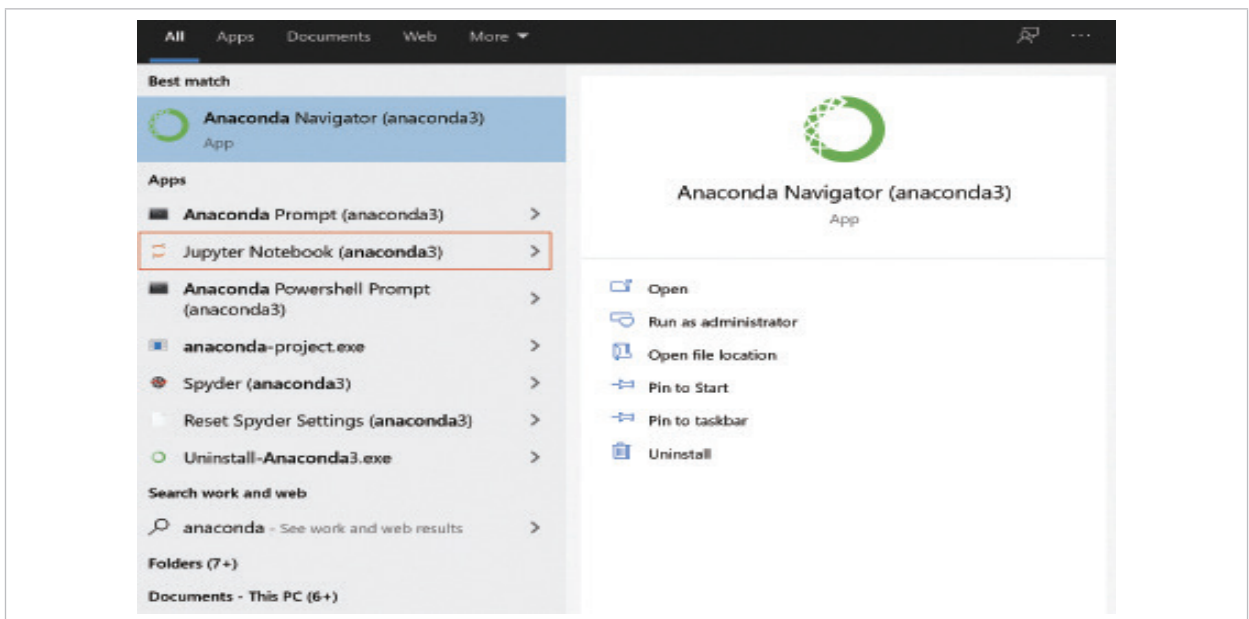
⑪ 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치 되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.



3. 주피터 노트북 (Jupyter Notebook) 실행

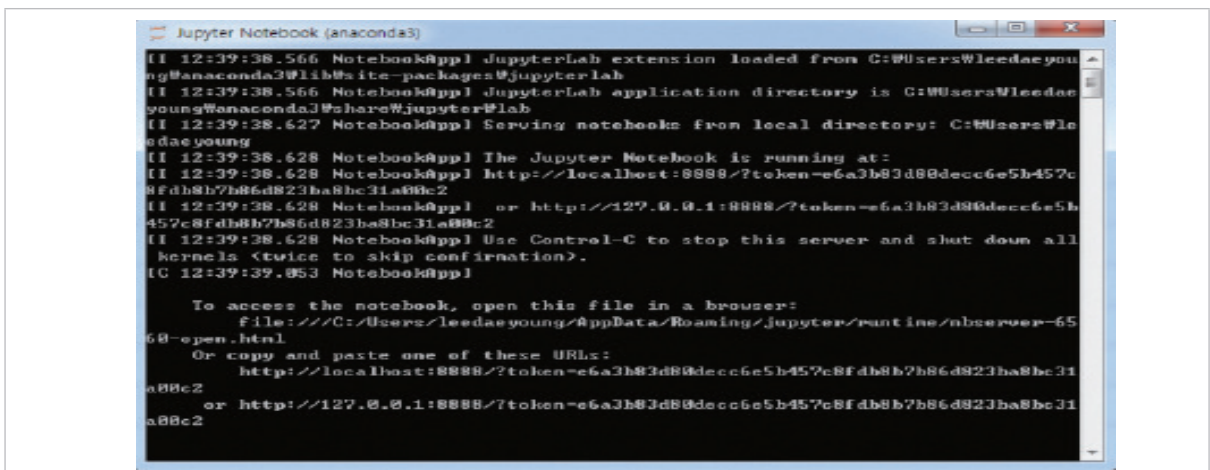
주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드 북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다. Anaconda를 설치한 이후 주피터 노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

- ① 화면상의 '홈(🏠)'키를 눌러서 화면과 같이 'anaconda' 검색 및 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.

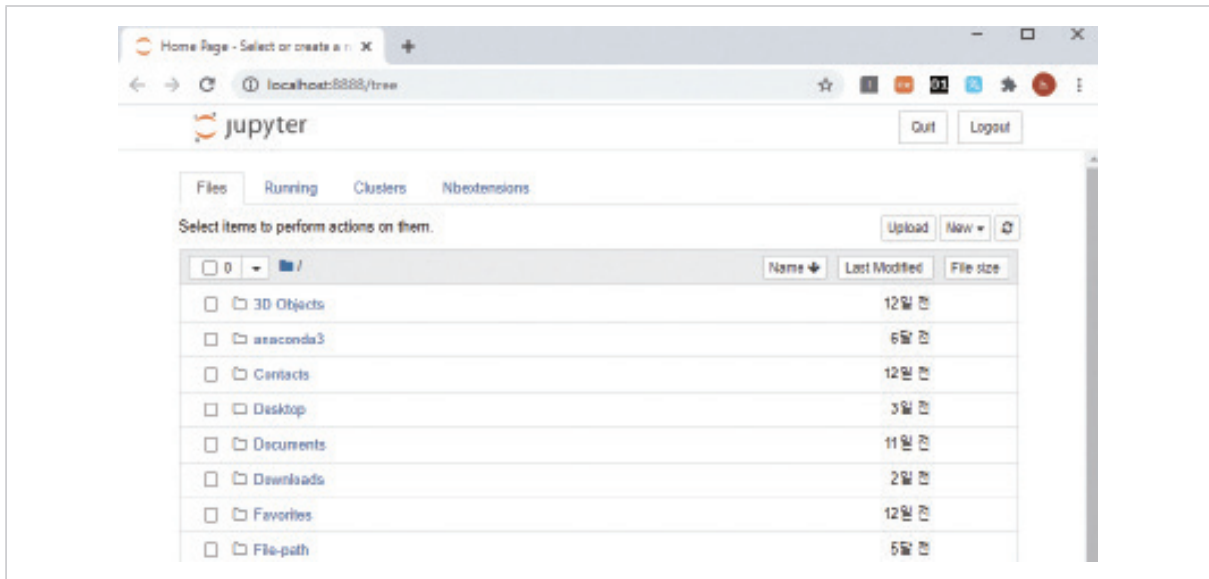


- ② Jupyter Notebook을 클릭시 아래처럼 2개의 윈도우가 실행된다.

- (1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내주는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안된다.



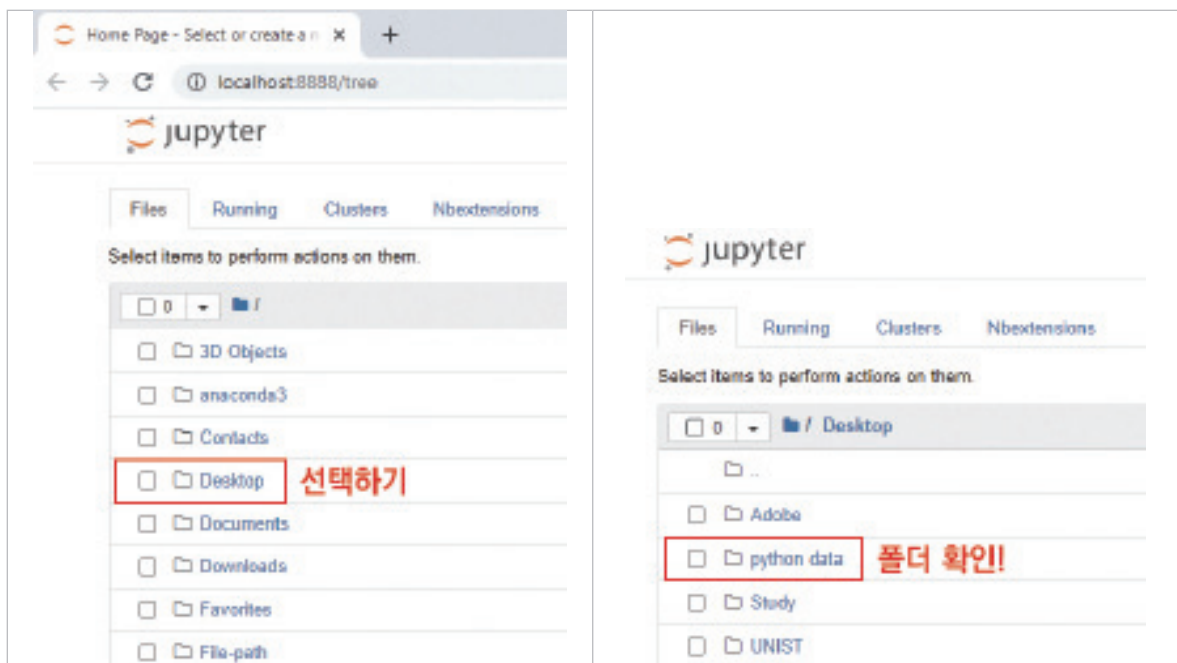
(2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야한다)



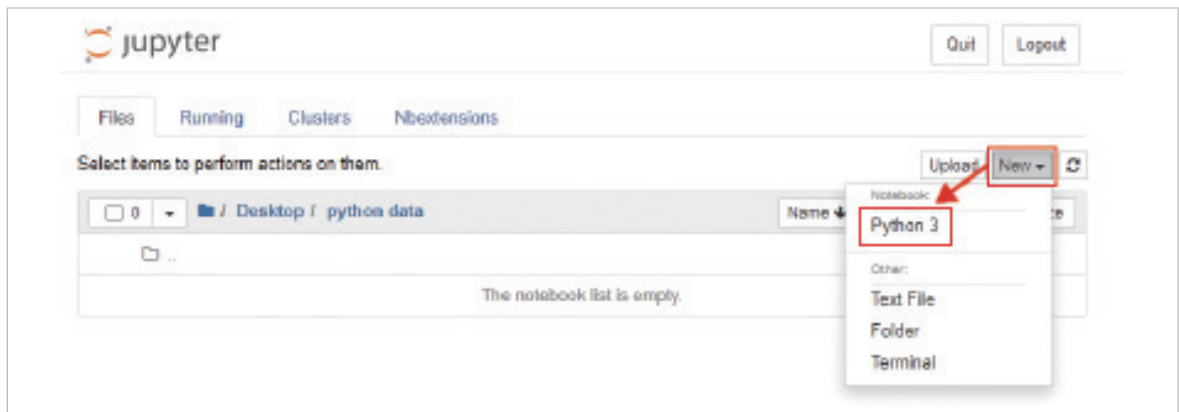
③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

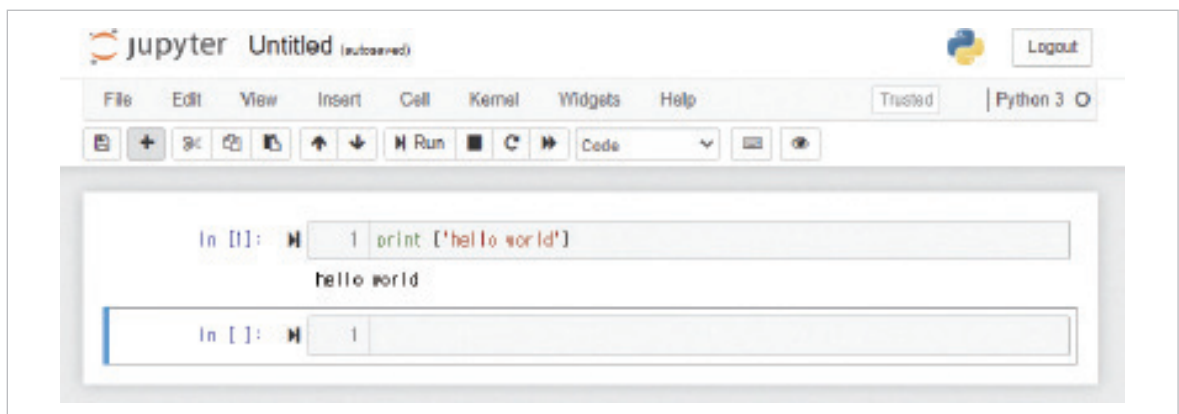
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



- (2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



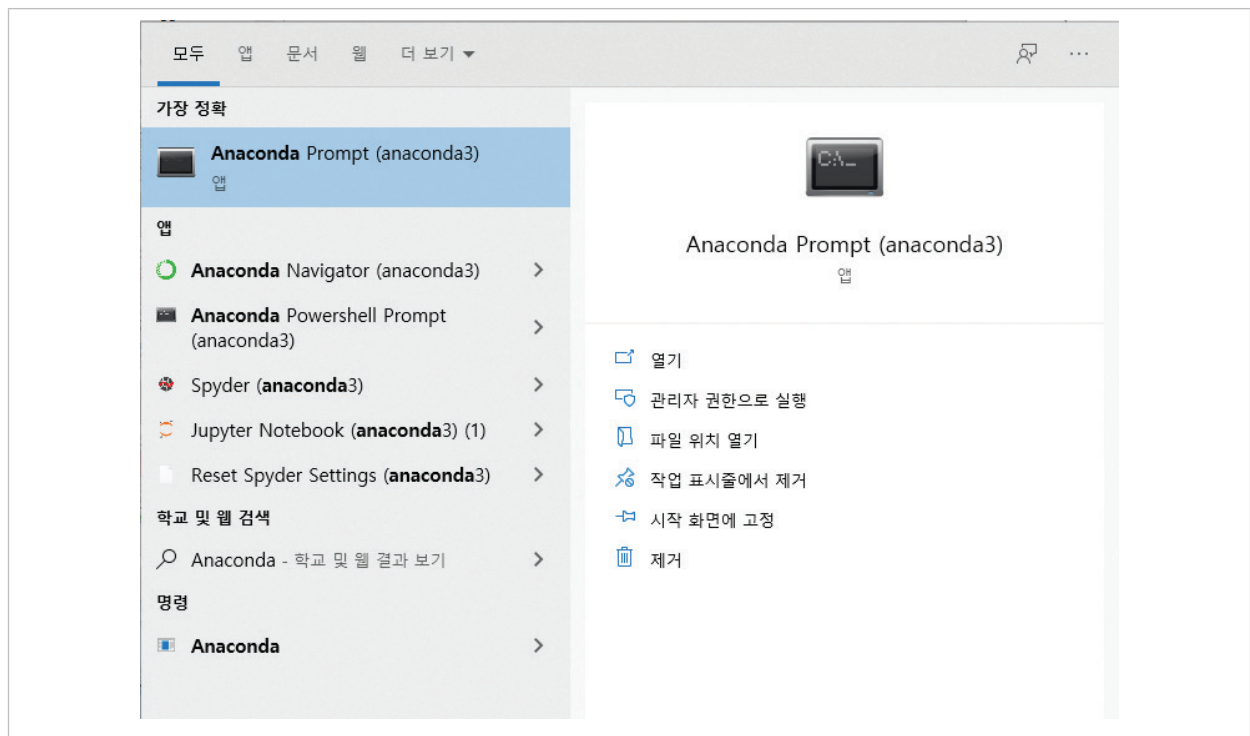
- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world') 입력 후, 상단의 Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



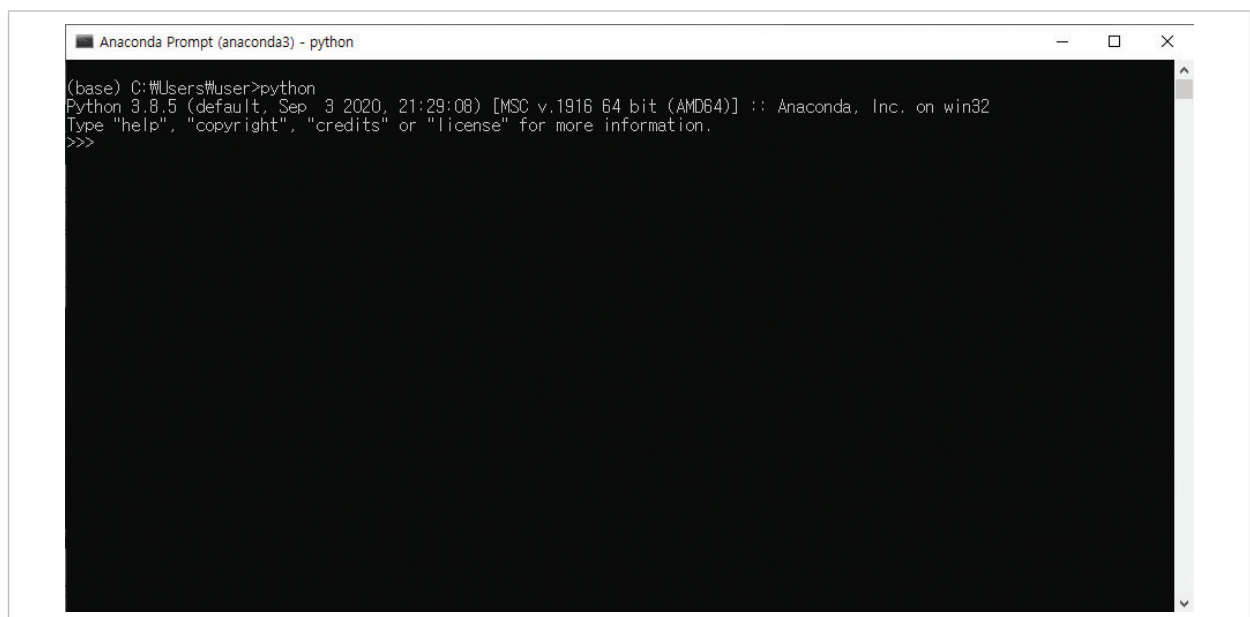
4. 아나콘다 가상환경 설정

가상환경 : 독립적인 작업환경에서 패키지 및 버전 관리를 하기 위한 가상의 환경을 의미한다. 각 패키지(모듈, 라이브러리)는 버전에 따라 의존성을 갖고 있으므로 오류가 발생할 수 있으며, 이를 관리하기 위해 가상환경을 사용하여 가상환경마다 다른 패키지 버전을 사용할 수 있다.

① 'Anaconda Prompt' 실행

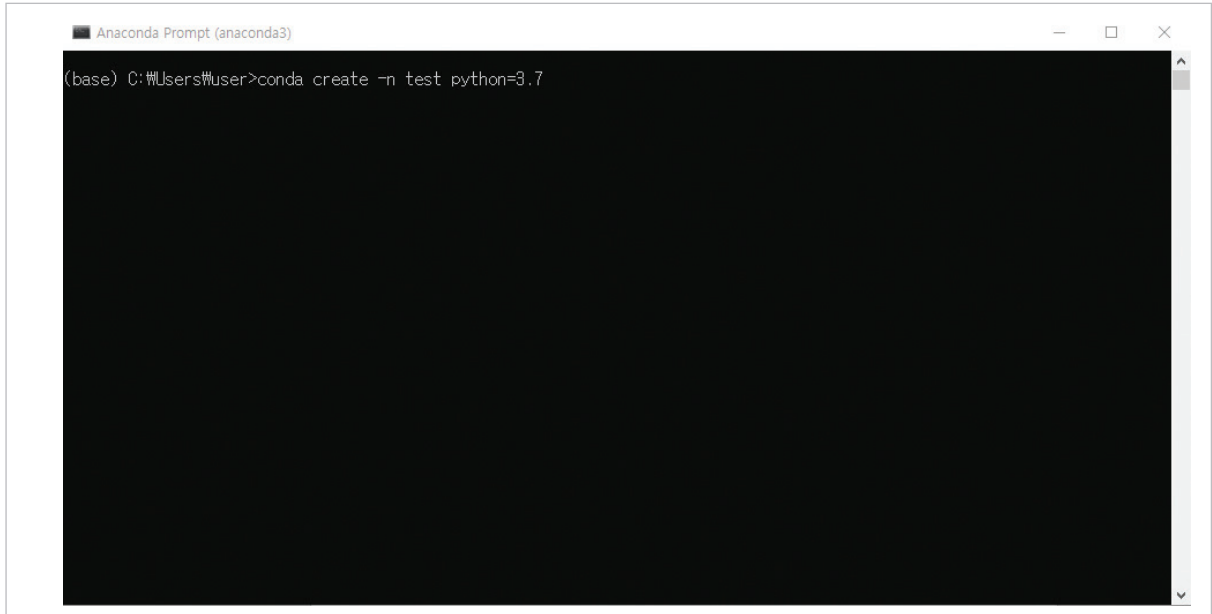


* 'Anaconda Prompt'는 실습을 진행하는 동안 종료시키면 안 되며, 켜둔 상태로 진행



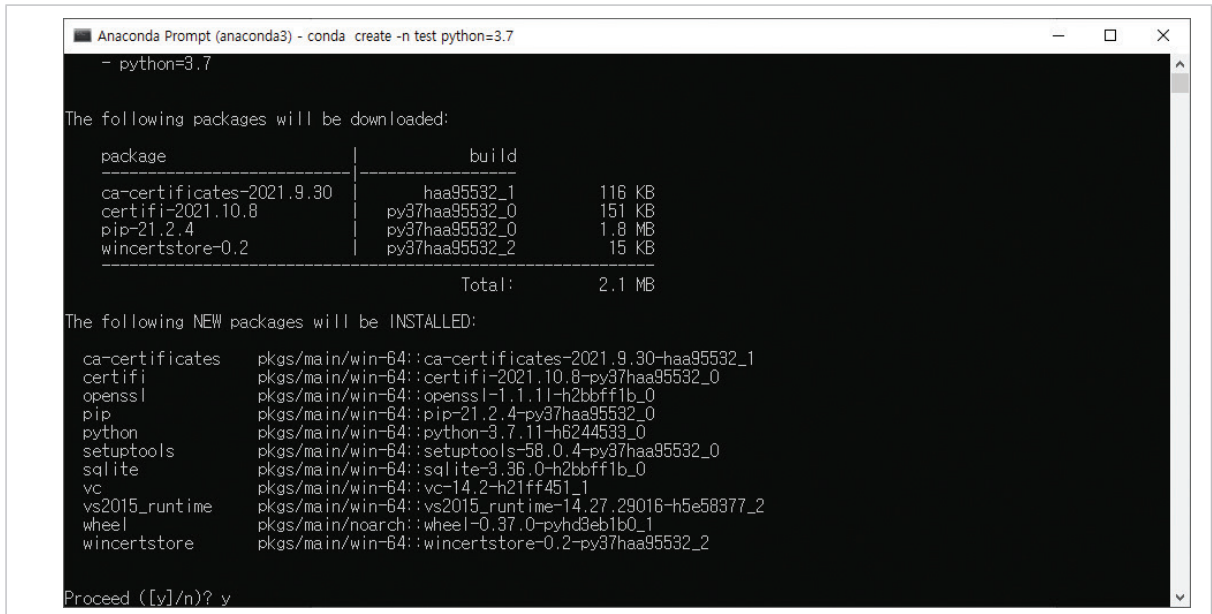
② 가상환경 생성

1) 'conda create -n test python=3.7' 입력 후 엔터



```
Anaconda Prompt (anaconda3)
(base) C:\Users\User>conda create -n test python=3.7
```

2) 'y' 입력 후 엔터



```
Anaconda Prompt (anaconda3) - conda create -n test python=3.7
- python=3.7

The following packages will be downloaded:

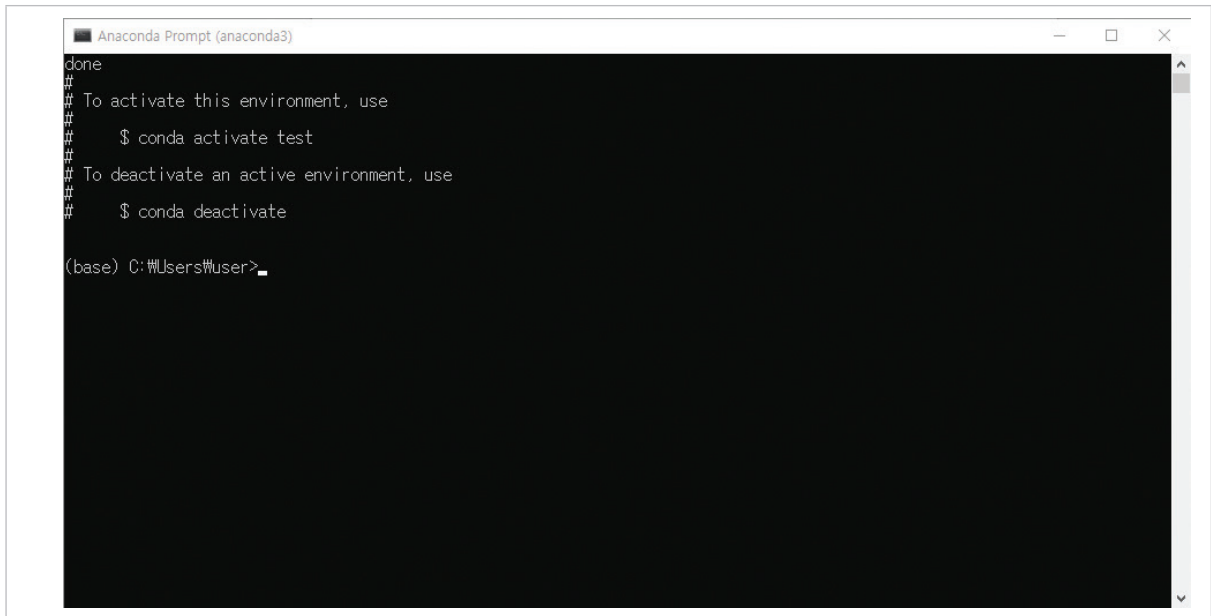
package | build | size
-----|-----|-----
ca-certificates-2021.9.30 | haa95532_1 | 116 KB
certifi-2021.10.8 | py37haa95532_0 | 151 KB
pip-21.2.4 | py37haa95532_0 | 1.8 MB
winertstore-0.2 | py37haa95532_2 | 15 KB
Total: 2.1 MB

The following NEW packages will be INSTALLED:

ca-certificates pkgs/main/win-64::ca-certificates-2021.9.30-haa95532_1
certifi pkgs/main/win-64::certifi-2021.10.8-py37haa95532_0
openssl pkgs/main/win-64::openssl-1.1.1-h2bbff1b_0
pip pkgs/main/win-64::pip-21.2.4-py37haa95532_0
python pkgs/main/win-64::python-3.7.11-h6244533_0
setuptools pkgs/main/win-64::setuptools-58.0.4-py37haa95532_0
sqlite pkgs/main/win-64::sqlite-3.36.0-h2bbff1b_0
vc pkgs/main/win-64::vc-14.2-h21ff451_1
vs2015_runtime pkgs/main/win-64::vs2015_runtime-14.27.29016-h5e58377_2
wheel pkgs/main/noarch::wheel-0.37.0-pyhd3eb1b0_1
winertstore pkgs/main/win-64::winertstore-0.2-py37haa95532_2

Proceed ([y]/n)? y
```

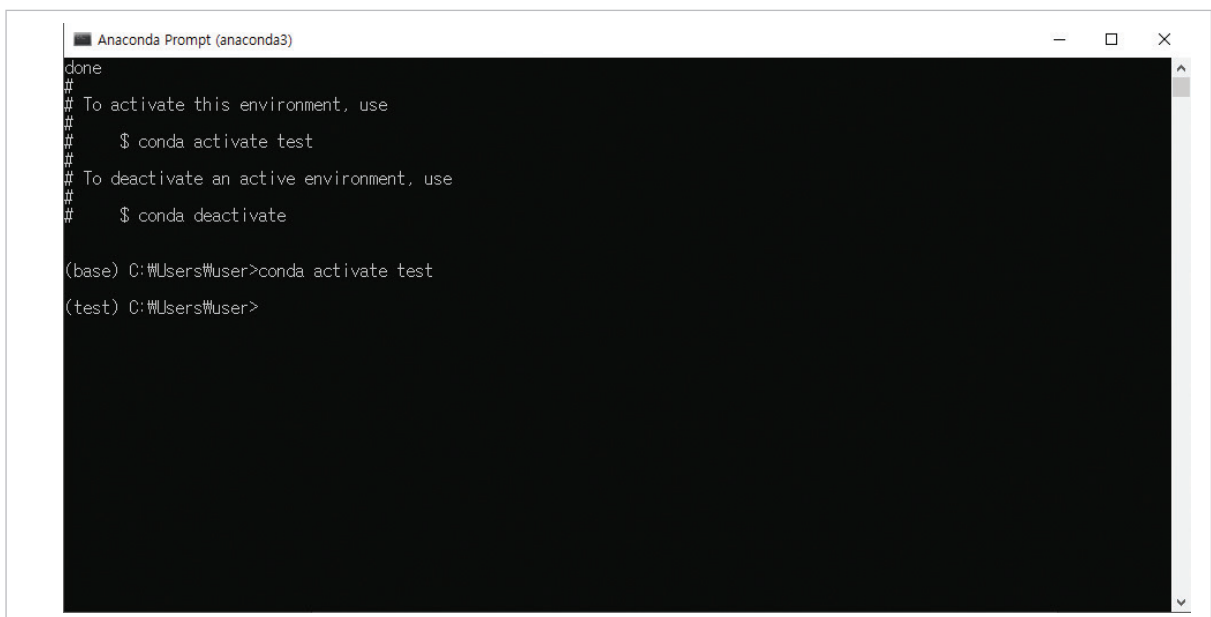
3) 가상환경 생성 완료



The screenshot shows the Anaconda Prompt window with the following text:

```
done
#
# To activate this environment, use
#
#     $ conda activate test
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\User>
```

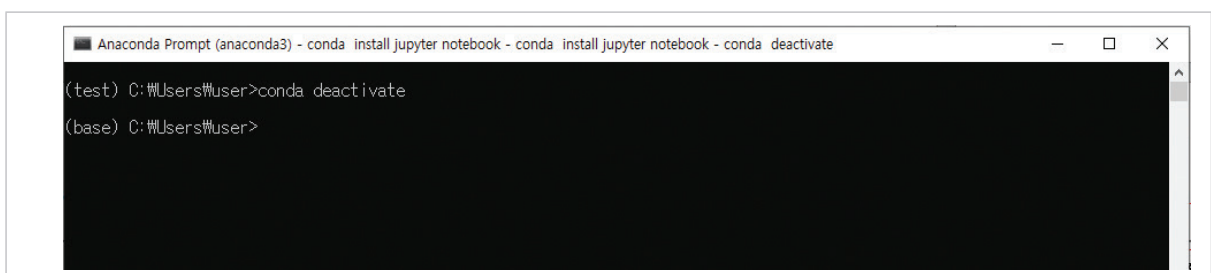
4) 가상환경 진입 - 'conda activate test' 입력 후 엔터



The screenshot shows the Anaconda Prompt window with the following text:

```
done
#
# To activate this environment, use
#
#     $ conda activate test
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\User>conda activate test
(test) C:\Users\User>
```

5) 가상환경 종료 - 'conda deactivate' 입력 후 엔터



The screenshot shows the Anaconda Prompt window with the following text:

```
Anaconda Prompt (anaconda3) - conda install jupyter notebook - conda install jupyter notebook - conda deactivate
(test) C:\Users\User>conda deactivate
(base) C:\Users\User>
```

5. 주피터 노트북(Jupyter Notebook) 설정

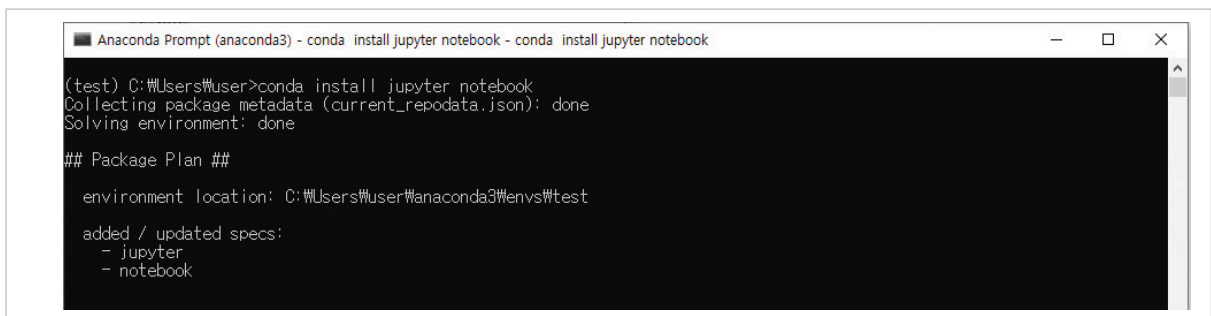
주피터 노트북(Jupyter Notebook) : 파이썬 코딩을 할 수 있는 대화형 인터프리터 (Interpreter)로 웹 브라우저 환경에서 파이썬 코드를 작성 및 실행할 수 있는 툴이다.

① 가상환경 진입

‘conda activate test’ 입력 후 엔터

② 주피터 노트북 설치

‘conda install jupyter notebook’ 입력 후 엔터, 중간에 ‘y’ 입력 후 엔터



```
Anaconda Prompt (anaconda3) - conda install jupyter notebook - conda install jupyter notebook
(test) C:\Users\User>conda install jupyter notebook
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

environment location: C:\Users\User\anaconda3\envs\test

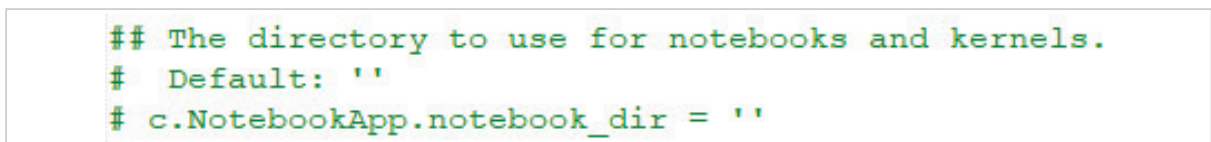
added / updated specs:
- jupyter
- notebook
```

③ 주피터 노트북 테스트

‘jupyter notebook’ 입력 후 엔터

④ 주피터 노트북 시작 폴더 설정

- 1) ‘jupyter notebook --generate-config’ 입력 후 엔터
- 2) 입력 후 아래에 출력되는 경로에 진입하여 ‘jupyter_notebook_config.py’ 파일을 메모장으로 열기
- 3) 메모장에서 ‘#c.NotebookApp.notebook_dir=’ 문장 찾기
- 4) ‘#c.NotebookApp.notebook_dir=’를 찾아서 ‘c.NotebookApp.notebook_dir=folder_directory’로 변경



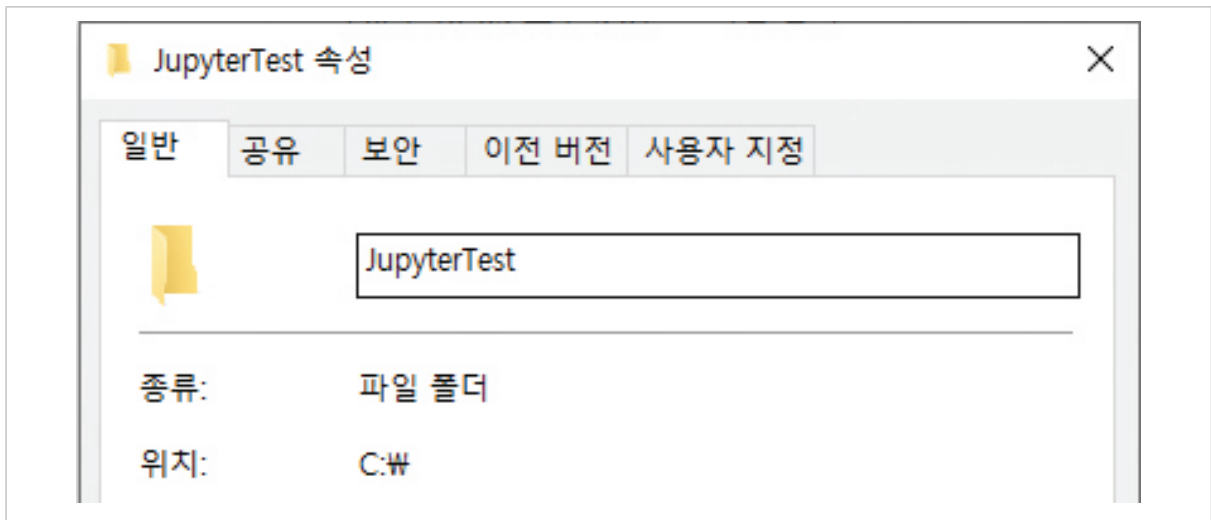
```
## The directory to use for notebooks and kernels.
# Default: ''
# c.NotebookApp.notebook_dir = ''
```

[그림 37] 기존 가상환경 Base Directory 코드

```
## The directory to use for notebooks and kernels.
# Default: ''
c.NotebookApp.notebook_dir = 'C:\JupyterTest'
```

[그림 38] 변경하는 가상환경 Directory 코드

- * folder_directory는 주피터 노트북을 시작하고 저장하는 기본 폴더로 해당 위치에 모든 작업물이 저장된다. 시작 전 folder_directory를 해당 위치에 생성시켜줘야 한다.
- * folder_directory 예시 : 'C:\JupyterTest' (folder_directory는 작은따옴표까지 포함되어야 한다.)



5) 변경 완료 후 저장 및 메모장 종료

⑤ 주피터 노트북 커널 추가

- 1) 'pip install ipykernel' 입력 후 엔터
- 2) 'python -m ipykernel install --user --name test --display-name "test"' 입력 후 엔터

※ 아나콘다를 이용하는 이유

데이터 분석이나 데이터 모델링을 하다 보면 패키지나 라이브러리(library)에 따라 같은 Python을 쓰더라도 3.6을 써야 할 때도 있고 3.7을 써야 할 때도 있다. 이렇게 되면 다른 패키지에서 버전 충돌이 발생하게 된다.

이를 방지하기 위해 버전의 호환성을 위해 가상환경 별로 독립적인 라이브러리(library)의 버전 관리가 가능하기 때문에 아나콘다를 주로 많이 사용한다. 그렇다면 왜 다른 버전 쓰는 것인지 간략하게 설명한다.

주관적으로 보았을 때 라이브러리의 버전의 안전성이 가장 큰 원인이라고 생각된다.

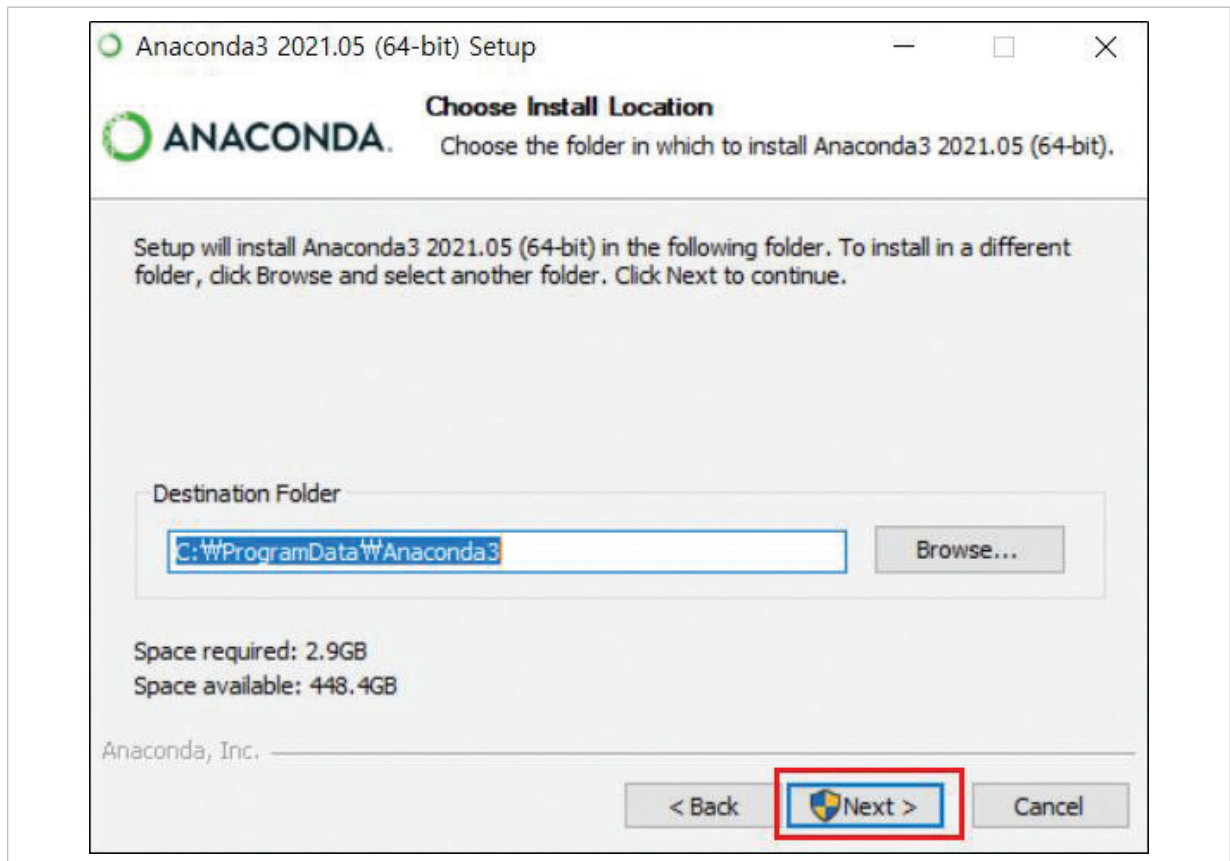
보통은 케라스(Keras)나, 텐서플로(TensorFlow)가 허용하는 버전들이 있다. 그리고 안정적이라고 판단하는 버전들이 대부분 하위 버전인 경우가 많다. 텐서플로(TensorFlow) 같은 경우 1버전과 2버전의 코드까지도 다르다. 코드의 알고리즘적인 부분이나 사용되는 원리나 구조는 같지만 코드가 다르다. 그런 경우 버전 1에서는 버전 2가 돌아가지 않는 상황이 발생한다. 그러므로 가상환경으로 각각의 텐서플로(TensorFlow) 버전을 분리해두고 상황에 따라 버전 1과 버전 2를 불러와서 사용한다.

※ 안되는 이유

설치 혹은 실행시 알 수 없는 에러(검색 해도 없는)가 발생하는 경우는 아래와 같이 예측해 볼 수 있다.

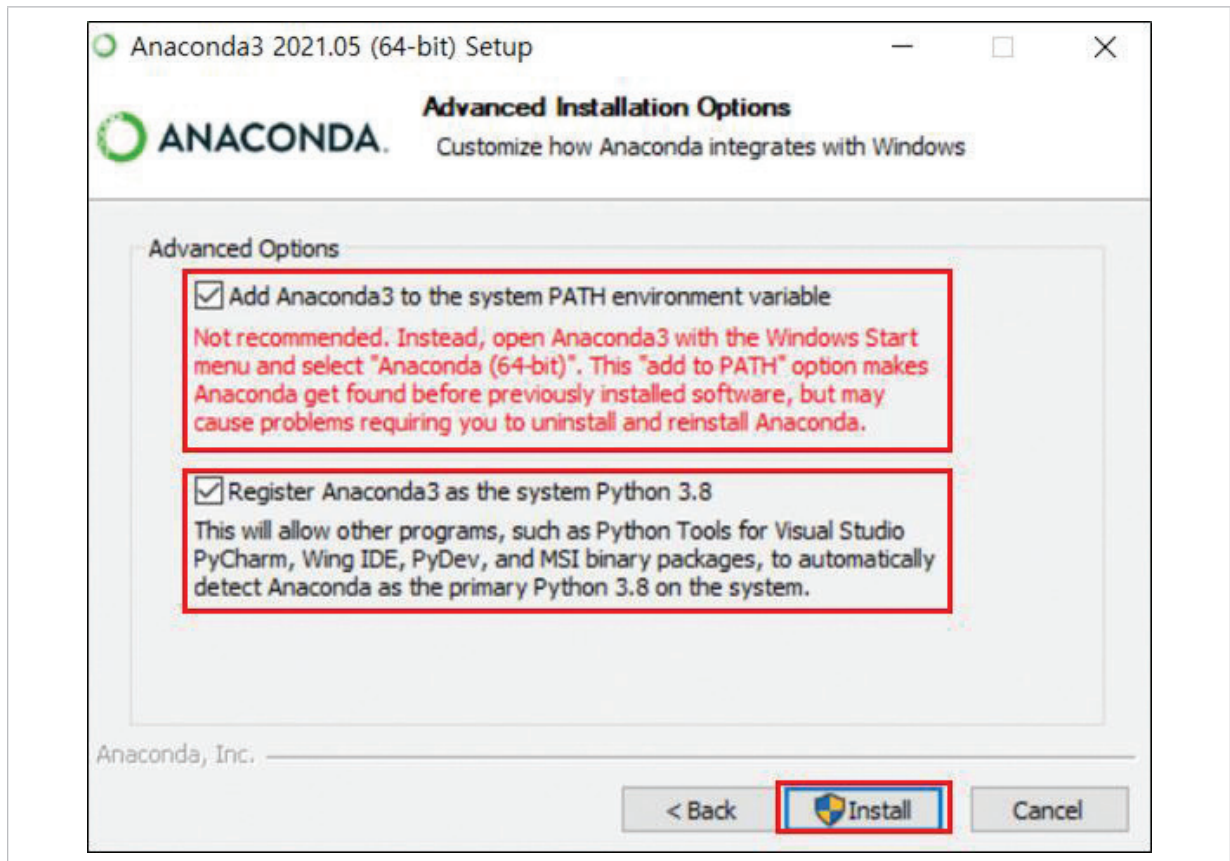
• 아나콘다 패스 설정 오류

- 아래의 경로를 최대한 변경하지 않는 것이 좋다. 모든 모듈은 저 베이스를 가본으로 하므로 특이한 사항이 아니라면 저 경로 그대로 진행하도록 한다.



만일 경로를 다르게 지정했다면 보통은 잘 진행된다. 하지만 특이 케이스에는 모듈을 불러올 때 에러가 발생할 수 있다는점 인지하고 설치하기 바란다. 이 이슈가 발생한다면 아나콘다를 삭제하고 재설치 작업을 진행하기를 추천한다.

- Add Anaconda3 to the system PATH environment variable 미 체크 경우



- 패키지끼리의 버전 충돌 혹은 호환되지 않는 각각의 패키지

- 케라스(Keras)라던지, 텐서플로(TensorFlow)처럼 타 패키지에 영향을 많이 받는 모듈은 최 대한 가상환경을 생성하고 base보다는 가상환경을 생성해서 설치하는 것이 좋다.

이유는 아래와 같다.

- conda의 경우는 아나콘다에서 지원하는 혹은 사용 가능한 패키지만 관리한다.

- pip 같은 경우는 python에서 사용 가능하다고 한 것만 관리하는 패키지이다.

여기서 주의할 점은 간혹 같은 패키지이지만 설치 방법이 다른 경우가 있고 호환이 안 될 수도 있다는 점 이다.

부득이하게 conda install 안될 때가 있다. 이렇게 설치가 안 된다면 pip로 시도해보거나 그래도 동작하지 않는다면

아래와 같이 진행하면 된다.

1. 콘솔 창을 연다.
2. 아나콘다 경로 아래에 bin이라는 폴더 경로까지 이동한후
3. 'pip install 설치 패키지'로 설치하면 동작하는 것을 확인 할 수 있다. 코드 1과 같이 설치하면 된다.

```
C:\Users\User>cd %  
C:\>cd C:\Users\User\anaconda3\bin pip install pandas
```

[코드 1] 콘솔 창에서 pip로 패키지 설치

4 부록 _ 데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다. (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

- 데이터 품질 지수 : 세부 설명

▶ 완전성 품질 지수 = $((1 - \text{결측치}) / \text{전체 데이터 수}) * 100$

- 다루는 Numerical 데이터는 dedicated_data, 한 가지임.

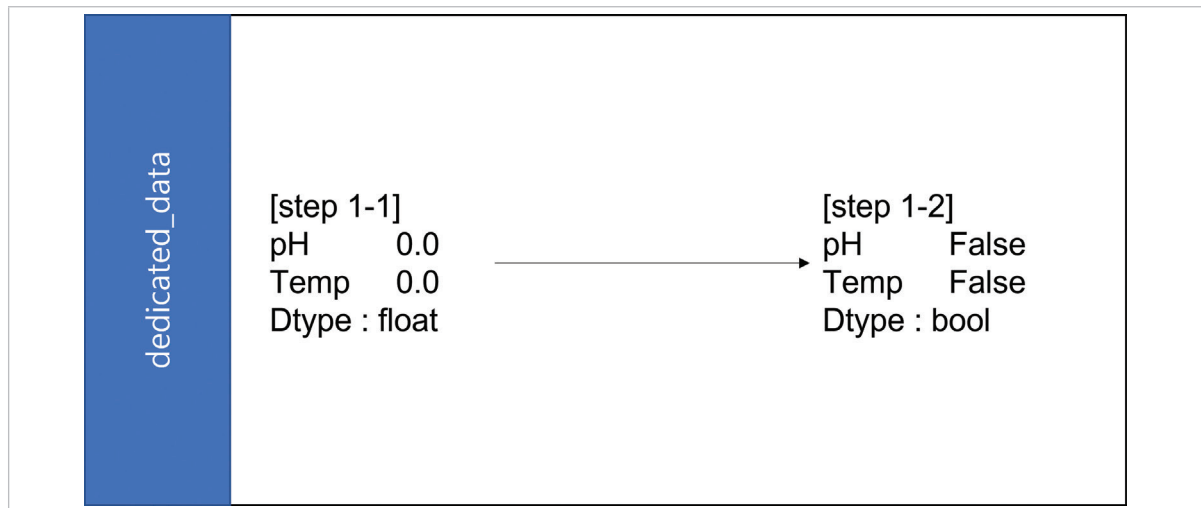
```
perc = 30
data_columns = ['pH', 'Temp']
for df_name in list(data_columns):
    print(f'DataFrame name: {df_name}')
    df = dedicated_data[df_name]
    print([step 1-1])
    print(round(df.isnull().sum()/len(df)*100,2))
    print([step 1-2])
    print(df.isnull().sum()/len(df)*100>perc)
    print([step 2-1])
    print(df.isnull().head())
    print([step 2-2])
    print(df.isnull().sum())
    print([step 2-3])
    cmpt_len = df.isnull().sum().sum()
    print(cmpt_len)
    print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
    print("\n\n")
```

[코드 34] 완전성 품질지수 전체 코드

① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 Null 값의 비율을 확인하여 삭제한다. 이 데이터는 30%를 넘는 열이 존재하지 않으므로 열을 제거하지 않는다.

```
print([step 1-1])
print(round(df.isnull().sum()/len(df)*100,2))
```

[코드 35] 완전성 품질지수 코드 (1)

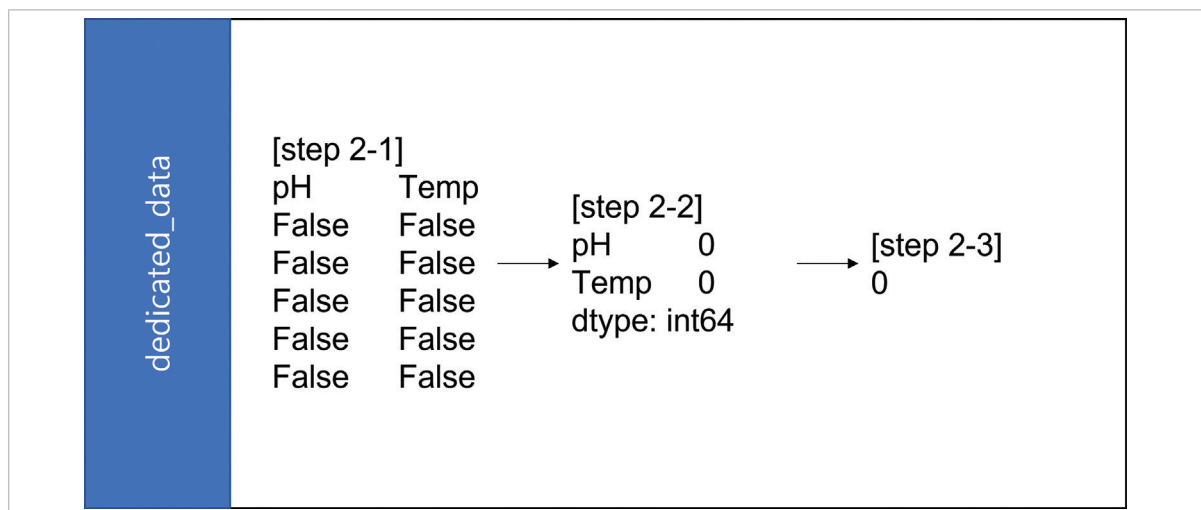


[그림 45] 분석 도구를 활용한 완전성 품질 지수 확인 (1)

- ② 데이터의 결측치를 확인하기 위하여 `isnull()` 함수를 사용한 뒤, `sum()` 함수를 이용하여 총 결측치 개수를 구한다.

```
print([step 2-1])
print(df.isnull().head())
```

[코드 36] 완전성 품질지수 코드 (2)



[그림 47] 분석 도구를 활용한 완전성 품질지수 확인 (2)

- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

```
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
#결과는 아래에서 확인 가능하다.
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 37] 분석 도구를 활용한 완전성 품질 지수 구하기

▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

- dedicated_data 데이터의 경우 [pH, Temp] 열이 유일한 값을 가지는 기본 key이다.

```
check_unique = dedicated_data[
    ['pH', 'Temp']
].value_counts().reset_index()
perc_check_unique_item_urgent_info = round(
    (len(check_unique) - len(check_unique[check_unique[0] > 1]))
    / len(check_unique) * 100, 2)

print(f'The percentage of uniqueness for pH(->Temp : {perc_check_unique_item_urgent_info})')
print("유일성 지수 : %.2f%%" % (perc_check_unique_item_urgent_info))
```

The percentage of uniqueness for pH(->Temp : 37.55
유일성 지수 : 37.55%

[코드 38] 유일성 품질 지수 코드

① value_counts() 함수를 이용하여 key 값들 개수를 확인한다. 개수가 큰 순서로 기본 정렬된다. reset_index() 함수는 데이터 프레임 인덱스를 재부여한다.

② key 값들 개수로 정렬했을 때, 1 이상인 행 개수 비율의 평균이 유일성 품질 지수다.

```
perc_check_unique_item_urgent_info = round(
    (len(check_unique) - len(check_unique[check_unique[0] > 1]))
    / len(check_unique) * 100, 2)

print(f'The percentage of uniqueness for pH(->Temp : {perc_check_unique_item_urgent_info})')
print("유일성 지수 : %.2f%%" % (perc_check_unique_item_urgent_info))
```

The percentage of uniqueness for pH(->Temp : 37.55
유일성 지수 : 37.55%

[코드 39] 분석 도구를 활용한 유일성 품질 지수 구하기

▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수)*100

- 유효성 지수를 만족하는 데이터만 vald_df에 저장, 최종적으로 완성된 데이터를 이용하여 유효성 품질 지수를 구한다.

```

df = dedicated_data
pH_lb = df['pH']>=1
pH_ub = df['pH']<=4
temp_lb = df['Temp']>=35
temp_ub = df['Temp']<=65
vald_df = df[pH_lb & pH_ub & temp_lb & temp_ub]
print(f'[Step 1] 데이터 범위를 벗어난 데이터 수: {len(df)-len(vald_df)}')
d0 = '2021-09-06'
d1 = '2021-10-27'
vald_df = vald_df.loc[d0:d1]
print(f'[Step 2] 수집된 날짜를 벗어나는 데이터 수: {len(df)-len(vald_df)}')
vald_df['DTime'] = vald_df['DTime'].apply(lambda x: isinstance(x, datetime.datetime))
vald_df['pH'] = vald_df['pH'].apply(lambda x: isinstance(x, float))
vald_df['Temp'] = vald_df['Temp'].apply(lambda x: isinstance(x, float))
print(f'[Step 3] 데이터 형식을 벗어나는 데이터 수: {len(df)-len(vald_df)}')
vald_df[
    (vald_df['pH']==True)
    & (vald_df['Temp']==True)
]
vald_len = len(vald_df)
item_vald = vald_len/len(df)*100
print("유효성 지수 : %.2f%%"%(item_vald))

[Step 1] 데이터 범위를 벗어난 데이터 수: 0
[Step 2] 수집된 날짜를 벗어나는 데이터 수: 0
[Step 3] 데이터 형식을 벗어나는 데이터 수: 0
유효성 지수 : 100.00%

```

[코드 40] 유효성 품질지수 코드

① 데이터가 유효범위 내에 들어가 있는가?

- 데이터셋에 정의된 수집 범위를 이용하여 조건(유효범위)을 모두 충족하는 데이터들을 vald_df에 저장한다.

```

df = dedicated_data
pH_lb = df['pH']>=1
pH_ub = df['pH']<=4
temp_lb = df['Temp']>=35
temp_ub = df['Temp']<=65
vald_df = df[pH_lb & pH_ub & temp_lb & temp_ub]
print(f'[Step 1] 데이터 범위를 벗어난 데이터 수: {len(df)-len(vald_df)}')

[Step 1] 데이터 범위를 벗어난 데이터 수: 0

```

[코드 41] 유효성 품질지수 확인 코드 (1)

② 수집된 날짜 안에 들어가 있는가?

- 데이터셋 중 날짜 데이터를 포함하는 데이터셋 만 확인한다.
- 2021.02.01 ~ 2021.10.27에 수집된 데이터이기 때문에, 데이터의 날짜가 이 기간을 벗어나지 않았는지 확인한다.

```

d0 = '2021-09-06'
d1 = '2021-10-27'
vald_df = vald_df.loc[d0:d1]
print(f'[Step 2] 수집된 날짜를 벗어나는 데이터 수: {len(df)-len(vald_df)}')

[Step 2] 수집된 날짜를 벗어나는 데이터 수: 0

```

[코드 42] 유효성 품질 지수 확인 코드 (2)

③ 데이터가 형식에 맞는가?

- 데이터 열은 다음 형식이어야 한다.

dedicated_data	
DTime	datetime
pH	float
Temp	float

- 'isinstance()' 함수를 사용하여 데이터 형식을 만족하는지 확인한다. 다음은 dedicated_data에 대한 예시이다.

```
vald_df['DTime']=vald_df['DTime'].apply(lambda x:isinstance(x,datetime.datetime))
vald_df['pH']=vald_df['pH'].apply(lambda x:isinstance(x,float))
vald_df['Temp']=vald_df['Temp'].apply(lambda x:isinstance(x,float))
print(f'[Step 3] 데이터 형식을 벗어나는 데이터 수: {len(df)-len(vald_df)}')
[Step 3] 데이터 형식을 벗어나는 데이터 수: 0
```

[코드 43] 유효성 품질 지수 확인 코드 (3)

④ 유효성 품질지수 값은 다음과 같다.

```
vald_df[
    (vald_df['pH']==True)
    &(vald_df['Temp']==True)
]
vald_len=len(vald_df)
item_vald=vald_len/len(df)*100
print("유효성 지수 : %.2f%%"%(item_vald))
order_info 유효성 지수 : 100.00%
```

[코드 44] 유효성 품질 지수 확인 코드 (4)

⑤ 최종 유효성 품질 지수는 모든 데이터 유효성 품질지수의 평균이다.

```
print("유효성 지수 : %.2f%%"%(item_vald))
유효성 지수 : 100.00%
```

[코드 45] 유효성 품질 지수 확인하기

▶ 일관성 품질 지수 = (일관성 만족 데이터수/전체 데이터 수)*100

- 모든 열값은 독립적이므로 정확성 지수를 확인하지 않음

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수))*100

- 모든 열값은 독립적이므로 정확성 지수를 확인하지 않음

▶ 무결성 품질지수 = (1-(유일성, 유효성, 일관성 지수중 100%가 아닌 지수 개수 / 3))*100

- dedicated_data 데이터는 세 가지 지수중 유일성 지수가 100%를 만족하지 못하므로 (37.55%) 무결성 품질 지수는 50%이다.

▶ 가중치 지수 = 품질 지수 * 가중치

▶ 데이터 품질 지수 = $\sum$ 가중치 지수

구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	60%	60%	0%
유일성 (중복)	37.55%	10%	3.755%	62.45%
유효성 (유효)	100%	20%	20%	0%
일관성 (표현)	-		-	-
정확성 (실제)	-	-	-	-
무결성	50%	10%	5%	50%
품질지수		100%	88.755%	11.245%

「공정운영 최적화 AI 데이터셋」 분석실습 가이드북

